

Université Sultan Moulay Slimane
Faculté Polydisciplinaire - Béni Mellal
« Département de Physique »



Support de Cours

Electronique Numérique

Sciences de la Matière Physique (LEF)
ISE-IEM-S4

Version 1.0

Auteur :
Pr. Ahmed GAGA

Année Universitaire : 2025/2026

Table des matières

1	Rappel sur l'électronique Analogique	5
1.1	Introduction	5
1.2	Les composants électroniques linéaires	6
1.2.1	La Résistance	6
1.2.2	Le Condensateur	10
1.2.3	La Bobine	12
1.3	Les composants électroniques Non-linéaires	15
1.3.1	La Diode	15
1.3.2	Le Transistor	17
1.3.3	L'amplificateur opérationnel (AOP)	23
2	Système de Numération	27
2.1	Introduction	27
2.2	Base d'un système de numération	27
2.2.1	Système Décimal	28
2.2.2	Système Binaire	28
2.2.3	Système Octal	29
2.2.4	Système Hexadécimal	30
2.3	Changement de base	31
2.3.1	Conversion Décimal en autres bases	31
2.3.2	Conversion des autres bases (2, 8 et 16) en Décimal	32
3	Le Codage	39
3.1	Code binaire naturel	39
3.2	Code binaire réfléchi (Code Gray)	40
3.2.1	Construction de la table par symétrie	41
3.3	Code BCD	42
3.4	Le codage ASCII	43
3.4.1	Principe du codage ASCII	43
3.4.2	Limitation du codage ASCII	44
3.5	Le codage ASCII étendu	44

3.6	Le codage UNICODE	45
3.7	Le codage 7-segments	46
4	Opérations Arithmétiques dans le Système Binaire	50
4.1	Représentation des nombres	50
4.1.1	Nombres signés / non signés	50
4.1.2	Complément à un	53
4.1.3	Complément à deux	53
4.1.4	Nombre à virgule fixe	54
4.1.5	Nombre a virgule flottante	55
4.2	Opérations arithmétiques	58
4.2.1	Addition binaire	58
4.2.2	Soustraction binaire	61
4.2.3	Multiplication binaire	61
4.2.4	Division binaire	61
5	Logique combinatoire	62
5.1	Opérations logiques de base	62
5.1.1	Opérateur OUI	62
5.1.2	Opérateur ET	63
5.1.3	Opérateur OU	63
5.1.4	Opérateur NON	64
5.1.5	Combinaisons de fonctions logiques	64
5.2	Autres Opérations logiques	65
5.2.1	Opérateur NON-ET (NAND)	65
5.2.2	Opérateur NON-OU (NOR)	65
5.2.3	Opérateur OU EXCLUSIF (XOR)	66
5.2.4	Symboles des Opérateurs Logiques	66
5.3	Théorèmes fondamentaux	67
5.3.1	Écriture algébrique	67
5.3.2	Table de vérité	68
5.3.3	Lois fondamentales	68
5.3.4	Théorème de Morgan	70
5.4	Les circuits logiques combinatoires	70
5.4.1	Le Multiplexeur	70
5.4.2	Le Démultiplexeur	71
5.4.3	Les Encodeurs prioritaires	72
5.4.4	Le Décodeur	73
5.4.5	Le Transcodeur	76
5.4.6	L'Additionneur	77

5.4.7	Le Soustracteur	81
5.4.8	Le soustracteur binaire n-bits	84
5.4.9	Le Comparateur	85
5.4.10	L'UAL	88
5.5	Formes canoniques	91
5.5.1	Utilisation d'opérateurs NAND	91
5.5.2	Utilisation d'opérateurs NOR	92
6	Table de KARNAUGH	94
6.1	Introduction	94
6.2	Méthode de recherche de l'équation	94
6.2.1	Méthode de la Somme des Produits	94
6.2.2	Méthode du Produit des Sommes	95
6.3	Avantages et utilisation	95
6.4	Exemples	95
7	Logique Séquentielle	96
7.1	Introduction	96
7.2	Le Signal d'horloge	96
7.3	Les Bascules	97
7.3.1	Bascule RS	97
7.3.2	Bascule D	102
7.3.3	Bascule JK	105
7.3.4	Bascule T	108
7.4	Les Registres	108
7.4.1	Introduction	108
7.4.2	Caractéristiques	109
7.4.3	Registre de Mémorisation	109
7.4.4	Registre à Décalage	110
7.5	Les Compteurs	115
7.5.1	Compteurs Asynchrones	115
7.5.2	Décompteurs Asynchrones	117
7.5.3	Limitations du comptage Asynchrones	118
7.5.4	Compteurs Synchrones	118
8	Applications	120
8.1	Génération du signal d'horloge (CLK) à base d'un circuit NE555	120
8.2	Conception et simulation d'un décodeur BCD/7-segment	120
8.3	Etude et Simulation d'une unité Arithmétique et logique (UAL 74LS181)	120
8.4	Conception, simulation et réalisation d'une Horloge Numérique	120

8.5	Étude et conception d'un dé électronique à base d'un compteur de Johnson	120
8.6	Étude et conception d'une PLL Numérique (<i>Phase Locked Loop</i>)	120

Chapitre 1

Rappel sur l'électronique Analogique

1.1 Introduction

L'électronique analogique est la discipline qui traite des systèmes électroniques sur des grandeurs (*tension, courant, charge*) à variation continue dans le temps. Elle diffère de l'électronique numérique dans laquelle ces dernières sont quantifiées. On emploie le terme « **Analogique** » car les grandeurs électriques utilisées sont à l'image du signal à traiter (*Analogues*).

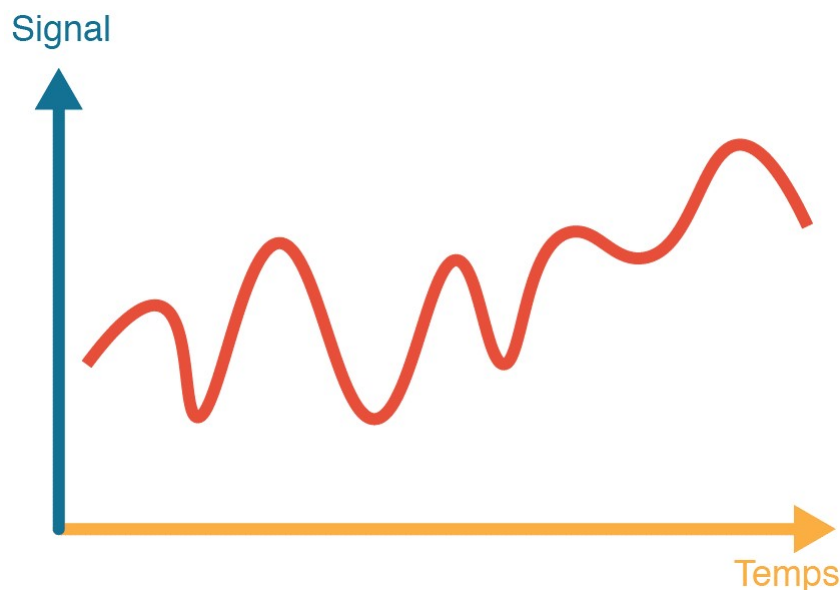


FIGURE 1.1 – Signale analogique

L'information n'est pas transcrite de la même façon dans les circuits analogiques et numériques. Les systèmes numériques utilisent la quantification et un codage de l'information tandis que les systèmes analogiques travaillent sur des valeurs à variation continue dont la richesse du contenu n'est pas limitée par un échantillonnage quelconque.

Historiquement, les premiers systèmes électroniques étaient de type analogique. Ceux-ci avaient en effet une structure plus simple. Une même fonction était réalisée avec moins de composants en analogique qu'en numérique.

Les progrès de l'intégration et l'essor de la micro-électronique ont favorisé le développement de l'électronique numérique. La plupart des systèmes électroniques actuels intègrent un mélange des systèmes numériques et analogiques. Si la part de la partie analogique se réduit au profit du numérique, l'électronique analogique reste cependant incontournable dans un certain nombre d'applications, notamment pour les capteurs et les transducteurs.

Le principal intérêt de l'électronique numérique est sa simplicité de fonctionnement qui rend son comportement très prédictible. Les règles de quantification et le synchronisme (*dans les circuits synchrones*) permettent de construire aisément des systèmes complexes et fiables. L'intégration a rendu cela possible et peu coûteux.

1.2 Les composants électroniques linéaires

1.2.1 La Résistance

En électricité, le terme résistance désigne :

- **Une propriété physique** : l'aptitude d'un matériau conducteur à s'opposer au passage d'un courant électrique sous une tension électrique donnée.
- **Un composant électrique** : figurant parmi les dipôles électriques, conçu pour approcher de manière très satisfaisante la loi d'Ohm dans une large plage d'utilisation.
- **Un modèle mathématique** : pour les éléments réels qui respectent idéalement la loi d'Ohm.



FIGURE 1.2 – Image réelle d'une résistance électrique $\frac{1}{4}$ Watts

Propriété physique

La résistance électrique traduit la propriété d'un composant à s'opposer au passage d'un courant électrique (*l'une des causes de perte en ligne d'électricité*). Elle est souvent désignée par la lettre R et son unité de mesure est l'ohm (*symbole* : Ω). Elle est liée aux

notions de résistivité et de conductivité électrique.

La résistance est responsable d'une dissipation d'énergie sous forme de chaleur. Cette propriété porte le nom de l'**effet de Joule**. Cette production de chaleur est parfois, représente un effet souhaité (*résistances de chauffage par exemple*), parfois un effet néfaste (pertes Joule) mais souvent inévitable.

Un des problèmes majeurs pour les ingénieurs est que la **conductivité**, et son inverse, la **résistivité**, dépendent fortement de la température. Lorsqu'un dipôle est traversé par un courant électrique, sa résistance provoque un échauffement qui modifie sa température, laquelle modifie sa résistance. La résistance d'un dipôle dépend donc fortement des conditions d'utilisation.

La résistance a ceci de particulier que c'est une des rares caractéristiques physiques dont la plage de valeurs peut aller de 0 (*supraconducteurs*) à l' ∞ (*isolants parfaits*), même dans la pratique.

Comportement électrocinétique

Lorsque l'on soumet une différence de potentiel U continue à un objet (*exprimée en volts, V*), on provoque une circulation de charges électriques quantifiées par l'intensité du courant I (*exprimée en ampères, A*). Si cette intensité n'est pas nulle, la résistance R est alors le rapport entre la différence de potentiel et l'intensité (*voir équation 5.3*) :

$$R = \frac{U}{I} \quad (1.1)$$

La résistance est exprimée en *ohms*, Ω . Si les grandeurs ne sont pas continues, on peut toutefois appliquer cette loi en considérant les **valeurs efficaces**. La loi d'Ohm postule que cette résistance est une caractéristique de l'objet et est indépendante de la différence de potentiel appliquée et de l'intensité du courant, ce qui n'est vrai que dans certains cas.

Puissance consommée

La puissance consommée par un conducteur ohmique de résistance R , qui constitue l'effet Joule, peut se calculer à partir de la loi de puissance électrique (*voir équation 1.2*) :

$$P = U \times I \quad (1.2)$$

Et de la loi d'Ohm : $U = R \times I$, On a donc deux cas :

- soit on connaît U , la valeur efficace de la tension effectivement appliquée aux bornes du dipôle (*cette dernière peut être différente de la tension délivrée par le*

générateur), on a alors :

$$P = \frac{U^2}{R} \quad (1.3)$$

- Soit, plus rarement, on connaît I , la valeur efficace de l'intensité du courant qui traverse effectivement le dipôle, on a alors :

$$P = R \times I^2 \quad (1.4)$$

Résistance équivalente

Les lois dites d'associations de résistances ne s'appliquent en toute rigueur qu'à des conducteurs ohmiques :

- Association des Résistances en série (*voir la figure 1.3*) :

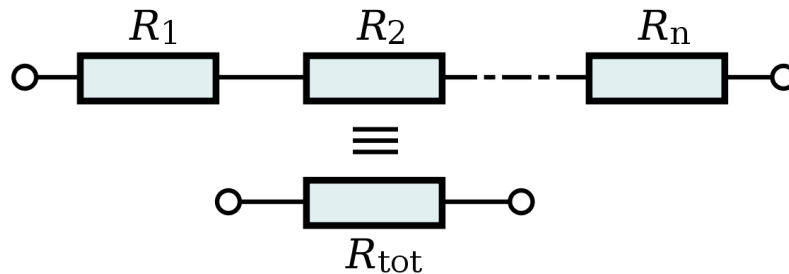


FIGURE 1.3 – Association de 'n' Résistances en série

$$R_{tot} = \sum_{i=1}^n R_i \quad (1.5)$$

- Association des Résistances en parallèle (*voir la figure 1.4*) : Le code de couleurs

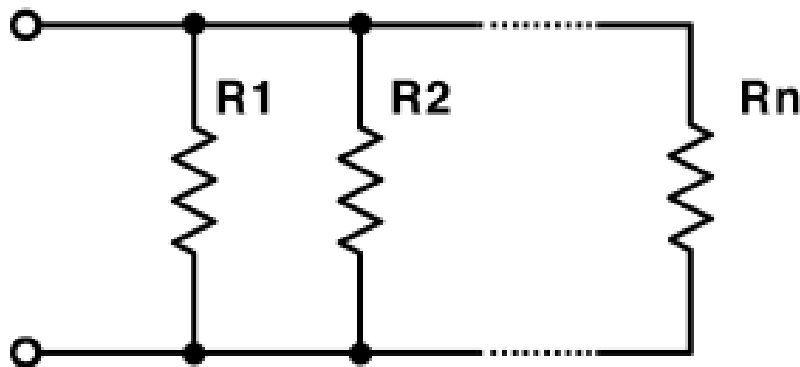


FIGURE 1.4 – Association de 'n' Résistances en parallèle

$$R_{tot} = \frac{1}{\sum_{i=1}^n \frac{1}{R_i}} \quad (1.6)$$

La valeur des résistances peut être indiquée sur le composant sous forme de trois à six anneaux de couleurs.

1. Le premier est celui qui est placé le plus près d'une extrémité de la résistance (*Le composant*).
 2. Les deux premiers (*ou les trois premiers*) indiquent deux chiffres (*un chiffre correspondant à une couleur*).
 3. L'avant-dernier (*troisième, parfois quatrième*) indique un facteur de multiplication du nombre formé par les deux premiers.
 4. Le suivant (*quatrième, parfois cinquième*) indique la tolérance du fabrication de 20% à 0,05%. Cet anneau est parfois d'avantage espacé des précédents. Il n'est pas toujours présent, son absence signifiant la tolérance la plus lâche qui est de 20%.
- Les résistances de précision peuvent porter un sixième anneau, indiquant la classe du coefficient thermique de la résistance en *ppm par kelvin*. Par exemple, une résistance avec une classe du coefficient thermique de *5ppm/K* signifie que la résistance changera de 5 parties par million par 1 Kelvin.
 - L'utilisation des composants à une valeur précise n'est pas toujours aussi nécessaire, tandis que le prix de revient, comptant la précision des machines, le contrôle, le stockage de nombreuses valeurs différentes, dépend de la tolérance sur cette valeur. En fonction des applications, on utilisera une série ayant des caractéristiques de précision différentes. La tolérance sur les condensateurs est généralement de 20%, et ceux-ci n'ont que trois marques de couleur.
 - Les couleurs employées pour le marquage de valeurs des résistances, rangées par ordre croissant des valeurs en partant de zéro sont données par le tableau 1.1 :

TABLE 1.1 – Le code de couleurs

Couleurs	Noir	Marron	Rouge	Orange	Jaune
Codes	0	1	2	3	4
Couleurs	Vert	Bleu	Violet	Gris	Blanc
Codes	5	6	7	8	9

Les valeurs sont normalement prises, pour chaque tolérance, dans une liste de valeurs normalisées. De ce fait, comme seules certaines séquences sont valides, le codage des couleurs comporte une redondance qui limite les erreurs de lecture.

Le marquage est habituel sur les résistances cylindrique de petite taille (*voir la figure*

1.5), mais les résistances de puissance, notamment celles munies d'un radiateur et les composants montés en surface ont un marquage directement en chiffres.

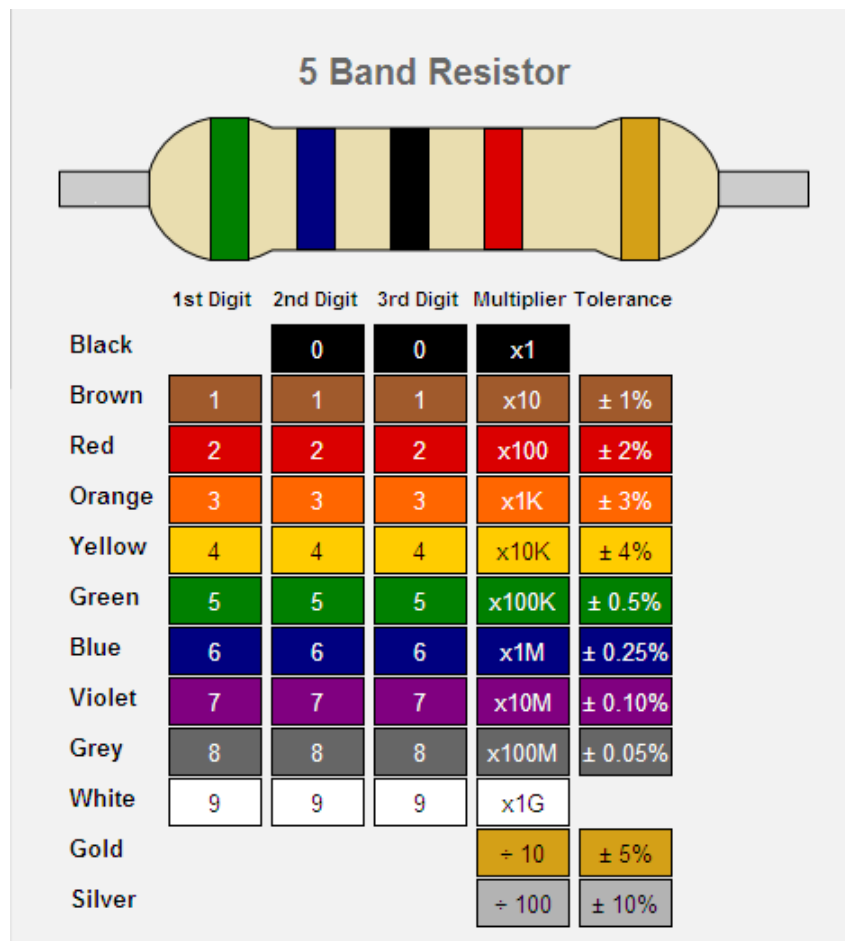


FIGURE 1.5 – Code de couleurs des résistances cylindriques

1.2.2 Le Condensateur

Le condensateur (*figure 1.6*) est un composant électronique élémentaire, constitué de deux armatures conductrices (appelées « **électrodes** ») en influence totale (*l'un est à l'intérieur de l'autre*) et séparées par un isolant polarisable (ou « **diélectrique** »). Sa propriété principale est de pouvoir stocker des charges électriques opposées sur ses armatures. La valeur absolue de ces charges est proportionnelle à la valeur absolue de la tension qui lui est appliquée.

Le condensateur est utilisé principalement pour :

- Stabiliser une alimentation électrique (*il se décharge lors des chutes de tension et se charge lors des pics de tension*) ;
- Traiter des signaux périodiques (*filtrage par exemple*) ;

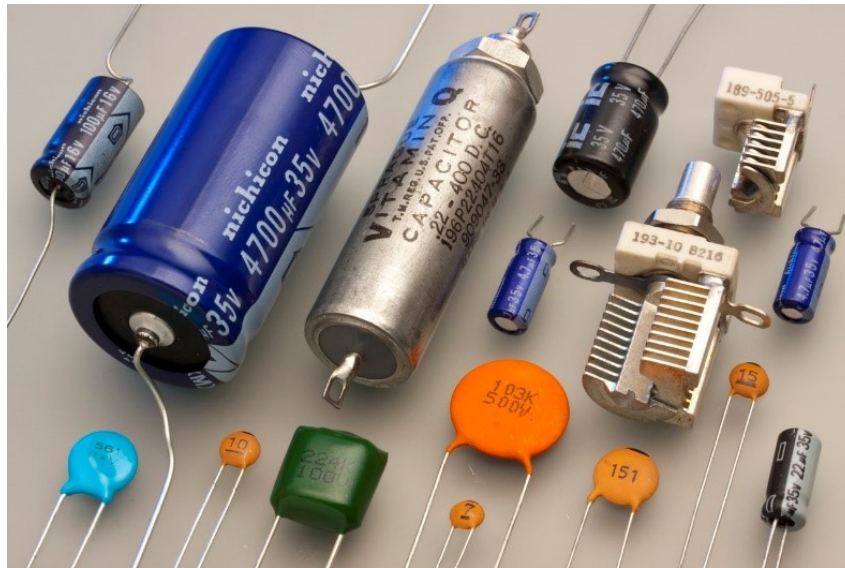


FIGURE 1.6 – Condensateurs réels

- Séparer le courant alternatif du courant continu, ce dernier étant bloqué par le condensateur ;
- Stocker de l'énergie, auquel cas on parle de *super-condensateur*.

Il existe deux catégories principales de condensateurs : les condensateurs **non polarisés** et les condensateurs **polarisés**. Les condensateurs non polarisés n'ont pas de sens de branchement, alors que les condensateurs polarisés doivent être branchés dans un sens et surtout pas dans l'autre, car ces derniers possèdent une borne "+" et une borne "-". Comme il est important que les condensateurs polarisés soient branchés dans le bon sens, les personnes qui dessinent les schémas électroniques utilisent un symbole qui permet de reconnaître le pôle + du pôle -. Le pôle positif peut être reconnu grâce à la présence d'un petit signe Plus. Quand il n'y a pas de signe Plus et que les deux traits représentant le condensateur sont différents, le pôle Moins est désigné par un trait courbé, par un rectangle plein ou par un rectangle avec des hachures comme le montre la figure 1.7 .

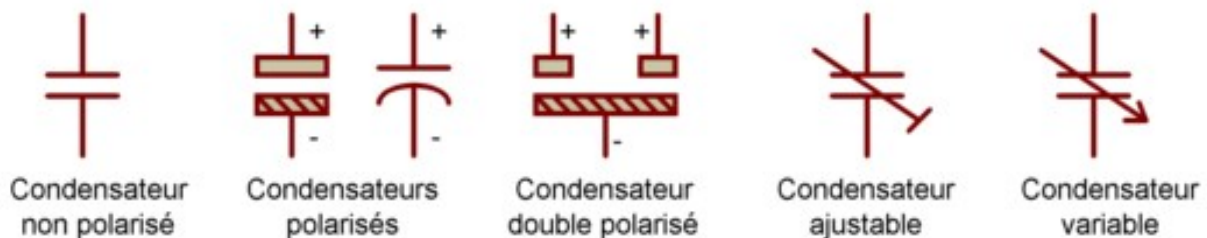


FIGURE 1.7 – Symbole des Condensateurs polarisés et non polarisés

Propriétés

Le condensateur est caractérisé par le coefficient de proportionnalité entre charge et tension, appelé capacité électrique et exprimé en farads (F). La relation caractéristique

d'un condensateur idéal est donnée par l'équation (1.7) :

$$i(t) = C \times \frac{du(t)}{dt} \quad (1.7)$$

Où :

- $i(t)$ désigne l'intensité du courant électrique qui passe par le composant, exprimée en ampères (symbole : A) ;
- $u(t)$ la tension aux bornes du composant, exprimée en volts (V) ;
- C la capacité électrique du condensateur, exprimée en farads (F) ;
- $\frac{du(t)}{dt}$ la dérivée de la tension par rapport au temps (V/s).

Les signes sont tels que l'électrode par laquelle entre le courant (*dans le sens conventionnel du courant : + vers -*) voit son potentiel augmenter. On définit la capacité par la relation suivante :

$$Q = C \times U \quad (1.8)$$

Où Q est la charge stockée sur sa borne positive (*qui s'exprime en coulombs*).

Association des condensateurs

les deux figures 1.8 et 1.9 représentent respectivement le principe d'association série (1.9) et parallèle (1.10) des condensateurs avec la formule de la capacité totale équivalente.

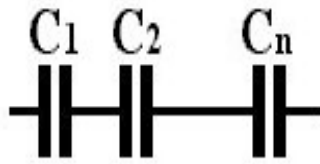


FIGURE 1.8 – Association série

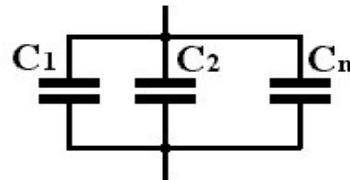


FIGURE 1.9 – Association parallèle

$$\frac{1}{C_{eq}} = \frac{1}{C_1} + \frac{1}{C_2} + \dots + \frac{1}{C_n} \quad (1.9)$$

$$C_{eq} = C_1 + C_2 + \dots + C_n \quad (1.10)$$

1.2.3 La Bobine

Une bobine, solénoïde, auto-inductance ou quelquefois self, est un composant courant en électrotechnique et électronique. Une bobine est constituée d'un enroulement de fil conducteur éventuellement autour d'un noyau en matériau ferromagnétique qui peut être un assemblage de feuilles de tôle ou un bloc de ferrite. Les physiciens et ingénieurs l'appellent souvent « **inductance** », ce terme désignant la propriété caractéristique de la bobine, qui est son opposition à la variation du courant dans ses spires.

les deux figures 1.10 et 1.11 représentent respectivement quelque types des bobines réelles existants dans le marché et le symbole générique d'une bobine.

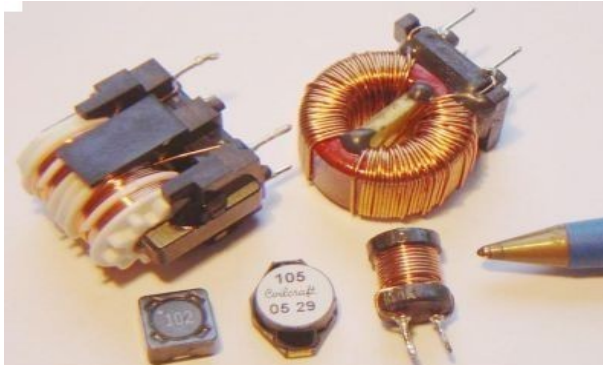


FIGURE 1.10 – Bobines réelles

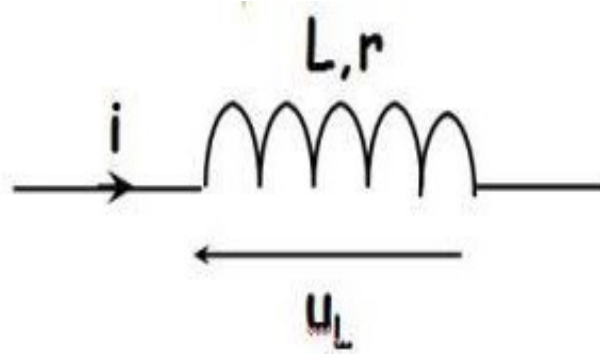


FIGURE 1.11 – Symbole générique

Description

La partie la plus visible est un enroulement de fils conducteurs. L'espace au milieu de ces spires s'appelle le noyau. Il peut être vide ou inclure une pièce en matériau ferromagnétique favorisant l'induction électromagnétique, afin d'augmenter la valeur de l'inductance. Le noyau peut être un circuit magnétique complètement ou partiellement fermé pour améliorer la linéarité de l'inductance. Le circuit magnétique d'une bobine avec noyau peut être « saturé » si l'on essaye d'induire un flux supérieur à la valeur limite acceptable par le noyau ; à ce moment, la valeur de l'inductance de la bobine s'effondre. Pour augmenter la réluctance de la bobine et retarder la saturation, on peut aménager une ouverture, appelée entrefer, dans le noyau. Un entrefer est indispensable au fonctionnement des têtes de lecture/écriture des appareils tels que : magnétophone à bande magnétique, disque dur d'ordinateurs, etc. Pour raisonner sur les circuits électroniques et calculer les valeurs nécessaires, on considère des objets idéaux, qui n'ont que les caractéristiques nécessaires au rôle que l'on veut leur faire jouer. Une bobine est considérée, dans ce cadre, comme un dipôle présentant une inductance pure. Si les autres caractéristiques, comme la résistance du fil de la bobine ou la capacité entre spires ne sont pas négligeables, on les représente sous la forme d'autres composants, non moins idéaux, séparés.

Perte dans une bobine réelle

Une bobine ne présente jamais une inductance propre pure. Les pertes peuvent provenir de plusieurs causes :

- Résistance ohmique du fil enroulé autour du noyau, accrue du fait de l'effet de peau dans le bobinage à partir de quelques centaines de kHz ;
- Pertes par hystérésis proportionnelles à la fréquence du courant qui traverse la bobine ;

- Perte par courants de Foucault proportionnelles au carré de la fréquence du courant qui traverse la bobine.

De plus, les capacités entre spires ne sont pas négligeables à haute fréquence.

Modèle réel d'une bobine

Les modèles les plus simples et les plus fréquemment utilisés sont ceux correspondant à l'association d'une bobine d'inductance et d'une résistance comme le montre la figure 1.12 :

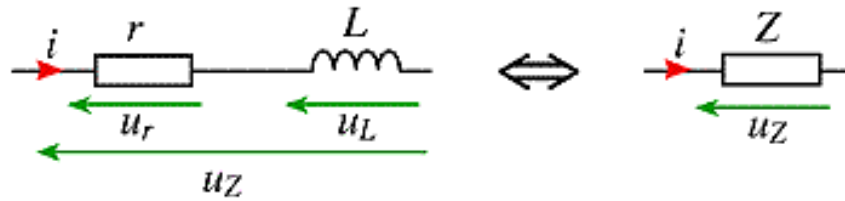


FIGURE 1.12 – Modèle équivalent d'une bobine réelle

La tension U_L aux bornes de la bobine et l'intensité i_L du courant sont reliés par l'équation différentielle 1.11 :

$$U_L = L_s \cdot \frac{di_L}{dt} + r_s \cdot i_L \quad (1.11)$$

où :

- **L** : est l'inductance de la bobine
- **r** : sa résistance propre (*dans le cas d'une bobine parfaite, $r = 0$*).

Association des Bobines

l'association série (équation 1.12) et parallèle (équation 1.13) des bobines sont illustrées respectivement dans les figures 1.13 et 1.14.

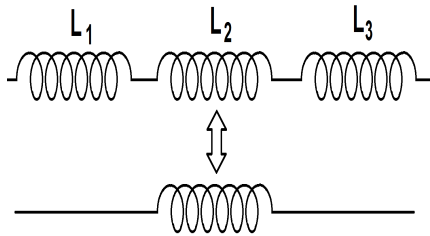


FIGURE 1.13 – Association série

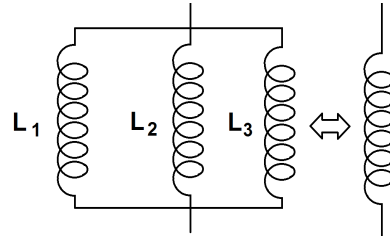


FIGURE 1.14 – Association parallèle

$$L_{eq} = L_1 + L_2 + L_3 \quad (1.12)$$

$$\frac{1}{L_{eq}} = \frac{1}{L_1} + \frac{1}{L_2} + \frac{1}{L_3} \quad (1.13)$$

1.3 Les composants électroniques Non-linéaires

1.3.1 La Diode

Description

La diode est un composant électronique. C'est un dipôle non linéaire et polarisé (*non symétrique*). Le sens de branchement d'une diode détermine le fonctionnement du circuit électronique dans lequel elle est placée.

Une diode désigne un dipôle qui ne laisse passer le courant électrique que dans un sens. Ce dipôle est appelé diode de redressement lorsqu'il est utilisé pour réaliser les redresseurs qui permettent de transformer le courant alternatif en courant continu. La figure 1.15 représente une diode réelle de référence 1N4004, ainsi que la figure 1.16 représente le symbole générique d'une diode.



FIGURE 1.15 – Diode réelles 1N4004

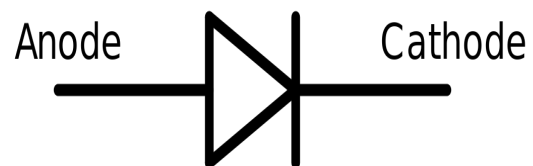


FIGURE 1.16 – Symbole générique

Technique de fabrication

Les diodes sont des composants fabriqués à partir de semi-conducteurs. Leur principe physique de fonctionnement est utilisé dans de nombreux circuits actifs en électronique. Une diode est créée en accolant un substrat déficitaire en électrons c'est-à-dire riche en trous (*semi-conducteur type P*) à un substrat riche en électrons libres (*semi-conducteur de type N ou métal*). La plupart des diodes sont réalisées par la jonction de deux semi-conducteurs : l'un dopé « P » l'autre dopé « N ».

La diode Schottky est constituée d'une jonction semi-conducteur/métal. La connexion du côté P s'appelle l'anode ; celle du côté N ou métal porte le nom de cathode.

Pour les diodes cylindriques, le côté de la cathode est généralement repéré par un anneau de couleur (figure 1.15). D'autres formes de repérage existent selon la nature de l'encapsulation de ces composants.

Principe de fonctionnement

La diode est un dipôle à semi-conducteur (*jonction P-N*), qui possède deux régimes de fonctionnement : **bloqué** et **passant**.

Ces régimes de fonctionnement ne sont pas contrôlables directement, mais dépendent de la tension V_{AK} aux bornes de la diode et de l'intensité du courant I_D (*courant direct, peut aussi s'écrire I_F avec F pour forward*) la traversant.

- **Diode bloquée** : état de la diode quand $V_{AK} < V_{seuil}$, ce qui empêche le passage du courant dans la diode ; $I_D = 0$.
- **Diode passante** : état de la diode quand $V_{AK} \geq V_{seuil}$, ce qui entraîne $I_D \neq 0$.

Caractéristique réelle

Lorsque la diode est bloquée, I_D n'est pas complètement nul mais vaut quelques nA (*courant de fuite*).

La tension de seuil V_{seuil} est une donnée fournie par les constructeurs et vaut typiquement :

- 0,3V pour les diodes au germanium.
- 0,7V pour les diodes au silicium.

Le courant I_D qui traverse la diode s'obtient par l'équation de **Shockley** (équation 1.14) :

$$I_D = I_0 \left[\exp\left(\frac{V_D}{nV_0} - 1\right) \right] \quad (1.14)$$

où :

- V_D est la tension aux bornes de la diode.
- V_0 appelé *tension thermique* est égal à $\frac{k_B T}{e}$ où k_B est la constante de **Boltzmann**, T la température absolue de la jonction et $-e$ la charge d'un électron. $V_0 = 26mV$ à $T = 20C$.
- n est le facteur de qualité de la diode, généralement compris entre 1 et 2.
- I_0 est la constante spécifique au type de diode considéré, homogène à un courant.

Cette constante est aussi appelée (*courant de saturation*) de la diode.

La Caractéristique (*courant/tension*) d'une diode réelle est donnée par la figure 1.17.

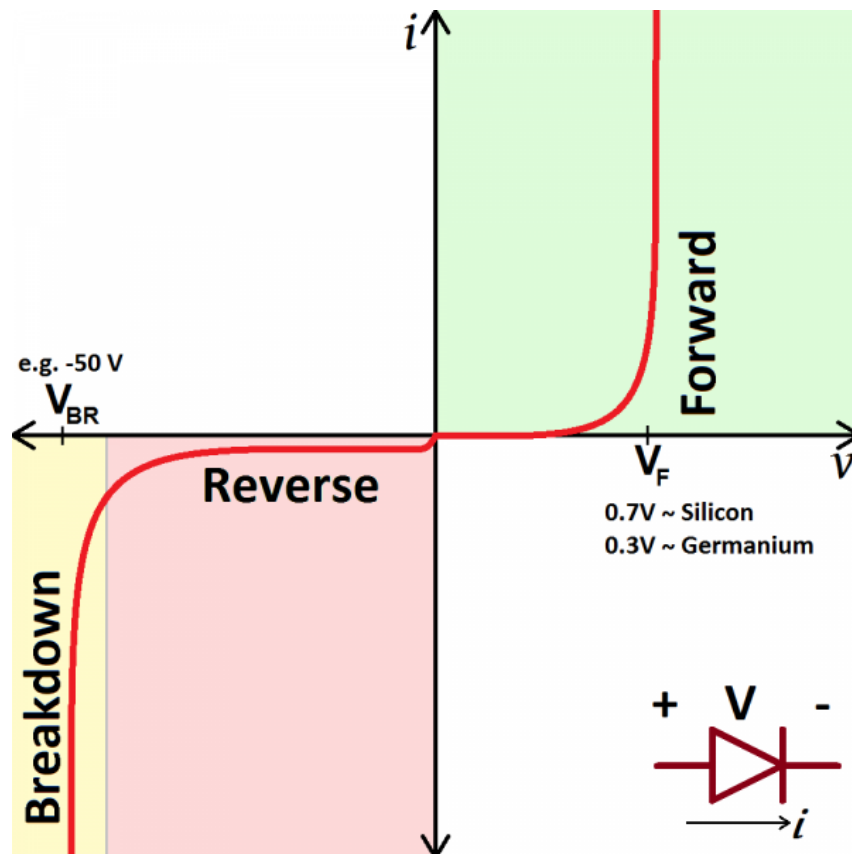


FIGURE 1.17 – Caractéristique courant/tension d'une diode

la figure suivante représente le symbole de quelques diodes spéciales :

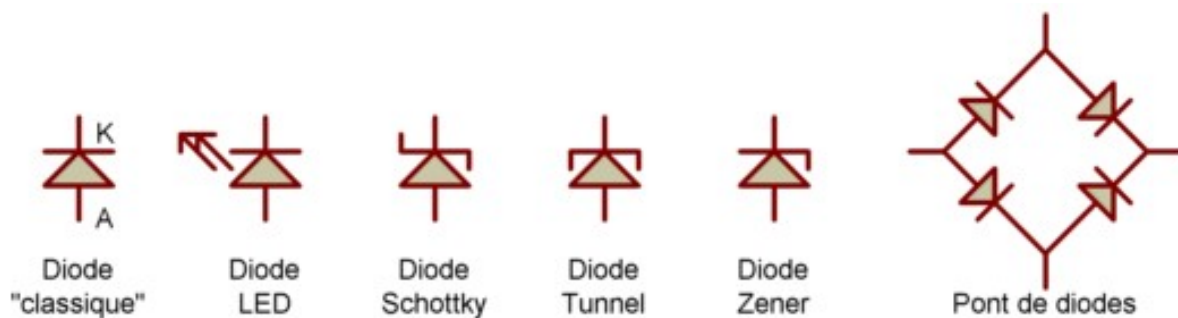


FIGURE 1.18 – Symboles de quelques diodes spéciale

1.3.2 Le Transistor

Description

Le transistor est un composant électronique qui est utilisé dans la plupart des circuits électroniques (*circuits logiques, amplificateur, stabilisateur de tension, modulation de signal, etc.*) aussi bien en basse qu'en haute tension.

Un transistor est un dispositif semi-conducteur à trois électrodes actives, qui permet de contrôler un courant ou une tension sur l'électrode de sortie (*le collecteur pour le transistor*

bipolaire et le drain sur un transistor à effet de champ) grâce à une électrode d'entrée (la base sur un transistor bipolaire et la grille pour un transistor à effet de champ). Le circuit

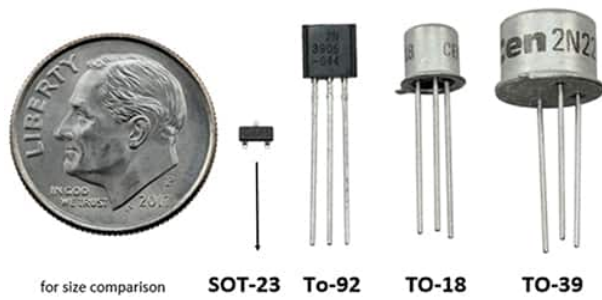


FIGURE 1.19 – Comparaison de tailles et variété de types de boîtiers populaires

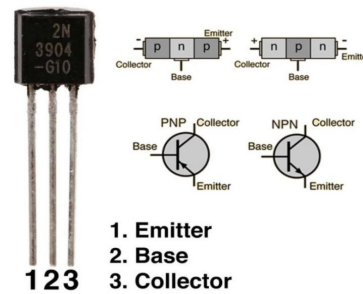


FIGURE 1.20 – le transistor 2N3204

étant connecté aux bornes « **collecteur** » et « **émetteur** », le transistor est isolant sans tension sur la borne Base, et conducteur avec une tension sur la borne Base.

En d'autres termes, c'est un interrupteur contrôlé électroniquement, sans partie mécanique comme dans le cas d'un relais. C'est un composant fondamental des appareils électroniques et des circuits logiques.

le transistor Bipolaire

Un transistor bipolaire est un dispositif électronique à base de semi-conducteur dont le principe de fonctionnement est basé sur deux jonctions PN, l'une en direct et l'autre en inverse.

Dans les deux types de transistors bipolaires (NPN et PNP), l'électrode traversée par l'ensemble du courant s'appelle l'émetteur. Le courant dans l'émetteur est égal à la somme des courants du collecteur et de la base. La flèche identifie l'émetteur et suit le sens du courant ; elle pointe vers l'extérieur dans le cas d'un **NPN**, vers l'intérieur dans le cas d'un **PNP**. L'électrode reliée au milieu de la barre centrale figure la base et la troisième électrode figure le collecteur.

le transistor à effet de champ

Contrairement au transistor bipolaire la grille agit par « **effet de champ** » (d'où son nom) et non par passage d'un courant électrique. Parmi les transistors à effet de champ (ou **FET**, pour *Field Effect Transistor*), on peut distinguer les familles suivantes :

- **Transistors MOSFET** : ils utilisent les propriétés des structures Métal/Oxyde/Semi-conducteur ; Dans le cas de l'effet de champ, la flèche disparaît, car le dispositif est symétrique (drain et source sont échangeables). Les traits obliques sont habituellement remplacés par des traits droits.

Pour le transistor MOS, la grille se détache des autres électrodes, pour indiquer l'isolation due à la présence de l'oxyde.

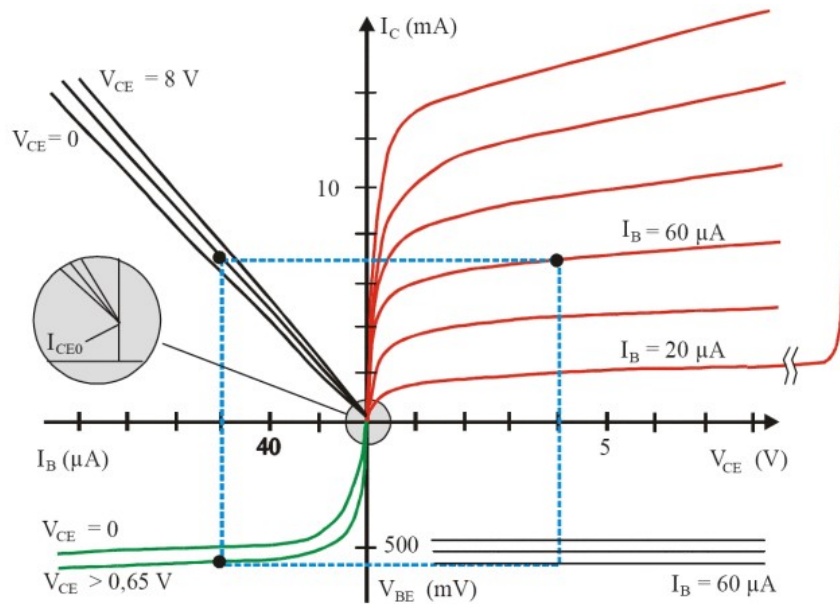


FIGURE 1.21 – Réseau des caractéristiques d'un transistor bipolaire NPN

En réalité, il existe une quatrième connexion pour les transistors à effet de champ, le substrat (parfois appelé bulk), qui est habituellement relié à la source (c'est la connexion entre S et les deux traits verticaux sur le schéma).

- **Transistors JFET** : ils utilisent les propriétés des jonctions PN. Un transistor de type JFET (Junction Field Effect Transistor) est un transistor à effet de champ dont la grille est directement en contact avec le canal. On distingue les JFET avec un canal de type N, et ceux avec un canal de type P.

Dans le cas de l'effet de champ, la flèche disparaît, car le dispositif est symétrique (drain et source sont échangeables). Les traits obliques sont habituellement remplacés par des traits droits.

Pour le transistor MOS, la grille se détache des autres électrodes, pour indiquer l'isolation due à la présence de l'oxyde.

En réalité, il existe une quatrième connexion pour les transistors à effet de champ, le substrat (*parfois appelé bulk*), qui est habituellement relié à la source (c'est la connexion entre S et les deux traits verticaux sur le schéma).

Symboles

Les symboles des transistors présentés ci-après se retrouvent parfois avec un cercle qui englobe le dessin : cercle ou pas cercle, c'est la même chose. Certains transistors intègrent une diode de protection, dans ce cas la diode est dessinée avec le transistor. C'est le cas des deux MOSFET entourés ci-après, et on en trouve aussi dans certains transistors darlington de puissance.

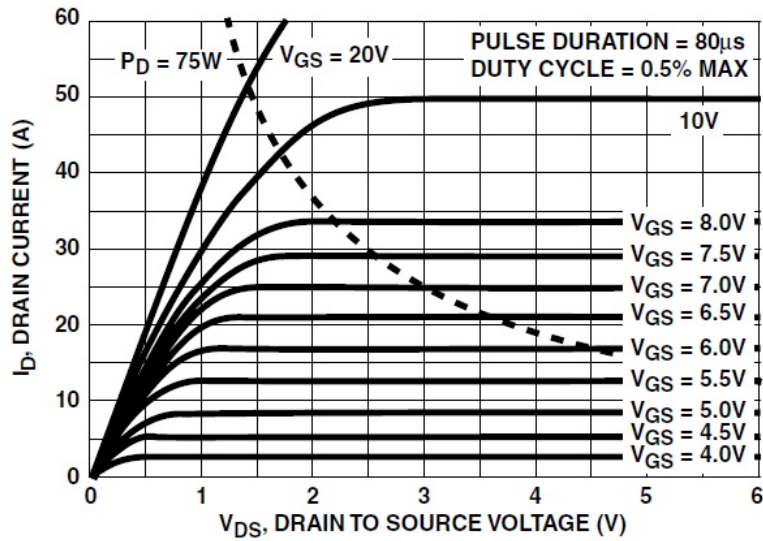


FIGURE 1.22 – Réseau des caractéristiques d'un transistor MOSFET

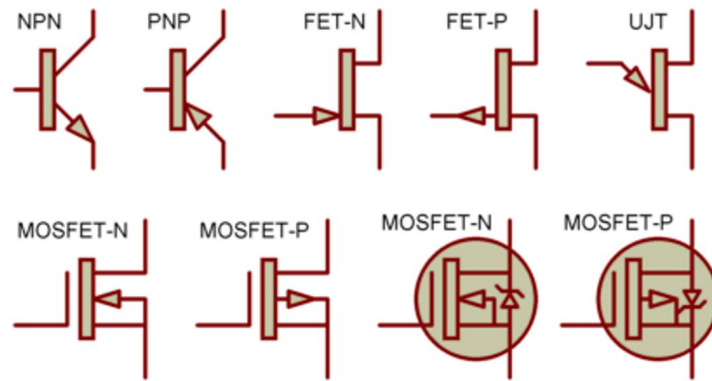


FIGURE 1.23 – Symboles des transistors présentés

Applications

Les deux principaux types de transistors permettent de répondre aux besoins de l'électronique **analogique** et **numérique** mais aussi à ceux de l'électronique de puissance et haute tension.

- La technologie bipolaire est plutôt utilisée en analogique et en électronique de puissance.
- Les technologies FET et CMOS sont principalement utilisées en électronique numérique (*réalisation d'opérations logiques*). Ils peuvent être utilisés pour faire des blocs analogiques dans des circuits numériques (*régulateur de tension par exemple*). Ils sont aussi utilisés pour faire des commandes de puissance (*moteurs*) et pour l'électronique haute tension (*automobile*). Ils offrent une meilleure linéarité dans le cadre d'amplificateurs Hi-Fi, donc moins de distorsion.

Un mélange des deux technologies est utilisé chez les IGBT.



FIGURE 1.24 – Transistor de puissance IGBT

Exemples de circuits de transistors

La Figure 1.25 illustre un exemple de circuit qui active la jonction collecteur-émetteur en alimentant la base, ou en polarisant le transistor pour l'activer, en alimentant la base avec $+5V_{dc}$ au moyen d'un interrupteur à glissière (*Slide Switch*). Cet exemple allume une LED qui représente la charge dans cet exemple.

L'utilisation appropriée de résistances pour éviter les surintensités est nécessaire lors de la polarisation de la base. La plupart des ingénieurs utilisent des composants à montage en surface (de format beaucoup plus petit qu'un boîtier TO-92) lorsqu'il s'agit d'utiliser des transistors dans la conception d'un nouveau produit à commercialiser (figure 1.19).

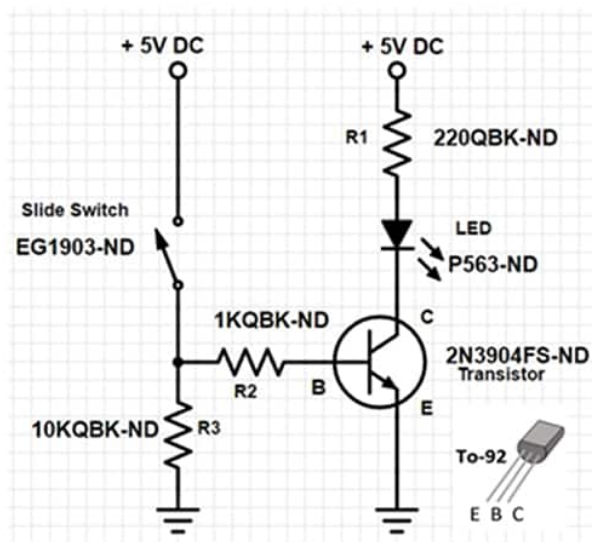


FIGURE 1.25 – Exemple de circuit 2N3904 pour allumer une LED avec un interrupteur

Puisque le 2N3904 est un transistor NPN, la base nécessite une polarisation positive pour activer la jonction collecteur-émetteur pour un flux de courant correct. L'utilisation d'une résistance de charge ($R1$) est également importante pour limiter la quantité de courant traversant la LED et le transistor. Pour plus d'informations sur ce transistor, voir la fiche technique (2N3904 Datasheet).

La Figure 1.26 est un exemple de circuit de veilleuse utilisant un transistor PNP.

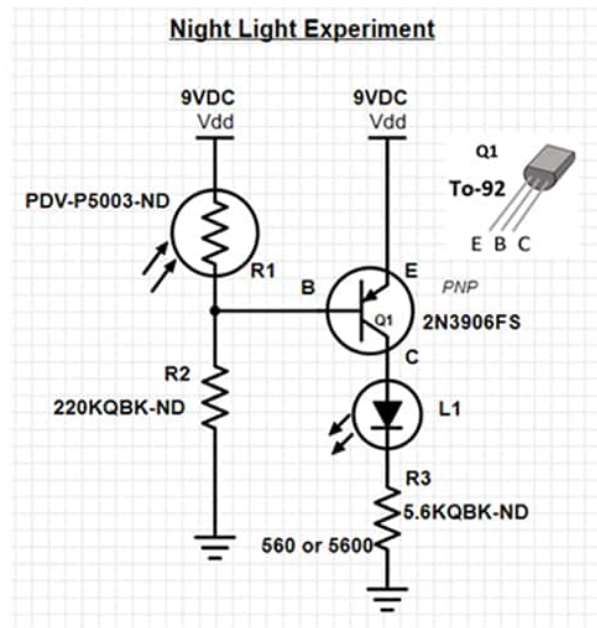


FIGURE 1.26 – Exemple de circuit de veilleuse 2N3906 pour allumer une LED avec une cellule photoélectrique PDV-P5003

En examinant les symboles schématiques dans la Figure 1.27, vous devez noter que :

- pour allumer un transistor de type NPN, vous devrez utiliser un courant positif sur le fil de base.
- Pour activer les transistors PNP, vous devrez utiliser un courant négatif ou au niveau du sol sur le fil de base.

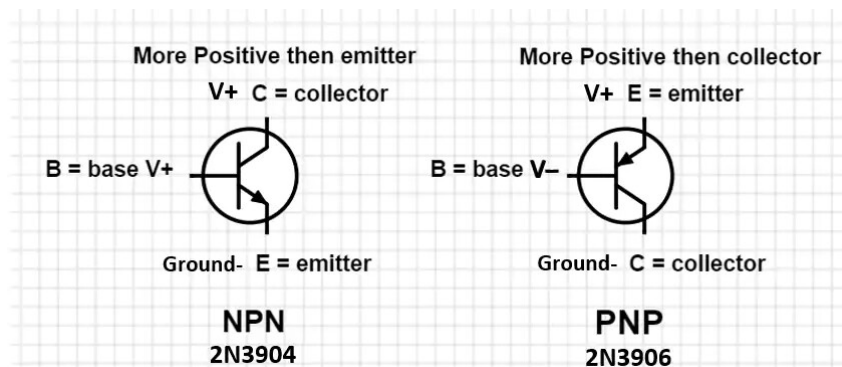


FIGURE 1.27 – Association des Résistances en série

Le transistor NPN doit avoir la charge du côté du collecteur qui est contrôlée par un courant positif circulant vers la base qui commute ensuite l'émetteur à la terre permettant au courant de circuler du collecteur à l'émetteur.

Pour les transistors de type PNP, l'émetteur est du côté positif de la tension d'alimentation et la charge est du côté du collecteur mais l'enfonce à la terre via un courant négatif allant à la base (répertorié comme V-, tension négative ou à la terre).

En ce qui concerne le circuit Night Light (Figure 1.26), nous voulions utiliser un transistor PNP pour l'explication du circuit, et nous voulions créer un circuit simple et intéressant pour aider les jeunes étudiants à construire un circuit utile qu'ils peuvent expérimenter.

En utilisant une cellule photo-conductrice CDS, nous pouvons créer une veilleuse (*ou un indicateur de faible niveau de lumière*) en utilisant seulement quelques résistances, un transistor (*dans ce cas, un PNP-2N3906FS*) et une LED dans le schéma.

1.3.3 L'amplificateur opérationnel (AOP)

Un amplificateur opérationnel (Figure 1.28) (*aussi dénommé ampli-op ou AOP*) est un amplificateur différentiel à grand gain, c'est un amplificateur électronique qui amplifie fortement une différence de potentiel électrique présente au bornes de ses deux entrées. Initialement, les AOP ont été conçus pour effectuer des opérations mathématiques dans les calculateurs analogiques, ils permettaient d'implémenter facilement les opérations mathématiques de base comme l'addition (+), la soustraction (−), l'intégration ($\int$), la dérivation ($\frac{d}{dt}$) et d'autres. Par la suite, l'amplificateur opérationnel est utilisé dans d'autres applications comme la commande des moteurs, la régulation de tension, les sources de courants ou encore la conception et la fabrication des oscillateurs.

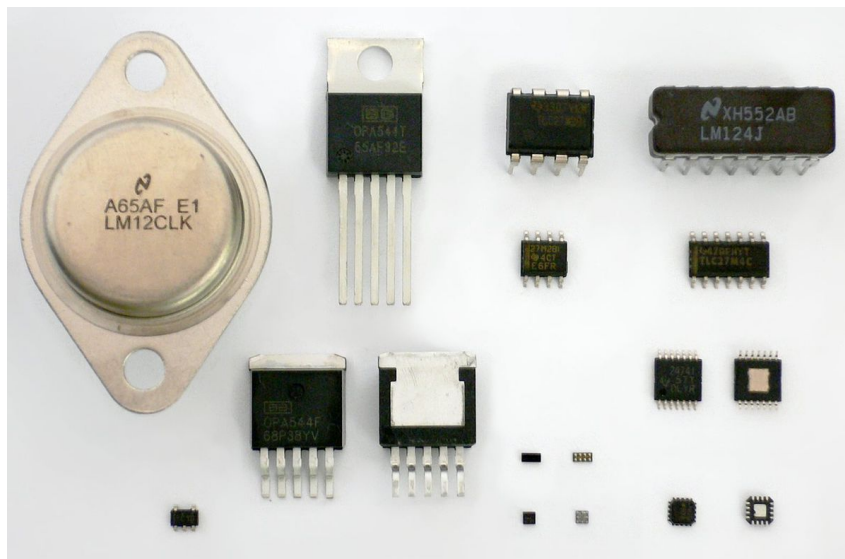


FIGURE 1.28 – Association des Résistances en série

Physiquement, un amplificateur opérationnel est constitué de transistors, de tubes électroniques ou de n'importe quels autres composants amplificateurs. On le trouve communément sous la forme d'un circuit intégré (Figure 1.28).

Le gain en tension très important d'un amplificateur opérationnel (AOP) en boucle ouverte fait de lui un composant utilisé dans une grande variété d'applications. Certains

amplificateurs opérationnels, de par leurs caractéristiques (*temps de montée, faible distortion harmonique, etc.*), sont spécialisés dans l'amplification de certains types de signaux comme les signaux audio ou vidéo.

Brochage

Un AOP dispose typiquement de deux entrées, deux broches d'alimentation et une sortie. L'entrée notée $e+$ est dite **non-inverseuse** tandis que l'entrée $e-$ est dite **inverseuse** (voir figure 1.29), ceci en raison de leur rôle respectif dans les relations entrée/sortie de l'amplificateur. La différence de potentiel entre ces deux entrées est appelée tension différentielle d'entrée (équation 1.15).

$$\varepsilon = e^+ - e^- \quad (1.15)$$

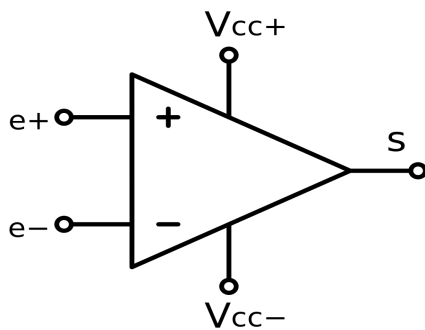


FIGURE 1.29 – Brochage théorique d'un AOP

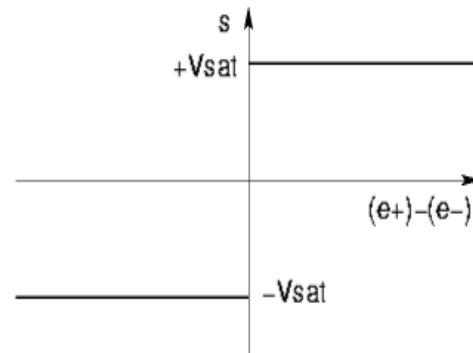


FIGURE 1.30 – Caractéristique entrée sortie d'un amplificateur opérationnel parfait

La broche d'alimentation positive repérée (V_{CC+}) est parfois aussi appelée (V_{DD}). La broche d'alimentation négative repérée (V_{CC-}) est parfois aussi appelée (V_{EE}). Le caractère doublé qui se trouve en indice de la lettre V fait référence au nom de la broche du transistor à laquelle cette alimentation sera généralement reliée.

Ainsi, les appellations (V_{CC}) et (V_{EE}) sont généralement réservées aux AOP bipolaires (**C** pour *Collecteur* et **E** pour *Émetteur*) tandis que les appellations (V_{DD}) et (V_{SS}) sont généralement réservées aux AOP à effet de champ (**D** pour *Drain* et **S** pour *Source*).

Suivant les applications, l'AOP peut aussi être doté de deux broches pour la compensation d'offset (Figure 1.31) ainsi que d'une broche pour le réglage de la compensation fréquentielle. Le mot "offset" signifie décalage, en électronique, Il représente le décalage en tension continue par rapport à la référence. C'est une tension de décalage de la sortie due à la constitution interne de l'AOP.

Il permet grâce à un potentiomètre (RV1) monté comme le montre la Figure 1.31 de régler $V_{out} = 0$ si $e = 0$ (ajustement du zéro en sortie).

Il existe des AOP possédant une sortie différentielle (Figure 1.32). De tels amplificateurs possèdent deux broches de sorties ainsi que quatre broches d'alimentation afin de pouvoir réaliser une isolation galvanique entre l'entrée et la sortie. Ces amplificateurs sont aussi appelés « *amplificateurs d'isolement* ».

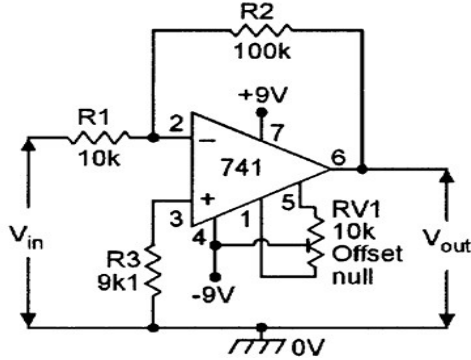


FIGURE 1.31 – AOP doté de deux broches pour la compensation d'offset

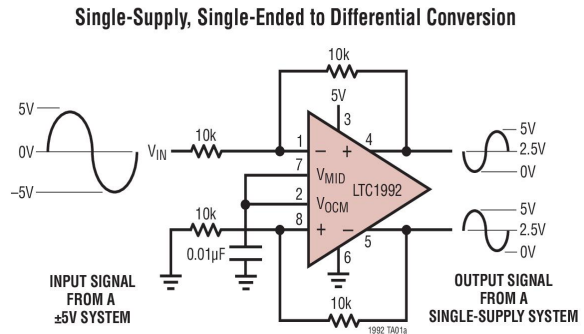


FIGURE 1.32 – AOP possédant une sortie différentielle

Le Mode Comparateur

Un simple AOP sans boucle de contre-réaction constitue un comparateur de tensions (voir Figure 1.33). Compte-tenu de son gain en boucle ouverte très élevé, l'AOP seul fonctionne selon les équations suivantes (1.16 et 1.17) :

$$Si \Rightarrow e^+ > e^- \text{ Alors } : V_s = +V_{sat} \approx +V_{cc} \quad (1.16)$$

$$Si \Rightarrow e^+ < e^- \text{ Alors } : V_s = -V_{sat} \approx -V_{cc} \quad (1.17)$$

V_s ne peut qu'être égal à $+V_{sat}$ ou $-V_{sat}$. On réalise donc la comparaison des deux tensions d'entrée. Au niveau vocabulaire technique, on parle de basculement du comparateur lors d'une transition $+V_{sat} \Rightarrow -V_{sat}$ ou $-V_{sat} \Rightarrow +V_{sat}$. Si l'on veut comparer une tension à une référence (*qui peut être la masse*) (voir Figure 1.33), il suffit de brancher cette dernière à une entrée de l'AOP, selon que l'on souhaite un niveau positif ou négatif en sortie lorsque la comparaison est vraie ou non. Cette tension constitue le seuil de basculement du comparateur.

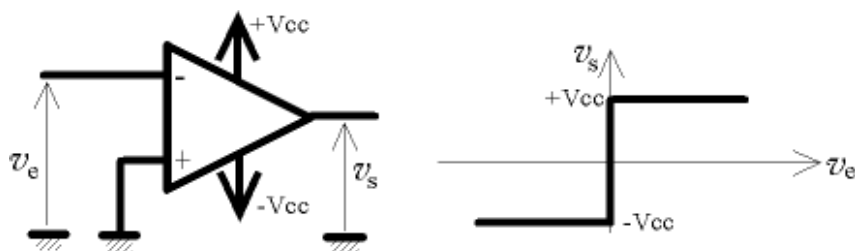


FIGURE 1.33 – AOP em mode comparateur

Le mode Amplificateur

Dans ce mode, l'amplificateur opérationnel utilisé est considéré parfait et fonctionne en « mode linéaire » car il utilise une contre-réaction sur l'entrée inverseuse de l'AOP (Figure 1.34). La contre-réaction sur l'entrée inverseuse permet d'effectuer une contre-réaction négative ; toute augmentation de la tension de sortie va diminuer la tension différentielle d'entrée de l'AOP. Ainsi, la différence de tension entre les deux entrées de l'amplificateur est maintenue à zéro. De plus, l'impédance d'entrée étant infinie, aucun courant ne circule dans ces entrées. On retrouve donc la tension V_e en sortie du pont diviseur de tension non chargé formé par R_1 et R_2 . On obtient alors :

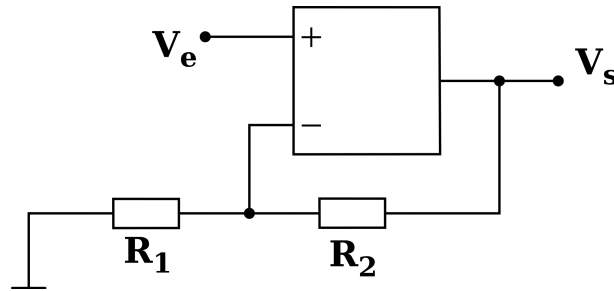


FIGURE 1.34 – Association des Résistances en série

$$V_e = V_s \times \frac{R_1}{R_1 + R_2} \quad (1.18)$$

et donc :

$$V_s = V_e \times \frac{R_1 + R_2}{R_1} = V_e \left(1 + \frac{R_2}{R_1} \right) \quad (1.19)$$

Chapitre 2

Système de Numération

2.1 Introduction

Un système de numération est une méthode qui permet de représenter des nombres en ordonnant des symboles : les chiffres. Dans la vie courante, on utilise dix chiffres : **0, 1, 2, ... , 9**. C'est le système de numération en base 10 ou système décimal. Un système de numération est un ensemble de symboles (les chiffres) qui sont assemblés en suivant des règles d'écriture précises permettant d'écrire, de lire et d'énoncer les nombres.

En informatique, le système de numération binaire (base 2) est couramment utilisé. Il utilise deux chiffres : **0** et **1**.

Selon le système de numération, un nombre a plusieurs écritures possibles. Il s'écrit par la succession de chiffres occupant chacun un rang. La valeur d'un chiffre dépend du rang et du système de numération adopté.

Dans l'Histoire, des dizaines de systèmes de numération ont été inventés pour répondre à différents besoins. Il existe des systèmes de numération additive tels que les chiffres romains et des systèmes de position comme la numération décimale (base 10) qu'on utilise dans la vie de tous les jours.

Depuis l'avènement des circuits électroniques et de l'informatique, les systèmes binaires (**base 2**) et hexadécimaux (**base 16**), qui sont également des systèmes de position, sont devenus indispensables. Mais, au juste, qu'est-ce qu'une base ?

2.2 Base d'un système de numération

Une base, dans un système de numération positionnel, est le nombre de symboles (*de chiffres*) qui sont utilisés pour représenter les nombres. En base 10 (*la numération décimale*), on utilise donc 10 chiffres, soit de 0 à 9, tandis qu'en base 2 (*la numération binaire*), on n'utilise que 2 chiffres, c'est-à-dire le zéro (**0**) et le un (**1**).

2.2.1 Système Décimal

Nous utilisons le système décimal (**base 10**) dans nos activités quotidiennes. Ce système est basé sur une logique à dix symboles, de 0 à 9 :

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9$$

avec une unité supérieure (*dizaine, centaine, etc.*) à chaque fois que dix unités sont comptabilisées. C'est un système positionnel, c'est-à-dire que l'endroit où se trouve le symbole définit sa valeur. Ainsi, le 2 de 523 n'a pas la même valeur que le 2 de 132. En fait 523 est l'abréviation de :

$$5 \times 100 + 2 \times 10 + 3$$

. On peut selon ce principe imaginer une infinité de systèmes numériques fondés sur des bases différentes. Ci on prend par exemple le nombre Binaire suivant :

$$N = (7924)_{10}$$

le dernier chiffre de droite (4) représente le symbole de poids le plus faible (**LSB**), ainsi que le chiffre le plus à gauche (7) représente le symbole de poids le plus fort (**MSB**). ce nombre s'écrit sous forme d'un polynôme comme le montre l'expression suivante :

$$N = (7924)_{10} = 7 \times 10^3 + 9 \times 10^2 + 2 \times 10^1 + 4 \times 10^0 \quad (2.1)$$

Donc, l'expression générale d'un nombre Binaire présenté sous la forme d'un polynôme est donne par l'expression :

$$N = \sum_{i=0}^n \alpha_i \times 10^i \quad (2.2)$$

avec α_i représente l'un des chiffre de la base 10.

2.2.2 Système Binaire

En informatique, outre la base 10, on utilise très fréquemment le système binaire (**base 2**) puisque la logique booléenne est à la base de l'électronique numérique. Deux symboles suffisent :

$$0, 1$$

. Cette unité élémentaire ne pouvant prendre que les valeurs 0 et 1 s'appelle un **bit** (*de l'anglais binary digit*). Une suite de huit bits s'appelle un octet.

On nomme couramment bit (de l'anglais binary digit, soit « chiffre binaire ») les chiffres de la numération binaire positionnelle. Un bit peut prendre deux valeurs, notées par convention 0 et 1. Le système binaire est utile pour représenter le fonctionnement de

l'électronique numérique utilisée dans les ordinateurs. Il est donc utilisé par les langages de programmation de bas niveau.

Le système binaire le plus courant est la base deux mathématique, permettant de représenter des nombres à l'aide de la numération de position avec seulement deux chiffres : le 0 et le 1.

Dans ce type de codage, chaque nombre est représenté de façon unique par une suite ordonnée de chiffres. Et chaque position m représente une puissance $(m - 1)$ de la base. Si l'on se limite dans un premier temps aux nombres entiers positifs, en base dix ces puissances sont : un (1), dix (10), cent (dix fois dix, représenté par 100), mille (dix fois cent, représenté par 1000), dix mille, etc. En base deux, ces puissances sont : un (1), deux (représenté lui aussi par 10), quatre (deux fois deux, représenté par 100), huit (deux fois quatre, représenté par 1000), seize (deux fois huit, représenté par 10000), etc.

On voit que la signification des représentations 10, 100, 1000, etc. dépend de la base utilisée : 10 est toujours égal à la base, c'est-à-dire dix en base dix, mais deux en base deux.

En base dix, on utilise dix chiffres, de zéro à neuf; en base n , on utilise n chiffres, de zro à $n-1$; donc en base deux on utilise les deux chiffres « 0 » et « 1 ».

Ci on prend par exemple le nombre Binaire suivant :

$$N = (11001010)_2$$

le dernier chiffre de droite représente le symbole de poids le plus faible (**LSB**), ainsi que le chiffre le plus à gauche représente le symbole de poids le plus fort (**MSB**). ce nombre s'écrit sous forme d'un polynôme comme le montre l'expression suivante :

$$N = (11001010)_2 = 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \quad (2.3)$$

Donc, l'expression générale d'un nombre Binaire présenté sous la forme d'un polynôme est donne par l'expression :

$$N = \sum_{i=0}^n \alpha_i \times 2^i \quad (2.4)$$

avec α_i représente l'un des chiffre de la base 2.

2.2.3 Système Octal

Le système de numération octal est le système de numération de base 8, et utilise les chiffres de 0 à 7 :

$$0, 1, 2, 3, 4, 5, 6, 7$$

Le système octal est quelquefois utilisé en calcul à la place de l'hexadécimal. Il possède le double avantage de ne pas requérir de symbole supplémentaire pour ses chiffres et d'être

une puissance de deux pour pouvoir grouper les chiffres.

Le décompte octal pourrait avoir été utilisé dans le passé à la place du décompte décimal, en comptant soit les trous entre les doigts ou les doigts différents des pouces. Ci on prend par exemple le nombre Octal suivant :

$$N = (3715)_8$$

5 représente le symbole de poids le plus faible (**LSB**), ainsi que 3 représente le symbole de poids le plus fort (**MSB**). ce nombre s'écrit sous forme d'un polynôme comme le montre l'expression suivante :

$$N = (3715)_8 = 3 \times 8^3 + 7 \times 8^2 + 1 \times 8^1 + 5 \times 8^0 \quad (2.5)$$

Donc, l'expression générale d'un nombre Octal présenté sous la forme d'un polynôme est donne par l'expression :

$$N = \sum_{i=0}^n \alpha_i \times 8^i \quad (2.6)$$

avec α_i représente l'un des chiffre de la base Octal.

2.2.4 Système Hexadécimal

On utilise aussi très souvent le système hexadécimal (**base 16**) du fait de sa simplicité d'utilisation et de représentation pour les mots machines (*il est bien plus simple d'utilisation que le binaire*). Ce système comprend dix chiffres et six symboles (*lettres*) supplémentaires qui sont :

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$$

Ci on prend par exemple le nombre Hexadécimale suivant :

$$N = (AC53)_{16}$$

3 représente le symbole de poids le plus faible (**LSB**), ainsi que A représente le symbole de poids le plus fort (**MSB**). ce nombre s'écrit sous forme d'un polynôme comme le montre l'expression suivante :

$$N = (AC53)_{16} = A \times 16^3 + C \times 16^2 + 5 \times 16^1 + 3 \times 16^0 \quad (2.7)$$

Comme nos chiffres (le mot "bit" est parfois emprunté au binaire, celui de "chiffre" est préférable) usuels ne peuvent exprimer que dix valeurs il fallait trouver une parade. Celle choisie consiste à utiliser des lettres. Nous utiliserons donc A,B,C,D,E et F, respectivement 10, 11, 12, 13, 14 et 15.

On précède généralement un digit hexadécimal de "0x" pour notifier que l'on utilise l'hexadécimal : en effet si le nombre évoqué est pas exemple **0x443**, cela donne **1091** en décimal ce qui diffère quelque peu de **443** vous en conviendrez.

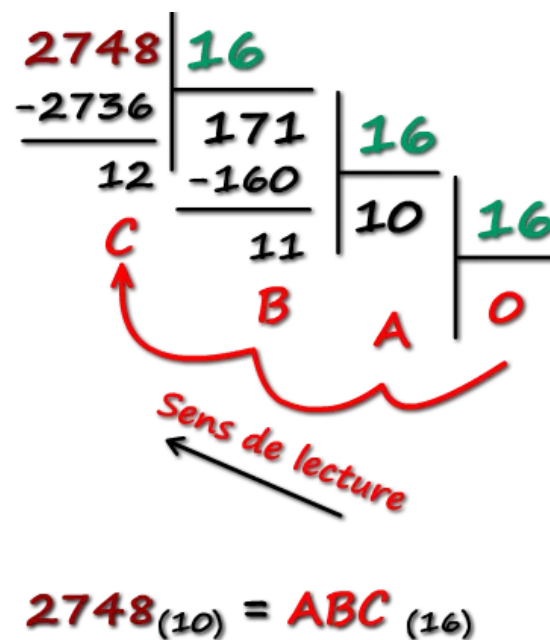


FIGURE 2.3 – Conversion décimal/hexadécimal

Sur la figure 2.3, c'est la conversion du nombre $(2748)_{10}$ en son équivalent en hexadécimal, ce qui donne :

$$(2748)_{10} = (0ABC)_{16}$$

2.3.2 Conversion des autres bases (2, 8 et 16) en Décimal

Conversion binaire vers le décimal

Le système binaire (de base deux) est fondé sur deux chiffres, le plus souvent 0 ou 1. À l'opposé, le système décimal (de base dix) est fondé sur dix chiffres (0, 1, 2, 3, 4, 5, 6, 7, 8 ou 9). Pour éviter toute confusion lors de l'utilisation de systèmes numériques différents, la base de chaque nombre peut être spécifiée en la mettant en indice à côté du nombre. Par exemple, le nombre binaire 10011100 peut être spécifié comme étant de « base deux » en l'écrivant : 10011100₂. Le nombre décimal 156 peut être écrit : 156₁₀ et sera lu « cent-cinquante-six en base dix ». Le système binaire est le langage de base des ordinateurs, donc tout bon programmeur informatique se doit de savoir convertir du binaire en décimal.

Étape 1 : Notez le nombre binaire et listez les puissances de 2 de droite à gauche. Dans cet exemple, nous allons convertir le nombre binaire 10011011₂ en nombre décimal. D'abord, notez-le. Inscrivez la liste des puissances de deux de droite à gauche. Commencez à 20, soit « 1 ». Incrémentez l'exposant d'une unité à chaque fois. Arrêtez-vous lorsque vous avez autant de puissances que de chiffres dans le nombre binaire. Dans notre exemple, 10011011 comporte huit chiffres, donc vous obtenez les 8 nombres suivants : 128, 64, 32, 16, 8, 4, 2, 1.

10011011 ₂							
2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
128	64	32	16	8	4	2	1

FIGURE 2.4 – Conversion Binaire to Décimale : Étape 1

Étape 2 : Sous cette liste, réécrivez les puissances, en respectant l'alignement. Écrivez 10011011 sous les nombres 128, 64, 32, 16, 8, 4, 2 et 1 de sorte que chaque chiffre binaire corresponde à sa puissance de deux. Le « 1 » tout à droite du nombre écrit en binaire doit correspondre au « 1 » tout à droite de la liste des puissances de deux et ainsi de suite. Vous pouvez aussi écrire les chiffres binaires au-dessus des puissances de deux, si vous préférez. Ce qui importe c'est qu'ils se correspondent.

128	64	32	16	8	4	2	1
1	0	0	1	1	0	1	1

FIGURE 2.5 – Conversion Binaire to Décimale : Étape 2

Étape 3 : Connectez chaque chiffre du nombre binaire à la puissance de deux correspondante. Tracez des petits tirets qui relient la puissance au chiffre binaire en commençant par la droite, en connectant chaque chiffre consécutif du nombre binaire à la puissance de deux qui arrive ensuite dans la liste du haut. Commencez par dessiner un trait reliant le premier chiffre du nombre binaire à la première puissance de deux dans la liste au-dessus. Ensuite, dessinez une ligne qui part du deuxième chiffre du nombre binaire et qui le relie à la seconde puissance de deux dans la liste. Continuez à connecter chaque chiffre binaire avec la puissance de deux correspondante. Cela vous aidera à visualiser la relation entre les deux groupes de nombre.

128	64	32	16	8	4	2	1
1	0	0	1	1	0	1	1

FIGURE 2.6 – Conversion Binaire to Décimale : Étape 3

Étape 4 : Écrivez la valeur finale de chaque puissance de deux. Passez en revue tous les chiffres du nombre en binaire. Si le chiffre est un 1, écrivez la puissance de deux correspondante en bas de la ligne, sous le chiffre. Si le chiffre est un 0, écrivez 0 en bas de la ligne, sous le chiffre. Puisque « 1 » correspond à « 1 », il reste « 1 ». Puisque « 2 » correspond à « 1 », il devient « 2 ». Puisque « 4 » correspond à « 0 », il devient « 0 ». Puisque « 8 » correspond à « 1 », il devient « 8 » et puisque « 16 » correspond à « 1 », il devient « 16 ». « 32 » correspond à « 0 » et devient « 0 » et « 64 » correspond à « 0 » et donne donc « 0 » alors que « 128 » correspond à « 1 » et devient « 128 ».

128	64	32	16	8	4	2	1
1	0	0	1	1	0	1	1
128	0	0	16	8	0	2	1

FIGURE 2.7 – Conversion Binaire to Décimale : Étape 4

Étape 5 : Additionnez les valeurs finales. À présent, additionnez les nombres écrits sous la ligne. Voici ce que cela donne : $128 + 0 + 0 + 16 + 8 + 0 + 2 + 1 = 155$. C'est l'équivalent décimal du nombre binaire 10011011.

128	64	32	16	8	4	2	1
1	0	0	1	1	0	1	1
128	0	0	16	8	0	2	1
= 155							

FIGURE 2.8 – Conversion Binaire to Décimale : Étape 5

Étape 6 : Notez la réponse à côté de sa base notée en indice. À présent, il ne vous reste plus qu'à écrire 155₁₀, pour indiquer que vous travaillez en nombre décimal, soit en puissances de 10. Plus vous vous habituerez à convertir les nombres binaires en décimaux, plus il vous sera facile de mémoriser les puissances de deux et plus vous serez capable d'exécuter cette tâche rapidement.

$$10011011_2 = \boxed{155_{10}}$$

FIGURE 2.9 – Conversion Binaire to Décimale : Étape 6

Étape 7 : Utilisez cette méthode pour convertir un nombre binaire avec une virgule en nombre décimal. Vous pouvez aussi utiliser cette méthode si vous voulez convertir un nombre binaire tel que 1,12 en nombre décimal. Tout ce que vous devez savoir c'est que le nombre du côté gauche de la virgule constitue un entier naturel, comme à l'ordinaire, tandis que le nombre à droite de la virgule forme la partie « fractionnaire » représentant un demi, soit $1 \times (1/2)$. Le « 1 » à gauche de la virgule est égal à 20, soit 1. Le « 1 » à droite de la virgule est égal à 2^{-1} , soit 0,5. Additionnez 1 et 0,5 et vous obtiendrez 1,5, ce qui est l'équivalent décimal de 1,12.

$$\begin{array}{c} \text{e.g.) } 1.1_2 \\ \begin{array}{cc} 1 & . & 1 \\ \downarrow & & \downarrow \\ 2^0 & & 2^{-1} \end{array} \\ \hline 1 + 0.5 = \boxed{1.5_{10}} \end{array}$$

FIGURE 2.10 – Conversion Binaire to Décimale : Étape 7

Conversion octal vers le décimal

La conversion octale en décimale a lieu lorsque nous voulons connaître l'équivalent d'un nombre dans le système de numération. Le système de numération est de quatre types - système de numération binaire, système de numération octale, système de numération décimale et système de numération hexadécimale. Chaque système de numération a ses propres numéros de base qui aident à identifier de quel type de numéro il s'agit. Ces nombres de base aident également à la conversion octale en décimale. Le nombre de base pour les nombres octaux est 8 et le nombre de base pour les nombres décimaux est 10.

Comme pour toute autre conversion dans le système numérique, la conversion octale en décimale est également effectuée en utilisant son nombre de base. Pour convertir octal en décimal, nous devons multiplier les chiffres octaux avec la puissance de 8 en partant du côté droit et en diminuant progressivement jusqu'à zéro pour résumer tous les produits. Voici les étapes pour convertir un nombre octal en décimal (*l'exemple de conversion du nombre octal (140)₈ en un nombre décimal sera traiter*) :

Étape 1 : Puisqu'un nombre octal n'utilise que des chiffres de 0 à 7, nous organisons d'abord le nombre octal avec la puissance de 8.

- Écrivez $(140)_8$ avec la puissance de 8. Commencez par le côté droit :

$$1 \times 8^2 + 4 \times 8^1 + 0 \times 8^0$$

Étape 2 : Nous évaluons toute la puissance de 8 valeurs telles que 8^0 est 1, 8^1 est 8, etc., et notons la valeur de chaque nombre octal.

- Évaluez les valeurs de la puissance de 8 pour chaque nombre octal :

$$8^2 = 64, 8^1 = 8, 8^0 = 1$$

Étape 3 : Une fois la valeur obtenue, nous multiplions chaque nombre.

- Multipliez chacun des nombres de puissance de 8 avec les nombres respectifs :

$$1 \times 64 + 4 \times 8 + 0 \times 1 = 64 + 32 + 0$$

Étape 4 : La dernière étape consiste à additionner le produit de tous les nombres pour obtenir le nombre décimal.

- Additionnez les valeurs pour obtenir le nombre décimal :

$$64 + 32 + 0 = 96$$

Par conséquent :

$$(140)_8 = (96)_{10}$$

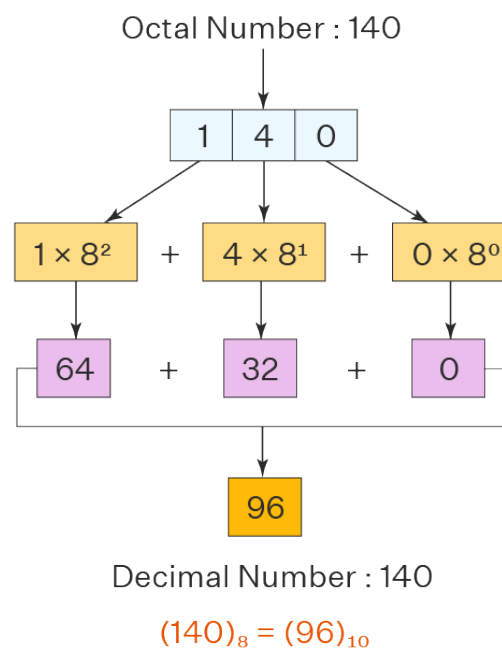


FIGURE 2.11 – Étapes de conversion octal en décimal

Pour convertir un nombre octal en nombre décimal avec un point décimal, nous devons suivre la même procédure que dans la section précédente. Cependant, la puissance ou les exposants de 8 varieront après la virgule décimale. Puisque nous nous déplaçons vers la droite avec les exposants croissants, les exposants après la virgule diminueront ou seront négatifs. Prenons un exemple.

pour Convertir le nombre octal $(246.28)_8$ en un nombre décimal. Nous suivrons les mêmes étapes que précédemment (voir Figure 2.12).

• **Étape 1 :** Écrivez $(246.28)_8$ avec la puissance de 8. Commencez par le côté droit. Ici, la puissance de 8 sera négative après la virgule.

$$(2 \times 8^2) + (4 \times 8^1) + (6 \times 8^0) + (2 \times 8^{-1}) + (8 \times 8^{-2})$$

• **Étape 2 :** Évaluez les valeurs de la puissance de 8 pour chaque nombre octal.

$$(8^2 = 64), (8^1 = 8), (8^0 = 1), (8^{-1} = \frac{1}{8}), (8^{-2} = \frac{1}{8^2} \text{ or } \frac{1}{64})$$

• **Étape 3 :** Multipliez chacun des nombres de puissance de 8 avec les nombres respectifs :

$$(2 \times 64) + (4 \times 8) + (6 \times 1) + (2 \times \frac{1}{8}) + (8 \times \frac{1}{64}) = 128 + 32 + 6 + 0.25 + 0.125$$

• **Étape 4 :** Additionnez les valeurs pour obtenir le nombre décimal :

$$128 + 32 + 6 + 0.25 + 0.125 = 166.375$$

Par conséquent :

$$(246.28)_8 = (166.375)_{10}$$

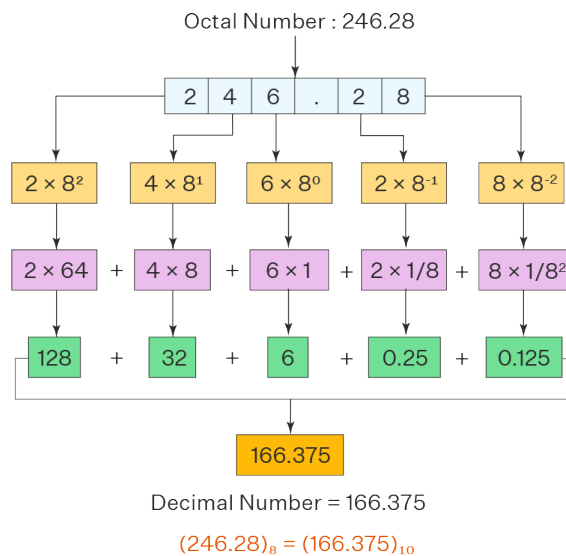


FIGURE 2.12 – Étapes de conversion octal en décimal avec un point décimal

Conversion hexadécimal vers le décimal

La conversion hexadécimale en décimale est effectuée en gardant à l'esprit les nombres de base de chacun des systèmes de numération. Le système numérique hexadécimal fonctionne avec des chiffres et des caractères, en particulier pour représenter les caractères à deux chiffres. Il y a un total de 16 notations : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 et 10 – 15 est représenté par A, B, C, D, E, F . Alors que dans le système décimal, 10 notations sont représentées par 0, 1, 2, 3, 4, 5, 6, 7, 8 et 9. Pour convertir l'hexadécimal en décimal, nous obtenons d'abord l'équivalent décimal de l'hexadécimal à partir de la table de conversion, multipliez le chiffre par 16 à la puissance de l'emplacement du chiffre et additionnez le tout. La table de conversion est mentionnée ci-dessous :

TABLE 2.1 – Table de conversion Hexadécimal en décimal

Hexadécimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Décimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

La conversion de l'hexadécimal en décimal se fait en utilisant le nombre de base 16. Le chiffre hexadécimal est développé pour multiplier chaque chiffre par la puissance de 16. La puissance commence à 0 à partir de la droite en avançant vers la gauche avec l'augmentation de la puissance. Pour que la conversion se termine, les nombres multipliés sont additionnés.

$$DecimalNumber = (d_{n-1} \times 16^{r-1}) + \dots + (d_2 \times 16^2) + (d_1 \times 16^1) + (d_0 \times 16^0)$$

Où,

- n = Nombre de chiffres.
- r = Placement du chiffre.

Les étapes pour convertir un nombre hexadécimal en décimal sont :

• **Étape 1** : Obtenez l'équivalent décimal de l'hexadécimal à partir de la table de conversion. (tableau mentionné ci-dessus) • **Étape 2** : Multipliez chaque chiffre avec la puissance de 16 en partant de 0 à partir de la droite. • **Étape 3** : Additionnez tous les nombres ensemble. Pour mieux comprendre cela, Prenons un exemple :

$$(25A)_{16} = (2 \times 16^2) + (5 \times 16^1) + (10 \times 16^0)$$

$$(25A)_{16} = 512 + 80 + 10$$

$$(25A)_{16} = (602)_{10}$$

Chapitre 3

Le Codage

Un code binaire représente un texte, des instructions de processeur ou toute autre donnée utilisant un système à deux symboles. Le système à deux symboles utilise souvent des "0" et "1" dans le système de numération binaire. Le code binaire assigne une combinaison de chiffres binaires, également appelé bits, à chaque caractère, instruction, etc. Par exemple, une chaîne binaire de huit bits peut représenter n'importe laquelle des 255 valeurs possibles et, par conséquent, représenter une grande variété d'éléments différents.

En informatique et en télécommunications, les codes binaires sont utilisés pour diverses méthodes de codage de données, telles que des chaînes de caractères, en chaînes de bits. Ces méthodes peuvent utiliser des chaînes à largeur fixe ou à largeur variable. Dans un code binaire à largeur fixe, chaque lettre, chiffre ou autre caractère est représenté par une chaîne de bits de même longueur ; cette chaîne de bits, interprétée comme un nombre binaire, est généralement affichée dans les tables de codes en notation octale, décimale ou hexadécimale. Il existe de nombreux jeux de caractères et de nombreux codages de caractères pour ceux-ci.

Une chaîne de bits, interprétée comme nombre binaire, peut être traduite en nombre décimal. Par exemple, la lettre minuscule (**a**), si elle est représentée par la chaîne de bits $(01100001)_2$ (*comme dans le code ASCII standard*), peut également être représentée par le nombre décimal $(97)_{10}$.

3.1 Code binaire naturel

Ce code utilise l'expression naturelle du nombre en base 2 (*système de numération binaire*). Il est donc intéressant pour effectuer les opérations arithmétiques.

On dit que ce code est un code pondéré : le i^{eme} bit (*en partant de la droite*) « pèse » 2^{i-1} .

Rappel : Sinon, voici une méthode « *manuelle* » simple :

1. Réaliser la division euclidienne du nombre à coder par 2, mettre le reste à droite

dans le tableau.

2. Recommencer la division par 2 avec le quotient à gauche du chiffre binaire précédent.
3. Continuer jusqu'à ce que le quotient ne soit plus divisible par 2.

Pour un nombre entier naturel (*positif*), on peut présenter son écriture binaire dans un tableau (Figure 3.1) faisant apparaître les puissances successives de 2 (*la plus grande puissance de 2 doit être juste inférieure au nombre à coder*)

Nombre décimal	Code binaire naturel Sur 4-bits
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

FIGURE 3.1 – Code binaire naturel sur 4-bits

Inconvénient : 2 nombres successifs peuvent posséder 2 codes binaires naturels non adjacents, c'est à dire que plusieurs bits changent d'état lorsqu'on passe d'un nombre à son suivant. Ce changement n'étant pas forcément simultané pour tous les bits, on passe par un nombre non voulu (*parasite*), qui peut être source d'erreurs. Pour résoudre ce problème on peut employer le Code Binaire Réfléchi.

3.2 Code binaire réfléchi (Code Gray)

Le code de Gray, également appelé code Gray ou code binaire réfléchi, est un type de codage binaire permettant de ne modifier qu'un seul bit à la fois quand un nombre est

augmenté d'une unité. Cette propriété est importante pour plusieurs applications.

Le nom du code vient de l'ingénieur américain Frank Gray qui publia un brevet sur ce code en 1953.

Le code de Gray est un codage binaire, c'est-à-dire une fonction qui associe à chaque nombre une représentation binaire. Cette méthode est différente du codage binaire naturel. Le tableau de la Figure 3.2 montre partiellement le codage sur 4-bits (*seules les 8 premières valeurs sont présentées, les huit suivantes avec le premier bit à 1 n'y sont pas*).

Nombre décimal	Code binaire naturel Sur 4-bits	Code Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

FIGURE 3.2 – Code Gray sur 4-bits

La différence principale entre les deux est le fait que le codage de Gray de deux nombres consécutifs ne diffère que d'une position. Par exemple $(5)_{10}$ est codé par $(0111)_{Gray}$, et $(6)_{10}$ est codé par $(0101)_{Gray}$, on remarque clairement que seul le deuxième bit change.

3.2.1 Construction de la table par symétrie

Le nom de code binaire réfléchi, comme autre nom du code de Gray, vient d'une méthode de construction basée sur la réflexion de blocs de codes déjà construits (Figure 3.3) :

- On choisit un code de départ : zéro est codé 0 et un est codé 1,
- Puis, à chaque fois qu'on a besoin d'un bit supplémentaire, on symétrise la liste des codes déjà obtenus (*comme une réflexion dans un miroir*).
- Enfin, on rajoute un 0 puis un 1 au début (*à gauche*) de chacun des codes. On a ainsi doublé le nombre de codes formés.

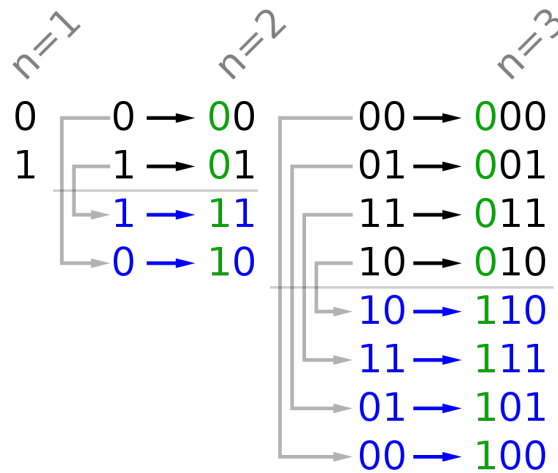


FIGURE 3.3 – Principe de construction du code Gray

3.3 Code BCD

Le code BCD (*Binary Coded Decimal*) qui signifie Décimal codé binaire permet de représenter un chiffre décimal de 0 à 9 par un ensemble de 4-bits.

C'est un système de numération utilisé en électronique numérique et en informatique pour coder des nombres en se rapprochant de la représentation humaine usuelle, en base 10. Dans ce format, les nombres sont représentés par un ou plusieurs chiffres compris entre 0 et 9, et chacun de ces chiffres est codé sur quatre bits (Figure 3.4).

Nombre décimal	Code binaire naturel Sur 4-bits	Code BCD
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1000
9	1001	1001
10	1010	-----
11	1011	-----

FIGURE 3.4 – Codes BCD

Par exemple, pour coder un nombre tel que 246, il suffit de coder chacun des chiffres 2, 4 et 6 séparément, et l'on obtient la valeur 0010 0100 0110. De même pour le nombre

593, chacun des chiffres 5, 9 et 3 est codé en binaire, ce qui donne 0101 1001 0011.

Étant donné que 4 bits peuvent en théorie représenter 16 valeurs distinctes et que l'on se restreint à 10 valeurs possibles, on note une augmentation du nombre de bits requis pour représenter un nombre lors de l'utilisation du format BCD. Ainsi par exemple, pour représenter la valeur $(12)_{10}$ ($(1100)_2$ en binaire naturel, $(0001\ 0010)$ en DCB), il faudra utiliser 4-bits de plus au format BCD qu'en binaire naturel. Plus généralement 8-bits permettent de coder une valeur comprise entre 0 (00000000) et 99 (10011001) en BCD, là où l'utilisation d'un code binaire naturel permettrait de représenter des valeurs allant jusqu'à 255 (11111111)₂.

Certains systèmes mettent à profit la possibilité pour les 4-bits BCD de prendre des valeurs supérieures à 9 afin de coder des informations supplémentaires, comme notamment le signe du nombre représenté. Par exemple, les valeurs 12 $(1100)_{BCD}$ et 13 $(1101)_{BCD}$ sont couramment utilisées pour coder le signe plus (+) et le signe moins (−) respectivement, le nombre négatif (-420) pouvant ainsi être représenté par 0100 0010 0000 1101 (*avec le signe à la fin*).

Remarque : Le codage du signe peut également utiliser des valeurs différentes.

3.4 Le codage ASCII

Le Code américain normalisé pour l'échange d'information (En anglais : **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange), plus connu sous l'acronyme **ASCII**, est une norme informatique de codage de caractères apparue dans les années 1960. C'est la norme de codage de caractères la plus influente à ce jour. ASCII définit 128 codes à 7-bits, comprenant **95 caractères imprimables** :

- Les chiffres arabes de 0 à 9,
- Les 26 lettres de l'alphabet latin en minuscules et en capitales,
- Des symboles mathématiques et de ponctuation.

Le codage ASCII suffit pour représenter les textes en anglais, mais il est trop limité pour les autres langues, dont le français et ses lettres accentuées.

3.4.1 Principe du codage ASCII

Le code ASCII définit 128 caractères numérotés de 0 à 127 et codés en binaire de $(00000000)_2$ à $(11111111)_2$. Sept bits suffisent donc. Toutefois, les ordinateurs travaillant presque tous sur un multiple de huit bits (*un octet*) depuis les années 1970, chaque caractère d'un texte en ASCII est souvent stocké dans un octet dont le 8^e bit est 0.

Les caractères de numéro 0 à 31 et le 127 ne sont pas affichables ; ils correspondent à des commandes de contrôle de terminal informatique. Le caractère numéro 127 est la commande pour effacer. Le caractère numéro 32 est l'espace. Le caractère 7 provoque

l'émission d'un signal sonore. Les autres caractères sont les chiffres arabes, les lettres latines majuscules et minuscules sans accent, des symboles de ponctuation, des opérateurs mathématiques et quelques autres symboles. La Figure 3.5 représente la table des codes ASCII standards codés sur 7-bits.

MSB LSB		0 000	1 001	2 010	3 011	4 100	5 101	6 110	7 111
0	0000	NUL	DLE	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	}
C	1100	FF	FS	,	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	{
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

FIGURE 3.5 – Table du code ASCII standard sur 7-bits

3.4.2 Limitation du codage ASCII

L'absence des caractères des langues étrangères à l'anglais rend ce standard insuffisant à lui seul pour des textes étrangers (*par exemple en langue française*), ce qui rend nécessaire l'utilisation d'autres encodages.

3.5 Le codage ASCII étendu

Le code ASCII ayant été établi par des Américains, dont la langue ne comporte pas d'accents, les caractères accentués du français et les signes diacritiques nécessaires à d'autres langues n'y étaient pas prévus. C'est ce qui a conduit les organismes de normalisation à définir d'autres codages plus étoffés, qui ajoutent à l'**ASCII** les caractères

qui manquent à une langue, un pays ou une culture.

Par commodité, et comme les 128 premiers caractères restent souvent identiques à ceux de l'ASCII, on parle alors d'**ASCII étendu**. Cette appellation un peu fourre-tout recouvre en fait différents types de codage, comme la norme ISO-8859-1 sur 8 bits (*pour un codage maximal de 256 caractères*), qu'on appelle aussi **Latin-1** ou Europe occidentale. Ou encore le plus ambitieux codage Unicode que nous verrons juste après. Tous les systèmes d'exploitation sont capables de basculer d'un jeu de caractères à l'autre.

La Figure 3.6 représente la table de codage ISO-8859-1, parfois présentée comme l'ASCII étendu.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
8x	PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTJ	PLD	PLU	RI	SS2	SS3
9x	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SGCI	SCI	CSI	ST	OSC	PM	APC
Ax	NBSP	ı	ć	£	¤	¥	¦	§	¨	©	ª	«	¬	­	®	¯
Bx	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
Cx	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Dx	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
Ex	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
Fx	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

FIGURE 3.6 – Table de codage ISO-8859-1

3.6 Le codage UNICODE

Unicode est un standard informatique qui permet des échanges de textes dans différentes langues, à un niveau mondial. Il est développé par le Consortium Unicode qui est une organisation privée sans but lucratif qui coordonne le développement du standard Unicode. Le standard Unicode vise au codage de texte écrit en donnant à tout caractère de n'importe quel système d'écriture un nom et un identifiant numérique, et ce de manière unifiée, quels que soient la plateforme informatique ou le logiciel utilisé.

Unicode est une alternative qui couvre un large éventail de caractères. Il contient des représentations de lettres dans des langues telles que l'anglais, le grec, l'arabe, etc., des symboles mathématiques, des emoji et bien d'autres.

Unicode est disponible en trois types d'encodage. Ils sont **UTF-8**, **UTF-16** et **UTF-**

32. UTF-8 utilise 8-bits par caractère, UTF-16 utilise 16-bits par caractère et UTF-32 utilise 32-bits pour un caractère. En UTF-8, les 128 premiers caractères sont les caractères ASCII. Par conséquent, ASCII est valide dans UTF-8. Unicode est généralement utilisé pour l'internationalisation et la localisation de logiciels. Cette norme est également utilisée dans les systèmes d'exploitation, XML, .NET Framework et les langages de programmation tels que Java.

Remarque : Unicode est un sur-ensemble de l'ASCII

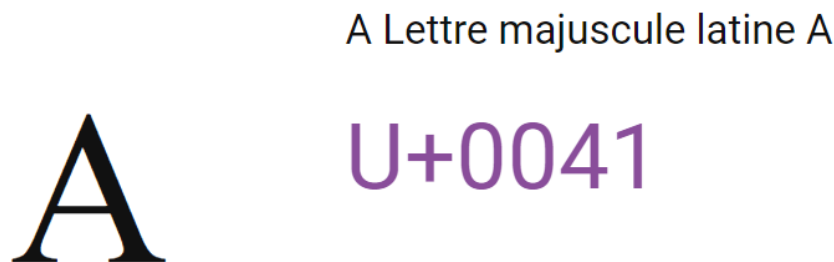


FIGURE 3.7 – Codage de la lettre majuscule latine **A** en Unicode UTF-16BE

Codage	hex	dec (bytes)	dec	binary
UTF-8	41	65	65	01000001
UTF-16BE	00 41	0 65	65	00000000 01000001
UTF-16LE	41 00	65 0	16640	01000001 00000000
UTF-32BE	00 00 00 41	0 0 0 65	65	00000000 00000000 00000000 01000001
UTF-32LE	41 00 00 00	65 0 0 0	1090519040	01000001 00000000 00000000 00000000

FIGURE 3.8 – Codage de la lettre majuscule latine **A** en Unicode (*plusieurs types d'encodage*)

Lien : <https://unicode-table.com/fr/blocks/basic-latin/>

3.7 Le codage 7-segments

Les écrans 7 segments de type LED (*Light Emitting Diode*) ou LCD (*Liquid Crystal Display*) constituent un moyen très pratique d'afficher des informations ou des données numériques sous forme de chiffres, de lettres ou même de caractères alphanumériques.

En général, les afficheurs à 7-segments sont constitués de sept diodes électroluminescentes de couleur (*appelées segments*), dans un seul boîtier d'affichage. Afin de produire les chiffres ou les caractères HEX de 0 à 9 et de *A* à *F* respectivement, sur l'écran, la combinaison correcte de segments de LED doit être allumée (*les décodeurs d'affichage BCD à 7-segments tels que le 74LS47 font exactement cela*).

Un afficheur LED standard à 7-segments possède généralement huit connexions d'entrée, une pour chaque segment (**LED**) et une qui sert de borne ou de connexion commune pour tous les segments internes de l'afficheur. Certains afficheurs simples ont également une broche d'entrée supplémentaire pour afficher un point décimal dans leur coin inférieur droit ou gauche.

En électronique, il existe deux types importants d'afficheurs numériques LED à 7-segments (Figure 3.9).

1. **L'affichage à cathode commune (CCD)** : Dans l'affichage à cathode commune, toutes les connexions de cathode des LEDs sont reliées au "0" logique ou à la masse. Les segments individuels sont éclairés par l'application d'un signal "HIGH", logique "1", aux bornes individuelles de l'anode.
2. **L'affichage à anode commune (CAD)** : Dans l'affichage à anode commune, toutes les connexions anodiques des LEDs sont reliées au niveau logique "1" et les segments individuels sont éclairés en reliant les bornes cathodiques individuelles à un signal "BAS", logique "0".

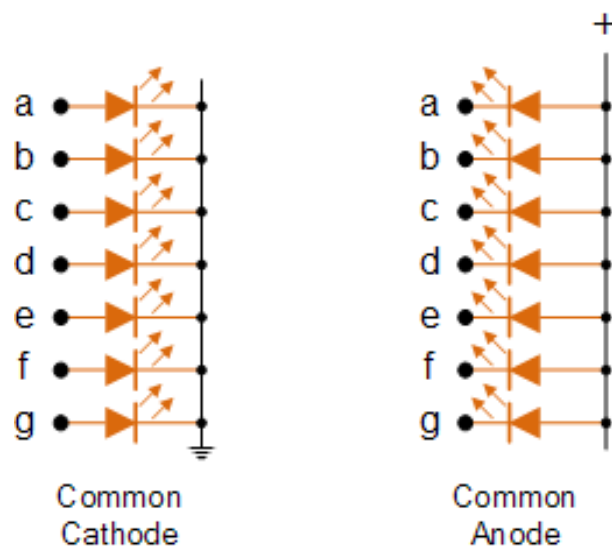


FIGURE 3.9 – Format de la cathode commune et de l'anode commune pour un afficheur 7-segments

En connectant électriquement les diodes individuelles pour un affichage à cathode commune et un affichage à anode commune et en illuminant chaque diode électroluminescente individuellement, on peut faire en sorte qu'elles affichent une variété de chiffres, de caractères ou de symboles (voir Figures 3.10 et 3.11).

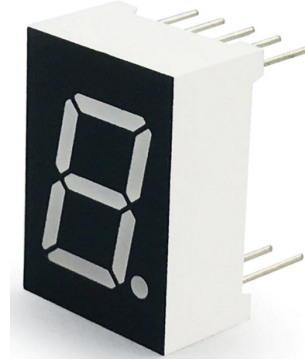
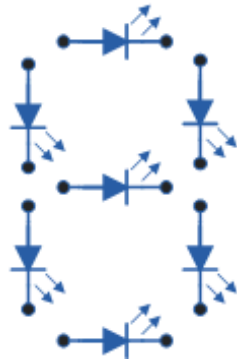
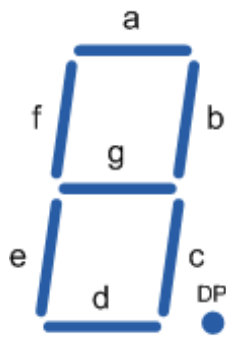


FIGURE 3.10 – Forme géométrique d'un afficheur 7-segment

FIGURE 3.11 – Afficheur 7-segments utilisé dans la pratique

Ainsi, pour afficher le nombre "6" par exemple, les segments **a**, **c**, **d**, **e**, **f** et **g** doivent être allumés. Si nous voulions afficher un autre chiffre ou une autre lettre, un autre ensemble de segments devrait être allumé. Pour un affichage à 7-segments, nous pouvons produire une table de vérité indiquant les segments qui doivent être allumés afin de produire le caractère requis, comme indiqué dans la Figure 3.12.

Segments Individuels							Affichage 7-segments
g	f	e	d	c	b	a	
0	1	1	1	1	1	1	0
0	0	0	0	1	1	0	1
1	0	1	1	0	1	1	2
1	0	0	1	1	1	1	3
1	1	0	0	1	1	0	4
1	1	0	1	1	0	1	5
1	1	1	1	1	0	1	6
0	0	0	0	1	1	1	7

Segments Individuels							Affichage 7-segments
g	f	e	d	c	b	a	
1	1	1	1	1	1	1	8
1	1	0	1	1	1	1	9
1	1	1	0	1	1	1	0
1	1	1	1	1	0	0	b
0	1	1	1	0	0	1	c
1	0	1	1	1	1	0	d
1	1	1	1	0	0	1	e
1	1	1	0	0	0	1	f

FIGURE 3.12 – Table de vérité pour un affichage à 7 segments

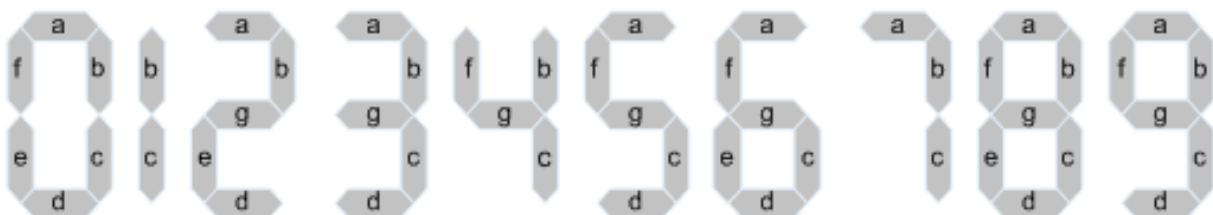


FIGURE 3.13 – Combinaisons d'affichage à 7-segments pour tous les chiffres

On peut voir que pour afficher un chiffre unique de 0 à 9 en binaire ou des lettres de A à F en hexadécimal, il faudrait sept connexions de segments distinctes plus une connexion supplémentaire pour la connexion "*commune*" des LEDs. De plus, comme les segments sont essentiellement des diodes électroluminescentes standard, le circuit de commande devrait produire jusqu'à $20mA$ de courant pour éclairer chaque segment individuel et pour afficher le nombre "8", les sept segments devraient être allumés, ce qui entraînerait un courant total de près de $140mA$, ($8 \times 20mA$).

De toute évidence, l'utilisation d'un si grand nombre de connexions et la consommation d'énergie ne sont pas pratiques pour certains circuits électroniques ou à microprocesseur. Afin de réduire le nombre de lignes de signaux nécessaires pour piloter un seul affichage, des décodeurs d'affichage tels que le (*BCD to 7-Segment Display Decoder and Driver IC's*) sont utilisés à la place.

Chapitre 4

Opérations Arithmétiques dans le Système Binaire

4.1 Représentation des nombres

4.1.1 Nombres signés / non signés

Notre système de nombres s'étend de l'infini négatif ($-\infty$) à l'infini positif ($+\infty$), zéro au milieu.

$$-\infty \longrightarrow 0 \longrightarrow +\infty$$

Cependant, dans la programmation, il existe essentiellement deux types de nombres ; le signé et le non signé. La principale différence entre un nombre signé et un nombre non signé est, bien, la capacité d'utiliser des nombres négatifs. Les nombres non signés ne peuvent avoir que des valeurs de zéro ou plus ($\mathbf{N}$). En revanche, les nombres signés sont plus naturels avec une plage qui inclut des nombres négatifs à positifs ($\mathbf{Z}$).

La création de nombres signés et non signés a été motivée par la nécessité de représenter des valeurs numériques avec des ressources très limitées. Par exemple, en utilisant 8 bits, vous n'avez qu'un total de 256 combinaisons. Avec un numéro non signé, cela signifie toute valeur comprise entre 0 et 255. généralement, une représentation des nombres non signés sur n bits peut représenter tous les entiers de l'intervalle :

$$[0, 2^n - 1] \tag{4.1}$$

- 8-bits : de $[0, 255]$
- 16-bits : de $[0, 65.535]$
- 32-bits : de $[0, 4.294.967.295]$

En revanche, un numéro signé signifie que vous perdez déjà un bit pour représenter le signe. Avec 7 bits, vous avez un maximum de 128 combinaisons. Votre gamme avec un numéro signé de 8 bits est donc comprise entre -128 et 127 . Donc, si vous avez des ressources limitées, il était préférable d'utiliser des nombres non signés. Généralement, une représentation des nombres non signés sur n bits peut représenter tous les nombres relatifs de l'intervalle :

$$[-2^{n-1}, 2^{n-1} - 1] \quad (4.2)$$

- 8-bits : de $[-128, +127]$
- 16-bits : de $[-32.768, +32.767]$
- 32-bits : de $[-2.147.483.648, +2.147.483.647]$

Lors de l'utilisation de nombres signés et non signés dans la programmation, il est préférable de n'utiliser que l'un ou l'autre, car l'utilisation simultanée des deux peut entraîner des problèmes :

- Le premier est lorsque vous comparez un numéro non signé à un numéro signé. Cela se traduit généralement par des avertissements, mais le compilateur peut le compiler de toute façon.
- Le deuxième problème, plus grave, concerne l'affectation du contenu d'un numéro non signé à un numéro signé. Si la valeur du nombre non signé est supérieure à la valeur maximale du nombre signé, cela entraînerait une erreur.

Remarque : Avec le développement de la technologie, les ressources sont devenues plus abondantes et l'utilisation de nombres non signés devient de moins en moins nécessaire. Pour illustrer cela, gardez à l'esprit qu'un nombre long de 32-bits peut avoir une valeur maximale de 2 ou 4 millions si elle n'est pas signée. Lorsque vous passez en 64-bits, la différence se situe entre 90 et 180 quintillions ; valeurs qui sont rarement utilisées, voire pas du tout, dans les programmes.

Résumé :

1. Les numéros non signés comprennent uniquement les chiffres zéro et positifs, tandis que les numéros signés comprennent les nombres négatifs.
2. Les numéros signés ont la moitié de la valeur maximale des numéros non signés.
3. Le mélange de chiffres signés et non signés peut entraîner des problèmes.
4. Utiliser des nombres signés ou non signés a peu d'influence dans les applications modernes.

Envisageons maintenant les nombres entiers relatifs ou autrement dit, munis d'un signe (+) ou (−)

En décimal,

- +1, +2, +3 etc. sont des nombres positifs. Ils sont supérieurs à 0 ($n > 0$)
- −1, −2, −3 etc. sont des nombres négatifs. Ils sont inférieurs à 0 ($n < 0$)

De même en binaire,

- +1, +10, +11, +100, +101 etc. sont des nombres binaires positifs.
- −1, −10, −11, −100, −101 etc. sont des nombres binaires négatifs.

Le problème est que les circuits électroniques digitaux ne peuvent enregistrer que des 0 ou des 1 mais pas de signes (+) ou (−). Le seul moyen est alors de convenir que si un nombre est susceptible d'être négatif, on lui réserve un bit pour indiquer le signe (*Figures 4.1 et 4.2*). Reste à déterminer le bit qui dans un nombre binaire conviendrait le mieux pour symboliser le signe et quelle valeur de ce bit (0 ou 1) conviendrait le mieux pour représenter le signe "*plus*" ou le signe "*moins*". Observons d'abord le fait que les nombres

7 6 5 4 3 2 1 0
b 0 1 0 1 1 1 0 1
 MSB LSB

7 6 5 4 3 2 1 0
b 1 1 0 1 0 1 0 0
 MSB LSB

FIGURE 4.1 – Un nombre binaire positif sur 8 bits

FIGURE 4.2 – Un nombre binaire négatif sur 8 bits

codés en machine ont une dimension fixe, ainsi que sur papier, les nombres ont des dimensions variables :

- L'addition de deux nombres de 2 chiffres donne un nombre de 2 ou 3 chiffres.
- La multiplication de deux nombres de 2 chiffres donne des nombres de 3 ou 4 chiffres.

En machine par contre, les nombres ne sont pas extensibles. Ils ont des dimensions fixes. Dans une voiture par exemple, le compteur kilométrique s'il ne possède que 6 chiffres ne pourra indiquer plus de 999.999km.

De même, dans les ordinateurs les nombres (*binaires*) ont aussi des dimensions fixes de 1, 2, 4 ou 8 octets.

Le bit le plus à gauche du mot binaire est celui qui va représenter le signe. Signe négatif si ce bit vaut 1, signe positif quand ce bit vaut 0.

Si on admet que le nombre peut représenter des valeurs négatives, on parle de nombres "signés".

Comme pour les nombres "non signés", on peut représenter $2^8 = 256$ codes avec 8 bits (*voir l'expression 4.1*).

mais ici le bit de gauche est le signe

- 1 = signe moins (−)

- 0 = signe plus (+)

Il y a donc moyen de représenter (*voir l'expression 4.2*) :

- 128 codes avec le bit de signe à 1, ce sont 128 nombres négatifs (de -1 à -128)
- 128 codes avec le bit de signe à 0 le nombre 0 et 127 nombres positifs (de $+1$ à $+127$)

4.1.2 Complément à un

Le complément à un d'un nombre binaire est la valeur obtenue en inversant tous les bits de ce nombre (*en permutant les 0 par des 1 et inversement*) (voir Figure 4.3). Le complément à un d'un nombre se comporte alors comme le négatif du nombre original dans certaines opérations arithmétiques.

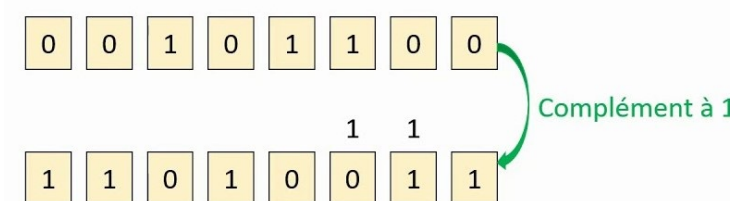


FIGURE 4.3 – Principe de calcul du complément à 1

Dans le système de complément à un, la valeur 0 a deux représentations : « $+0$ » et « -0 » (exemple sur 4 bits : 0000 et 1111), ce qui oblige à réaliser deux tests pour tester la valeur nulle d'un résultat. Afin de pallier ce défaut, on a introduit la représentation par complément à deux.

4.1.3 Complément à deux

En informatique, le complément à deux est une méthode de représentation des entiers relatifs en binaire permettant d'effectuer simplement des opérations arithmétiques.

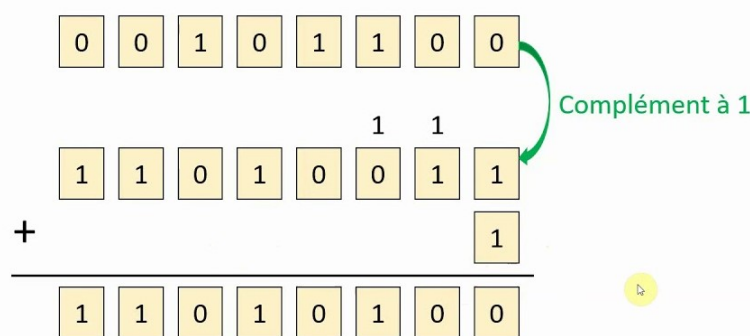


FIGURE 4.4 – Principe de calcul du complément à 2

Le complément à deux ne s'applique qu'à des nombres ayant tous la même longueur : avec un codage sur n bits, cette méthode permet de représenter toutes les valeurs entières de :

$$-2^{n-1} \Rightarrow 2^{n-1} - 1$$

La Figure 4.5 illustre une deuxième procédure à suivre pour trouver le complément à deux d'un nombre binaire code sur n -bits. Il suffit de parcourir la représentation binaire en partant du bit le moins significatif (LSB) vers le plus significatif (MSB). On recopie tous les bits jusqu'à rencontrer un 1. Ensuite, on recopie ce 1 et pour tous les bits restants, on les recopie en les inversant : pour les 1 on note donc 0 et inversement.

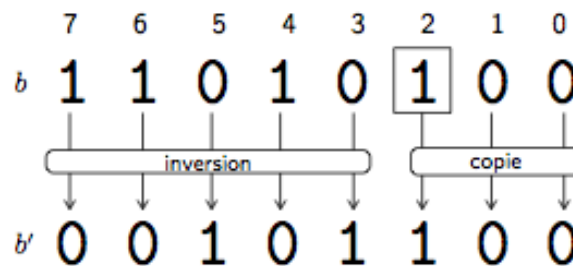


FIGURE 4.5 – Deuxième méthode de calcul du complément à 2

4.1.4 Nombre à virgule fixe

En informatique, une représentation d'un nombre en virgule fixe est un type de donnée correspondant à un nombre qui possède (*en base deux ou en base dix*) un nombre fixe de chiffres après la virgule. Les nombres en virgule fixe sont utiles pour représenter des quantités fractionnaires dans un format utilisant le complément à deux quand le processeur de l'ordinateur n'a aucune unité de calcul en virgule flottante ou quand une virgule fixe permet d'augmenter la vitesse d'exécution ou d'améliorer l'exactitude des calculs. La plupart des processeurs à faible coût (ex. : *microcontrôleurs*) ne disposent pas d'unité de calcul en virgule flottante.

Les bits à gauche de la virgule représentent la partie entière du nombre, c'est-à-dire l'entier se trouvant à gauche de la virgule. Chaque bit à droite de la virgule, ou « décimale binaire », correspond à l'inverse d'une puissance de 2. Ainsi la première décimale binaire est $\frac{1}{2}$, la seconde est $\frac{1}{4}$, la troisième est $\frac{1}{8}$ et ainsi de suite. Pour un nombre en virgule fixe dans un format de complément à deux, la borne maximale est :

$$2^{e-1} - \frac{1}{2^f} \quad (4.3)$$

et la borne minimale est :

$$-2^{e-1} \quad (4.4)$$

où e correspond au nombre de bits de la partie entière signée et f au nombre de bits

de la partie fractionnaire.

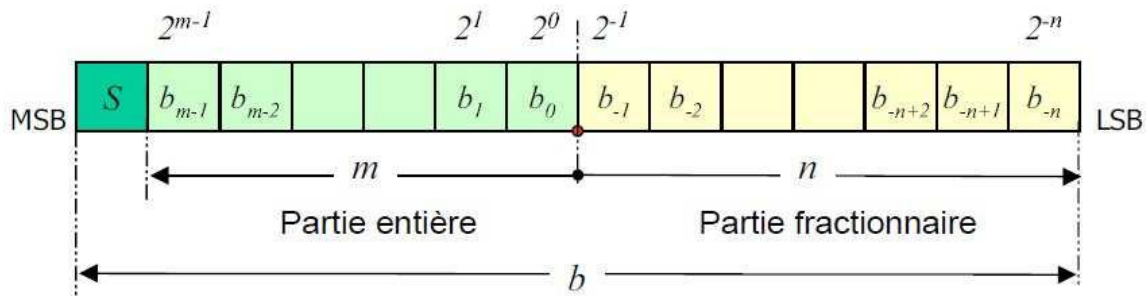


FIGURE 4.6 – Représentation d'un nombre binaire en virgule fixe

4.1.5 Nombre a virgule flottante

La virgule flottante est une méthode d'écriture de nombres fréquemment utilisée dans les ordinateurs, équivalente à la notation scientifique en numération binaire. Elle consiste à représenter un nombre par :

- Un signe (égal à -1 ou +1) ;
- Une mantisse (aussi appelée significande) ;
- Un exposant (entier relatif, généralement borné).

Un tel triplet représente le nombre (Équation 4.5) :

$$(\text{Nombre})_b = \text{Mantisse} \times b^{\text{Exposant}} \quad (4.5)$$

Le bit de poids fort est le bit de signe, si ce bit est à 1, le nombre est négatif, et s'il est à 0, le nombre est positif. Les e bits suivants représentent l'exposant biaisé (*sauf valeur spéciale*), et les m bits suivants (m bits de poids faible) représentent la mantisse.



FIGURE 4.7 – Format Général IEEE754

Biais de l'exposant : L'exposant peut être positif ou négatif. Cependant, la représentation habituelle des nombres signés (*complément à 2*) rendrait la comparaison entre les nombres flottants un peu plus difficile. Pour régler ce problème, l'exposant est « **biaisé** », afin de le stocker sous forme d'un nombre non signé.

Ce biais est de $2^{e-1} - 1$ (e représente le nombre de bits de l'exposant) ; il s'agit donc d'une valeur constante une fois que le nombre de bits e est fixé.

L'interprétation d'un nombre (autre qu'infini) est donc :

$$Valeur = Signe \times Mantisse \times 2^{(exposant - biais)} \quad (4.6)$$

avec :

- signe = ± 1
- biais = $2^{e-1} - 1$

Pour un nombre flottant en base 10, on a l'expression d'écriture suivante (Équation 4.7) :

$$(Nombre)_{10} = Mantisse \times 10^{Exposant} \quad (4.7)$$

Et pour un nombre flottant en base 2, on a l'expression d'écriture suivante (Équation 4.8) :

$$(Nombre)_2 = Mantisse \times 2^{Exposant} \quad (4.8)$$

La base de représentation est généralement 2 sur ordinateur, mais aussi 8 ou 16 sur certaines anciennes machines, 10 sur de nombreuses calculatrices, ou éventuellement toute autre valeur. En faisant varier l'exposant, on fait « **flotter** » la virgule. La mantisse est une suite de chiffres en base **b**, généralement de taille fixée. La valeur de l'« exposant » indique le multiplicateur, c'est-à-dire la position de la virgule virtuelle.

La virgule flottante s'oppose à la représentation en virgule fixe, qui indique un nombre entier d'un sous-multiple de l'unité : « **1234,567** » est une façon commode d'écrire 1234 et 567 millièmes, soit 1234567 millièmes1.

La représentation en virgule flottante peut, avec le même nombre de bits, gérer un intervalle numérique plus important.

Par exemple, une représentation décimale en virgule fixe ayant 7chiffres décimaux dont 2 après la virgule peut représenter les nombres 12345.67, 123.45, 1.23, etc.

La représentation décimale en virgule flottante (comme le format **IEEE 754 decimal32**) peut, avec le même nombre de chiffres décimaux, représenter en plus 1.234567, 123456.7 = 1.234567×10^5 , $0.00001234567 = 1.234567 \times 10^{-5}$, $1234567000000000 = 1.234567 \times 10^{15}$, etc.

La représentation en virgule flottante préserve la précision. La mantisse a toujours le même nombre de chiffres significatifs, sans zéro au début.

la norme IEEE 754

En informatique, l'**IEEE 754** est une norme sur l'arithmétique à virgule flottante mise au point par le Institute of Electrical and Electronics Engineers. Elle est la norme la plus employée actuellement pour le calcul des nombres à virgule flottante avec les CPU et les FPU. La norme définit les formats de représentation des nombres à virgule flottante (signe, mantisse, exposant, nombres dénormalisés) et valeurs spéciales (infinis et NaN),

en même temps qu'un ensemble d'opérations sur les nombres flottants. Il décrit aussi cinq modes d'arrondi et cinq exceptions (comprenant les conditions dans lesquelles une exception se produit, et ce qui se passe dans ce cas).

la FPU est une unité de calcul en virgule flottante (*UVF*, en anglais *floating-point unit*, *FPU*) est une partie d'un processeur, spécialement conçue pour effectuer des opérations sur des nombres à virgule flottante.

La norme IEEE 754 spécifie deux formats de nombres en virgule flottante en base 2, ainsi que quelques opérations associées : principalement l'addition, la soustraction, la multiplication, la division et la racine carrée. La quasi-totalité des architectures d'ordinateurs actuelles, y compris x86, PowerPC, et AMD64, incluent une implémentation matérielle des calculs sur flottants IEEE, directement dans le microprocesseur, garantissant une exécution rapide.

Les deux formats fixés par la norme IEEE 754 sont sur 32 bits (« simple précision ») et sur 64 bits (« double précision »). La répartition des bits est la suivante :

Format simple précision

Un nombre flottant simple précision est stocké dans un mot de 32 bits : 1 bit de signe, 8 bits pour l'exposant et 23 pour la mantisse.

L'exposant est donc biaisé de $2^{8-1} - 1 = 127$ dans ce cas. L'exposant d'un nombre normalisé va donc de -126 à $+127$. L'exposant -127 (*qui est biaisé vers la valeur 0*) est réservé pour zéro et les nombres dénormalisés, tandis que l'exposant 128 (*biaisé vers 255*) est réservé pour coder les **infinis** et les **NaN**



FIGURE 4.8 – Format simple précision (32-bits)

Un nombre flottant normalisé a une valeur v donnée par la formule suivante :

$$v = s \times 2^e \times m \quad (4.9)$$

- $s = \pm 1$ représente le signe (*selon le bit de signe*).
- e est l'exposant avant son biais de 127 ;
- $m = 1 + \text{mantisse}$ représente la partie significative (*en binaire*), d'où $1 \leq m < 2$ (*mantisse étant la partie décimale de la partie significative, comprise entre 0 et 1*)

Par exemple, pour :

0b00111111000100000000000000000000

: le signe est positif, l'exposant est $124 - 127 = -3$, et la partie significative est 0b1,01

soit 1,25 en décimal ($1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$) ; le nombre représenté est donc $+1,25 \times 2^{-3}$ soit +0,15625.

Format double précision

Le format double précision est identique au simple précision, mis à part le fait que les champs sont plus grands. En effet, il possède 52 *bits* de mantisse au lieu de seulement 23, et 11 bits d'exposant au lieu de seulement 8.



FIGURE 4.9 – Format double précision (64-bits)

La mantisse est très élargie, alors que l'exposant est peu élargi. Ceci est dû au fait que, selon les créateurs de la norme, la précision est plus importante que l'amplitude.

Les **NaN** et les **infinis** sont représentés en mettant tous les bits de l'exposant à 1 (2047), mais distingués en mettant tous les 52 *bits* de la mantisse à 0 pour les infinis et au moins un de ces 52 *bits* à 1 pour les NaN.

Pour les nombres normalisés, le biais de l'exposant est +1023. Pour les nombres dé-normalisés, l'exposant est -1022 (*l'exposant minimum pour un nombre normalisé*). Ce n'est pas -1023 car les nombres normalisés ont un 1 avant la virgule, et les nombres dénormalisés n'en ont pas. Comme précédemment, zéro et l'infini sont signés.

4.2 Opérations arithmétiques

4.2.1 Addition binaire

L'addition de nombres signés est utilisée dans la plupart des circuits numériques, trois cas peuvent se présenter :

- l'addition de deux nombres positifs,
- l'addition de deux nombres de signes contraires,
- l'addition de deux nombre négatifs,

La méthode consiste à écrire les nombres positifs en notation exacte et à remplacer les nombres négatifs par leur complément à deux avant d'additionner. Si le résultat est positif, il est en notation exacte, s'il est négatif, il est en notation complément à deux.

Addition de deux nombres positifs

Effectuons l'addition de $(+17)_{10}$ et $(+12)_{10}$ l'écriture de l'addition est donnée par la Figure 4.10 :

$$\begin{array}{rcl}
 \text{1}^{\text{er}} \text{ nombre } (+17)_{10} & & \mathbf{0} \ 10001 \\
 \text{2}^{\text{eme}} \text{ nombre } (+12)_{10} & + & \mathbf{0} \ 01100 \\
 \hline
 \text{Résultat} & & \mathbf{0} \ 11101 \\
 & \text{Bit de signe} & \text{---}
 \end{array}$$

FIGURE 4.10 – Exemple d'addition de deux nombres positifs

Addition de deux nombres négatifs

Effectuons l'addition de $(-17)_{10}$ et $(-12)_{10}$ Les compléments à deux s'écrivent :

$$(-17)_{10} \Rightarrow \mathbf{1}01111$$

$$(-12)_{10} \Rightarrow \mathbf{1}10100$$

l'écriture de l'addition est donnée par la Figure 4.11 :

$$\begin{array}{rcl}
 & \text{Report} & \mathbf{1111} \\
 \text{1}^{\text{er}} \text{ nombre } (-17)_{10} & & \mathbf{1} \ 01111 \\
 \text{2}^{\text{eme}} \text{ nombre } (-12)_{10} & + & \mathbf{1} \ 10100 \\
 \hline
 \text{Résultat} & & \mathbf{11} \ 00011 \\
 & \text{Débordement } a & \text{---} \\
 & \text{éliminer} & \uparrow \\
 & & \text{bit de signe}
 \end{array}$$

FIGURE 4.11 – Exemple d'addition de deux nombres négatifs

Cette somme est négative, le résultat est donc le complément à deux du résultat cherché qui s'écrit en notation exacte :

$$(111101)_2 = (-29)_{10}$$

Addition de deux nombres de signes différents

Deux cas se présentent :

Cas 1 : le nombre positif est supérieur du nombre négatif.

Effectuons l'addition des nombres suivants :

$$(+17)_{10} \Rightarrow \mathbf{0}10001$$

$$(-12)_{10} \Rightarrow \mathbf{1}10100$$

Effectuons l'addition :

$$\begin{array}{rcl}
 & \text{Report} & 11 \\
 1^{\text{er}} \text{ nombre } (+17)_{10} & & \mathbf{0} \ 10001 \\
 2^{\text{eme}} \text{ nombre } (-12)_{10} & + & \mathbf{1} \ 10100 \\
 \hline
 \text{Résultat} & \rightarrow & \mathbf{10} \ 00101 \\
 & \text{Débordement } a \text{ ---} & \uparrow \\
 & \text{éliminer} & \text{bit de signe}
 \end{array}$$

FIGURE 4.12 – Exemple d'addition de deux nombres de signes contraires (cas 1)

Nous avons additionner les bits de signe et la retenue, cela peut entraîner un débordement qui est toujours rejeté. Ce résultat est général, la somme étant positive, et le résultat est en notation exacte :

$$(000101)_2 = (+5)_{10}$$

Cas 2 : le nombre positif est inférieur du nombre négatif.

Effectuons l'addition des nombres suivants :

$$(-17)_{10} \Rightarrow 110001$$

$$(+12)_{10} \Rightarrow 001100$$

Le complément à deux de $(-17)_{10}$ s'écrit :

$$(-17)_{10} \Rightarrow 101111$$

L'écriture de l'addition est alors la suivante :

$$\begin{array}{rcl}
 & \text{Report} & 11 \\
 1^{\text{er}} \text{ nombre } (-17)_{10} & & \mathbf{1} \ 01111 \\
 2^{\text{eme}} \text{ nombre } (+12)_{10} & + & \mathbf{0} \ 01100 \\
 \hline
 \text{Résultat} & & \mathbf{1} \ 11011 \\
 & & \uparrow \\
 & & \text{bit de signe}
 \end{array}$$

FIGURE 4.13 – Exemple d'addition de deux nombres de signes contraires (cas 2)

Cette somme est négative, le résultat est le complément à deux du total cherché qui s'écrit en notation exacte :

$$(100101)_{10} = (-5)_{10}$$

Chapitre 5

Logique combinatoire

La logique combinatoire tout comme l'arithmétique repose sur quelques opérations élémentaires.

En arithmétique, ces opérations sont l'addition, la soustraction, la multiplication et la division ($+$, $-$, $\times$, $\div$). Il est possible à partir de là d'imaginer toutes les autres opérations telles que les exposants, les racines, les logarithmes etc.

En logique, les opérations fondamentales sont le **ET**, le **OU** et le **NON**. La manière la plus simple de comprendre les fonctions logiques est de se les représenter par des schémas électriques qui comportent un ou plusieurs boutons poussoirs et une lampe. Cette lampe s'allume à condition que les contacts électriques y laissent passer le courant.

5.1 Opérations logiques de base

5.1.1 Opérateur OUI

Le schéma ci-dessous traduit la condition la plus simple, la lampe s'allume si le bouton poussoir A est actionné. Autrement dit ($S = 1$) si ($A = 1$). Le fonctionnement de ce circuit s'exprime par l'équation logique :

$$S = A$$

Il n'y a que deux cas possibles. Ils sont représentés dans la table de vérité.

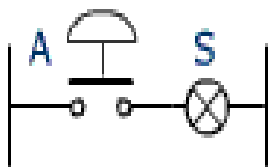


FIGURE 5.1 – Schéma électrique de la fonction OUI

A	S
0	0
1	1

FIGURE 5.2 – Table de vérité de la fonction OUI

Une table de vérité a pour le rôle de montrer la correspondance entre la sortie et toutes les combinaisons de valeurs que peuvent prendre la ou les entrées.

5.1.2 Opérateur ET

Plaçons maintenant deux contacts dans le circuit. La condition nécessaire pour allumer la lampe dépend de la manière dont les contacts sont connectés. Suivant les cas, la condition pour allumer la lampe fait appel aux opérateurs logiques ET ou OU.

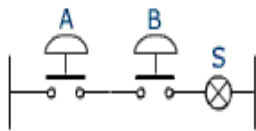


FIGURE 5.3 – Schéma électrique de la fonction ET

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

FIGURE 5.4 – Table de vérité de la fonction ET

La lampe s'allume si on active simultanément les contacts A et B : ($S = 1$) si ($A = 1$) ET ($B = 1$) L'opérateur ET est représenté dans l'équation logique par un point. Ce signe convient parfaitement puisque la fonction ET donne le même résultat qu'une multiplication.

$$S = A.B$$

5.1.3 Opérateur OU

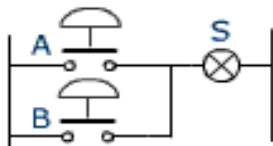


FIGURE 5.5 – Schéma électrique de la fonction OU

A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

FIGURE 5.6 – Table de vérité de la fonction OU

La lampe s'allume si on active le contact A ou le contact B : ($S = 1$) si ($A = 1$) OU ($B = 1$)

L'opérateur OU est représenté dans l'équation logique par un signe plus surmonté d'un point. Observons les trois premières lignes de la table de vérité, le résultat de l'opération OU y est semblable au résultat d'une addition. Le résultat de 1 ou 1 diffère cependant de 1+1. Nous mettons un point au-dessus du signe '+' pour indiquer que l'opération n'est pas qu'une simple addition.

$$S = A + B$$

5.1.4 Opérateur NON

Les contacts que nous avons utilisés jusqu'ici, sont des contacts "normalement ouverts". Quand le bouton poussoir est relâché (quand $A = 0$) le courant ne passe pas.

Nous utilisons maintenant un contact "normalement fermé" pour illustrer la fonction NON. Au repos, le courant passe mais il se coupe quand le contact est activé (quand $A = 1$).

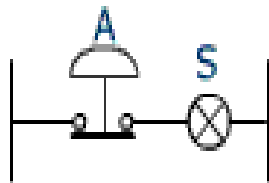


FIGURE 5.7 – Schéma électrique de la fonction NON

A	S
0	1
1	0

FIGURE 5.8 – Table de vérité de la fonction NON

$$S = \overline{A}$$

Lu : $S = NonA$

5.1.5 Combinaisons de fonctions logiques

Les trois fonctions de base que nous venons de voir se combinent de multiples façons. A chaque schéma imaginable correspond une équation. La correspondance entre un schéma et une fonction logique est systématique.

- Des contacts en parallèle correspondent à la fonction OU.
- Des contacts en série correspondent à la fonction ET
- Un contact normalement fermé représente la fonction NON

Exemples :

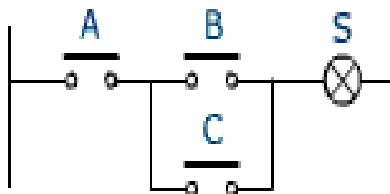


FIGURE 5.9 – Exemple 1

A	B	C	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

FIGURE 5.10 – TV de l'exemple 1

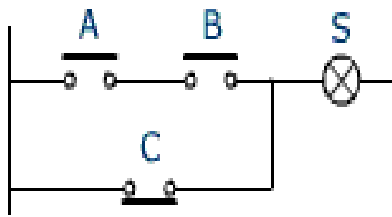


FIGURE 5.11 – Exemple 2

A	B	C	S
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

FIGURE 5.12 – TV de l'exemple 2

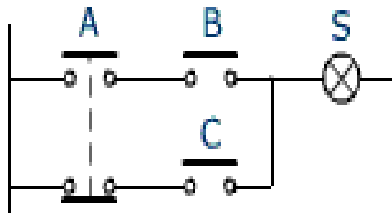


FIGURE 5.13 – Exemple 3

A	B	C	S
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

FIGURE 5.14 – TV de l'exemple 3

5.2 Autres Opérations logiques

5.2.1 Opérateur NON-ET (NAND)

La fonction NON-ET (NAND en anglais) est un opérateur logique de l'algèbre de Boole. À deux opérandes, qui peuvent avoir chacun la valeur VRAI ou FAUX, il associe un résultat qui a lui-même la valeur VRAI seulement si au moins l'un des deux opérandes a la valeur FAUX.

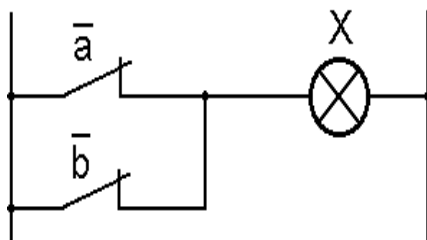


FIGURE 5.15 – Schéma électrique de la fonction logique NAND

b	a	X
0	0	1
0	1	1
1	0	1
1	1	0

FIGURE 5.16 – Table de vérité de la fonction logique NAND

$$X = \overline{a.b}$$

5.2.2 Opérateur NON-OU (NOR)

La fonction NON-OU (NOR en anglais) est un opérateur logique de l'algèbre de Boole. À deux opérandes, qui peuvent avoir chacun la valeur VRAI ou FAUX, il associe un

résultat qui a lui-même la valeur VRAI seulement si les deux opérandes ont la valeur FAUX.

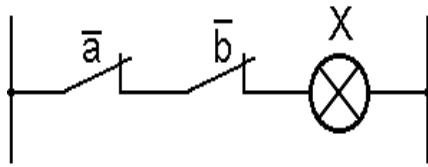


FIGURE 5.17 – Schéma électrique de la fonction logique NOR

b	a	X
0	0	1
0	1	0
1	0	0
1	1	0

FIGURE 5.18 – Table de vérité de la fonction logique NOR

$$X = \overline{a + b}$$

5.2.3 Opérateur OU EXCLUSIF (XOR)

La fonction OU exclusif, souvent appelé XOR (*eXclusive OR*), est un opérateur logique de l'algèbre de Boole. À deux événements, qui peuvent avoir chacun la valeur VRAI ou FAUX, il associe un résultat qui a lui-même la valeur VRAI ou FAUX.

Cet opérateur est très utilisé en électronique, en informatique, et aussi en cryptographie du fait de ses propriétés intéressantes. Son symbole est traditionnellement un signe plus dans un cercle ($\oplus$).

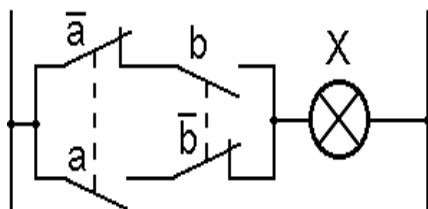


FIGURE 5.19 – Schéma électrique de la fonction logique XOR

b	a	X
0	0	0
0	1	1
1	0	1
1	1	0

FIGURE 5.20 – Table de vérité de la fonction logique XOR

$$X = a \oplus b = \bar{a}.b + a.\bar{b}$$

5.2.4 Symboles des Opérateurs Logiques

Deux ensembles de symboles sont utilisés pour représenter les fonctions logiques ; les deux sont définis par la norme ANSI/IEEE 91-1984 et son supplément 91a-1991. La représentation par « symboles distinctifs », basée sur les schématisations classiques, est utilisée pour les schémas simples et est plus facile à tracer à la main. Elle est parfois qualifiée de « militaire », ce qui reflète ses origines, sinon son usage actuel.

La représentation « rectangulaire » se base sur la norme CEI 60617-12 ; toutes les portes y sont représentées avec des bords rectangulaires et un symbole, ce qui permet la représentation d'un plus grand nombre de types de circuits. Ce système a été repris par d'autres standards comme EN 60617-12 :1999 en Europe et BS EN 60617-12 :1999 au Royaume-Uni.

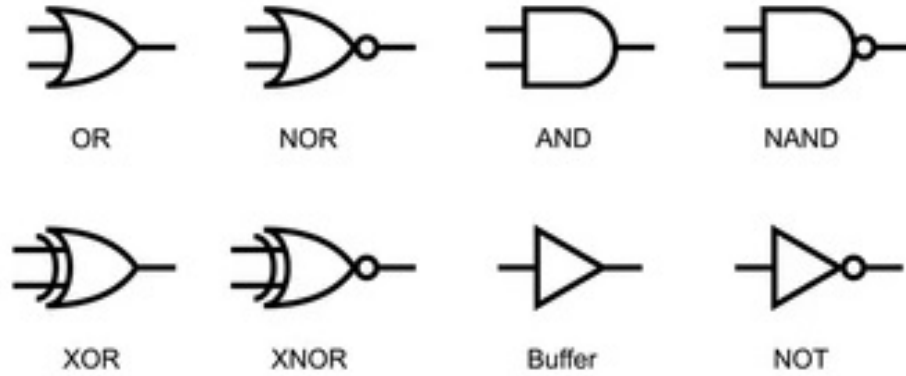


FIGURE 5.21 – Symboles américain des opérateurs logiques

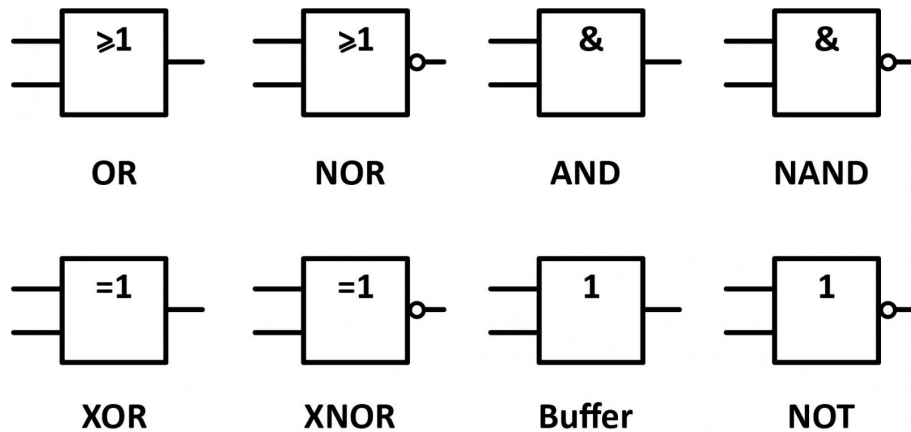


FIGURE 5.22 – Symboles Européens des opérateurs logiques

5.3 Théorèmes fondamentaux

5.3.1 Écriture algébrique

C'est l'écriture qui permet, par les opérations somme logique et produit logique, de relier des variables booléennes entre elles pour exprimer la valeur d'une fonction logique.

Par exemple, l'expression d'une fonction logique S dont les variables sont : E_1, E_2, E_3 est donnée par l'équation 5.1 :

$$S = E_1.E_2.E_3 + E_1.\overline{E_2}.\overline{E_3} + \overline{E_1}.E_2.\overline{E_3} + \overline{E_1}.\overline{E_2}.E_3 \quad (5.1)$$

5.3.2 Table de vérité

Une table de vérité est une table mathématique utilisée en logique combinatoire (l'algèbre de Boole) pour représenter de manière sémantique des expressions logiques et calculer la valeur de leur fonction de sortie relativement à chacun de leurs entrées (*chaque combinaison de valeur assumée par leurs variables logiques*).

En pratique, une table de vérité est composée d'une colonne pour chaque variable d'entrée (E_1, E_2 et E_3 par exemple dans l'équation 5.1), et d'une colonne où sont inscrits tous les résultats possibles de l'opération logique S représentée par le tableau.

Chaque ligne de la table de vérité contient ainsi une des configurations possibles des variables d'entrées, les valeurs sont placées dans l'ordre binaire naturel, ainsi que le résultat de l'opération pour ces valeurs. l'exemple suivant traite le cas d'un table de vérité d'une fonction logique de quatre variables :

a	b	c	d	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

FIGURE 5.23 – Table de vérité d'une fonction logique de quatre variables a,b,c et d et une sortie F

5.3.3 Lois fondamentales

On choisit trois variables booléenne E_1, E_2 et E_3 et trois opérations : NON(NOT), ET(AND) et OU(OR)

Lois de fermeture

$E_1.E_2$ est une fonction booléenne définie par la table de vérité de l'opération **AND**.

$E_1 + E_2$ est une fonction booléenne définie par la table de vérité de l'opération **OR**.

Lois de commutativité

$$E_1.E_2 = E_2.E_1$$

$$E_1 + E_2 = E_2 + E_1$$

Lois d'associativité

$$E_1.(E_2.E_3) = (E_1.E_2).E_3$$

$$E_1 + (E_2 + E_3) = (E_1 + E_2) + E_3$$

Lois de distributivité

- De l'opérateur AND sur OR

$$E_1.(E_2 + E_3) = (E_1.E_2) + (E_1.E_3)$$

- De l'opérateur OR sur AND

$$E_1 + (E_2.E_3) = (E_1 + E_2).(E_1 + E_3)$$

Lois d'idempotence

$$E + E = E$$

$$E.E = E$$

Lois complémentarité

$$E + \overline{E} = 1$$

$$E.\overline{E} = 0$$

Lois d'identité remarquable

$$1 + E = 1$$

$$1.E = E$$

$$0 + E = E$$

$$0.E = 0$$

5.3.4 Théorème de Morgan

Auguste De Morgan est reconnu pour sa redécouverte de la loi de dualité entre la somme et le produit, où le contraire d'un agrégat (somme logique) est le composé (produit logique) des contraires des agrégants ; le contraire d'un composé est l'agrégat des contraires des composants.

On considère deux variables logiques A et B.

• **Théorème 1 :** Le complémentaire du OU des variables est égal au ET des complémentaires de chaque variable.

$$\overline{A + B} = \overline{A}.\overline{B} \quad (5.2)$$

• **Théorème 2 :** Le complémentaire du ET des variables est égal au OU des complémentaires de chaque variable.

$$\overline{A.B} = \overline{A} + \overline{B} \quad (5.3)$$

Ces deux théorèmes sont très utilisés pour simplifier des propositions logiques. Ils sont généralisables à n variables.

5.4 Les circuits logiques combinatoires**5.4.1 Le Multiplexeur**

Un multiplexeur (*abréviation : MUX*) est un circuit permettant de concentrer sur une même voie de sortie différents entrées en sélectionnant une parmi N. Il possèdera donc une sortie et N entrées, ainsi qu'une entrée de commande ou de selection de $\log_2 N$ bits permettant de choisir quelle entrée sera sélectionnée.

$$N = 2^k$$

avec :

- N : Nombre de voies d'entrées
- k : Taille du bus de sélection (Adresses)

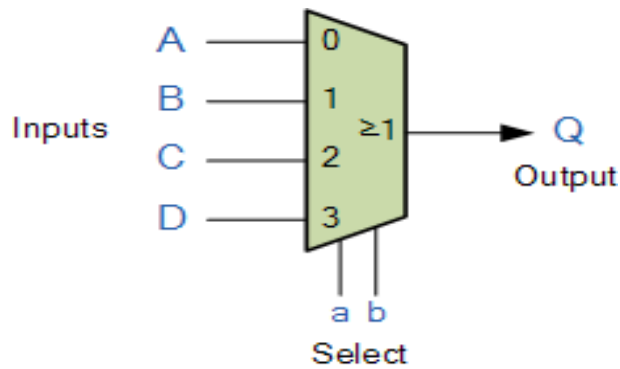


FIGURE 5.24 – Symbole d'un Multiplexeur 4-vers-1

a	b	Q
0	0	A
0	1	B
1	0	C
1	1	D

FIGURE 5.25 – Table de vérité d'un Multiplexeur 4-vers-1

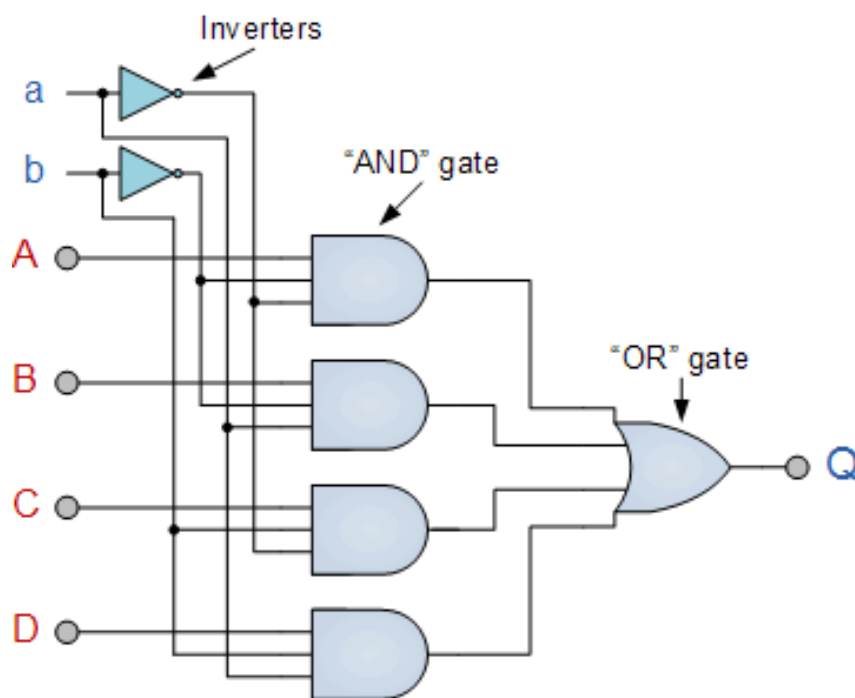


FIGURE 5.26 – Circuit électronique d'un Multiplexeur 4-vers-1

5.4.2 Le Démultiplexeur

Un démultiplexeur est un circuit combinatoire à $N + 1$ entrées et 2^N sorties. Les N entrées, appelées entrées d'adressage, permettent d'envoyer sur l'une des sorties la dernière entrée, appelée entrée de donnée.

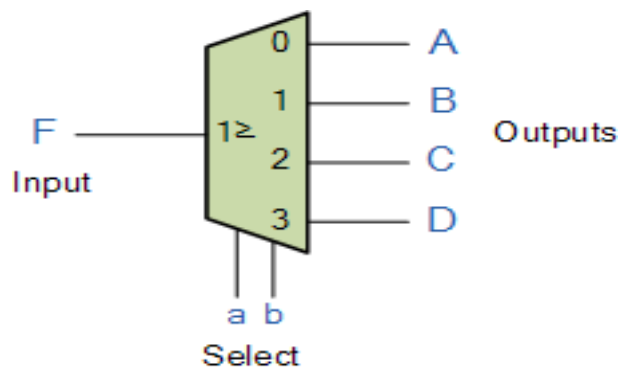


FIGURE 5.27 – Symbole d'un Démultiplexeur 1-vers-4

a	b	D	C	B	A
0	0	0	0	0	F
0	1	0	0	F	0
1	0	0	F	0	0
1	1	F	0	0	0

FIGURE 5.28 – Table de vérité d'un Démultiplexeur 1-vers-4

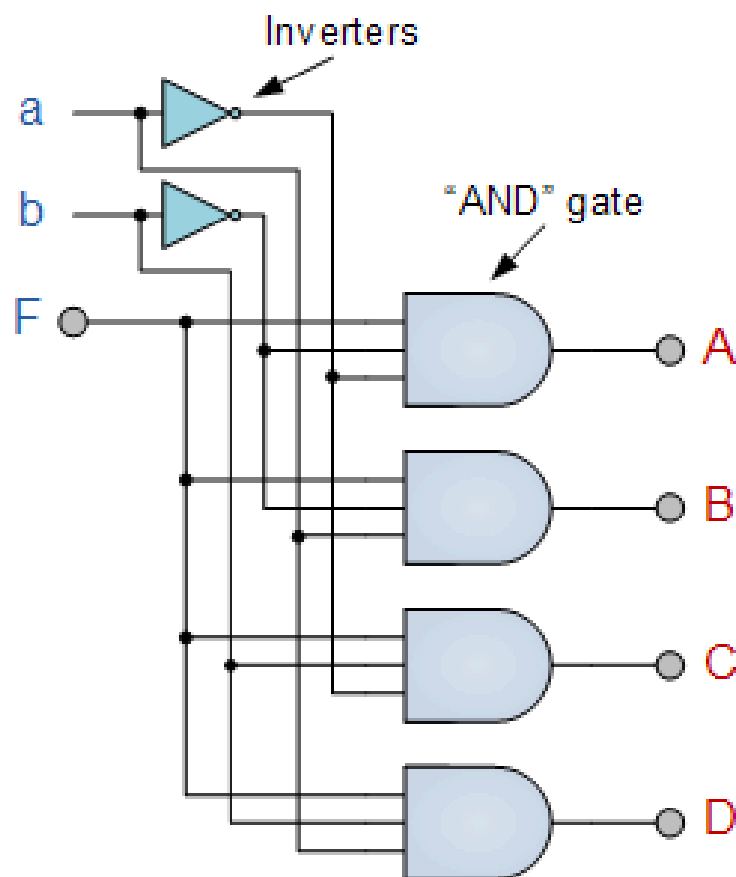


FIGURE 5.29 – Circuit électronique d'un Démultiplexeur 1-vers-4

5.4.3 Les Encodeurs prioritaires

Un codeur 2^n ne fonctionne correctement que si aucune des entrées n'est au niveau logique 1 ou just une seule entrée active. Lorsque plusieurs entrées prennent simultanément le niveau logique 1, le codage est erroné. Une solution, dans ce cas, consiste à utiliser un encodeur prioritaire.

Un encodeur prioritaire génère un code binaire correspondant au numéro de l'entrée

active ayant la priorité la plus élevée (*ou, le plus souvent, le nombre le plus élevé*). Il peut être utilisé dans les applications suivantes :

- **codeur clavier** : lorsque plusieurs touches sont enfoncées simultanément, seule la touche avec le numéro le plus élevé est prise en compte.
- **unité de traitement des demandes d'interruption dans un microprocesseur** : en cas de demandes d'interruption simultanées, seule la demande de priorité la plus élevée est acceptée.

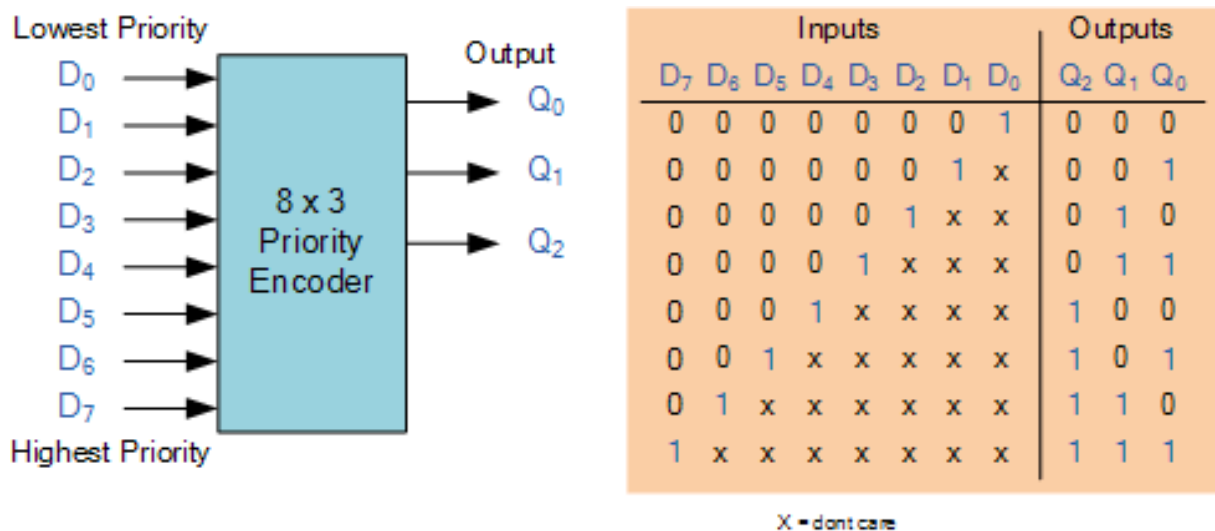


FIGURE 5.30 – Encodeur prioritaire 8-vers-3

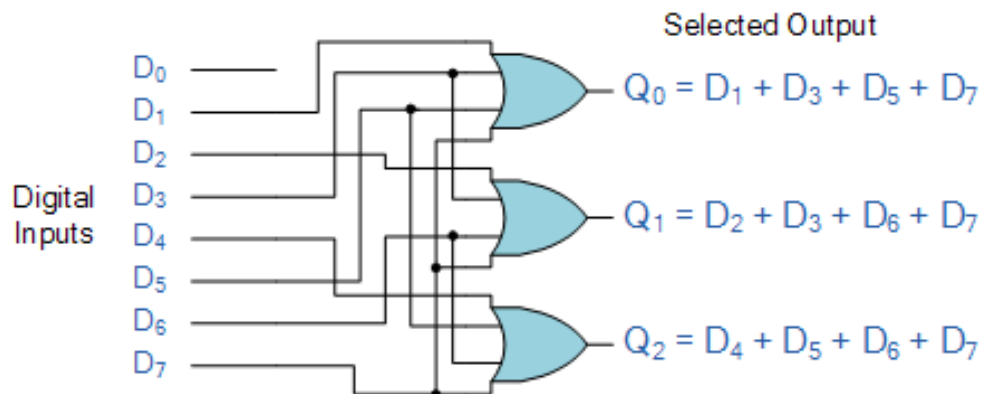


FIGURE 5.31 – Circuit électronique d'un Encodeur prioritaire 8-vers-3

5.4.4 Le Décodeur

Le décodeur binaire est un autre circuit logique combinatoire construit à partir de portes logiques individuelles et est l'exact opposé d'un encodeur.

Le terme "décodeur" signifie traduire ou décoder des informations codées d'un format à un autre, ainsi un décodeur binaire transforme " n " signaux binaires d'entrée en un code équivalent utilisant 2^n sorties.

Les décodeurs binaires sont un autre type de dispositif logique numérique qui a des entrées de codes de 2, 3 ou 4 bits selon le nombre de lignes d'entrée de données, ainsi un décodeur qui a un ensemble de deux bits ou plus sera défini comme ayant un code de n bits, et donc il sera possible de représenter 2^n valeurs possibles. Ainsi, un décodeur décode généralement une valeur binaire en une valeur non binaire en mettant exactement une de ses n sorties à la valeur logique "1".

Si un décodeur binaire reçoit n entrées (*généralement regroupées en un seul nombre binaire ou booléen*), il active une et une seule de ses 2^n sorties en fonction de cette entrée, toutes les autres sorties étant désactivées.

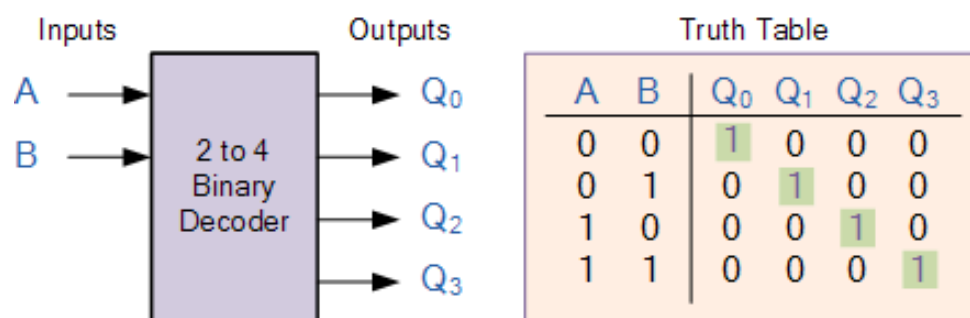


FIGURE 5.32 – Décodeur 2-vers-4 avec table de vérité

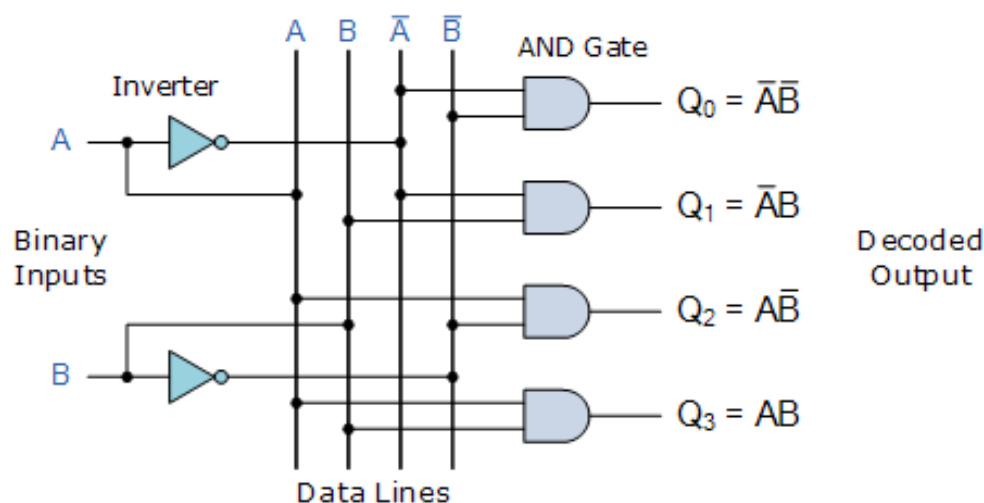


FIGURE 5.33 – Circuit électronique d'un décodeur 2-vers-4

Application : Décodeur d'adresse mémoire

Les décodeurs binaires sont très utilisés dans des systèmes numériques plus complexes pour accéder à un emplacement mémoire particulier en fonction d'une "adresse" produite par un dispositif informatique (MPU). Dans les systèmes modernes à microprocesseur, la quantité de mémoire requise peut être assez élevée et est généralement supérieure à celle d'une seule puce mémoire.

Une méthode permettant de surmonter ce problème consiste à connecter ensemble de nombreuses puces mémoire individuelles et à lire les données sur un "*bus de données*" commun. Afin d'éviter que les données soient "lues" sur chaque puce mémoire en même temps, chaque puce mémoire est sélectionnée individuellement, une par une, et ce processus est connu sous le nom de **décodage d'adresse**.

Dans ce type d'application, l'adresse représente l'entrée de données codées, et les sorties sont les signaux de sélection des éléments de mémoire particuliers. Chaque puce de mémoire possède une entrée appelée **Chip Select** ou **CS** qui est utilisée par le MPU (*unité microprocesseur*) pour sélectionner la puce de mémoire appropriée lorsque cela est nécessaire. En général, un "1" logique sur l'entrée de sélection de puce (**CS**) sélectionne le dispositif de mémoire tandis qu'un "0" logique sur l'entrée le désélectionne.

Ainsi, en sélectionnant ou en désélectionnant chaque puce une par une, nous pouvons sélectionner le dispositif d'adresse mémoire correct pour un emplacement d'adresse particulier. L'avantage du décodage d'adresse est que lorsque nous spécifions une adresse mémoire particulière, l'emplacement mémoire correspondant n'existe que dans l'une des puces.

Par exemple, supposons que nous ayons un système de microprocesseur très simple avec seulement 1Kbytes (mille octets) de mémoire RAM et 10 lignes d'adresse mémoire disponibles. La mémoire est constituée de dispositifs de 128×8 bits ($1288 = 1024 \text{ octets}$) et pour 1Ko, nous aurions besoin de 8 puces mémoire individuelles, mais pour sélectionner la bonne puce mémoire, nous aurions également besoin d'un décodeur binaire de 3 à 8 lignes, comme indiqué dans la Figure 5.34 .

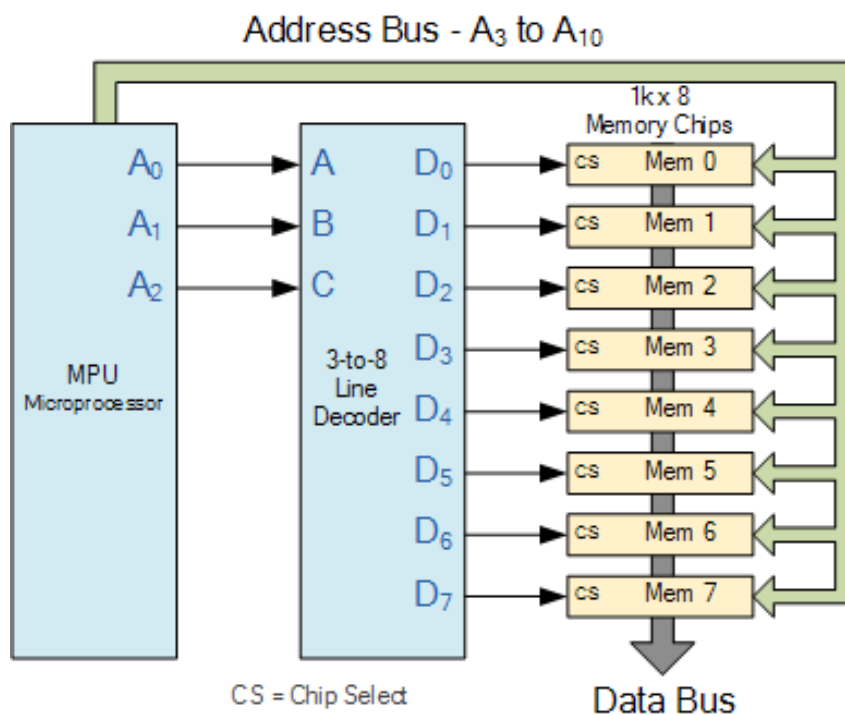


FIGURE 5.34 – Exemple d'application d'un Décodeur d'adresse mémoire

Le décodeur binaire n'a besoin que de 3 lignes d'adresse (A0 à A2) pour sélectionner chacune des 8 puces (*la partie inférieure de l'adresse*), tandis que les 8 autres lignes d'adresse (A3 à A10) sélectionnent l'emplacement de mémoire correct sur cette puce (*la partie supérieure de l'adresse*).

Après avoir sélectionné un emplacement de mémoire à l'aide du bus d'adresses, les informations contenues dans cet emplacement de mémoire interne particulier sont envoyées à un "bus de données" commun pour être utilisées par le microprocesseur. Il s'agit bien sûr d'un exemple simple, mais les principes restent les mêmes pour tous les types de puces ou de modules de mémoire.

Les décodeurs binaires sont des dispositifs très utiles pour convertir un format numérique en un autre, comme des données de type binaire ou BCD en décimal ou octal, etc. Les circuits intégrés décodeurs les plus courants sont le décodeur binaire **TTL 74LS138** à 3-8 lignes ou le décodeur **74ALS154** à 4-16 lignes.

5.4.5 Le Transcodeur

Les transcodeurs sont des circuits qui convertissent les différents codes entre eux. D'une manière générale, nous pouvons définir le transcodage comme étant la représentation d'un élément de l'ensemble A par un ou plusieurs éléments pris dans un ensemble B d'éléments. Le rapport entre les deux quantités est déterminé par une règle de correspondance.

La représentation numérique la plus connue et la plus répandue en technique numérique est le code binaire. Pour une meilleure lisibilité, le code binaire peut être reproduit en code hexadécimal (*chaque quadruplet de chiffres binaire est écrit en nombre hexadécimal décimal*). Parmi les autres codes nous citerons : les codes BCD (*nombre décimaux codés en binaire*), le code Aiken, le code 7-segment, le code Gray, le code 1 parmi 10...etc.

Chaque code présente deux attributs caractéristiques : la longueur **m** et le nombre de mot de code **n** possible.

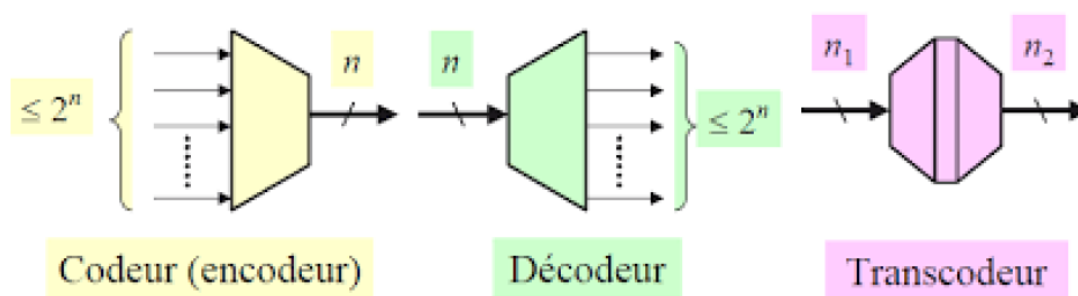


FIGURE 5.35 – Principe de fonctionnement du : Codeur / Décodeur / Transcodeur

Il faut prendre en considération la différence entre un code complet et un code incomplet. Le code complet contient tous les mots de code binaire ou combinaisons possibles. Par contre, le code incomplet n'utilise qu'une partie des combinaisons possibles. Le code

BCD fait donc partie du groupe des codes incomplets étant donné qu'il n'utilise que 10 mots de code binaire possibles sur 16.

Nombre décimal	Code BCD				Sorties						
	D	C	B	A	a	b	c	d	e	f	g
0	L	L	L	L	ON	ON	ON	ON	ON	ON	OFF
1	L	L	L	H	OFF	ON	ON	OFF	OFF	OFF	OFF
2	L	L	H	L	ON	ON	OFF	ON	ON	OFF	ON
3	L	L	H	H	ON	ON	ON	ON	OFF	OFF	ON
4	L	H	L	L	OFF	ON	ON	OFF	OFF	ON	ON
5	L	H	L	H	ON	OFF	ON	ON	OFF	ON	ON
6	L	H	H	L	OFF	OFF	ON	ON	ON	ON	ON
7	L	H	H	H	ON	ON	ON	OFF	OFF	OFF	OFF
8	H	L	L	L	ON	ON	ON	ON	ON	ON	ON
9	H	L	L	H	ON	ON	ON	OFF	OFF	ON	ON

ON = transistor saturé (niveau 0)

OFF = transistor bloqué (niveau 1)

FIGURE 5.36 – Exemple de table de vérité d'un transcodeur BCD/7-segments

5.4.6 L'Additionneur

Les additionneurs binaires sont des circuits arithmétiques sous forme de demi-additionneurs et d'additionneurs complets utilisés pour additionner deux chiffres binaires.

L'additionneur binaire est un autre circuit logique combinatoire courant et très utile qui peut être construit à l'aide de quelques portes logiques de base permettant d'additionner deux ou plusieurs nombres binaires.

Un circuit d'additionneur binaire de base peut être réalisé à partir de portes **ET** et **OU-Exclusif** standard, ce qui permet d'additionner deux nombres binaires à un seul bit, **A** et **B**.

L'addition de ces deux chiffres produit une sortie appelée SOMME de l'addition et une seconde sortie appelée CARRY ou Carry-out, (C_{OUT}) selon les règles de l'addition binaire. L'une des principales utilisations de l'additionneur binaire est dans les circuits arithmétiques et de comptage.

Pour le simple problème d'addition de 1-bit, le bit de retenue résultant pourrait être ignoré, mais vous avez peut-être remarqué autre chose concernant l'addition de ces deux bits, la somme de leur addition binaire ressemble à celle d'une porte OU exclusif. Si nous désignons les deux bits par A et B, la table de vérité résultante est la somme des deux bits, mais sans la retenue finale.

Le demi-additionneur

Un demi-additionneur est un circuit logique qui effectue une opération d'addition sur deux chiffres binaires. Le demi-additionneur produit une somme et une valeur de retenue qui sont toutes deux des chiffres binaires.

B	A	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

FIGURE 5.37 – Table de vérité du demi-additionneur

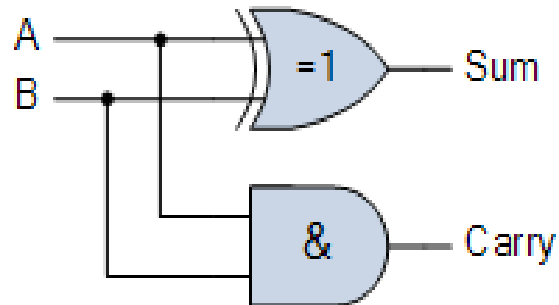


FIGURE 5.38 – Le circuit logique du demi-additionneur

D'après la table de vérité du demi-additionneur (Figure 5.37), nous pouvons voir que la sortie **SUM** est le résultat de la porte **OU-Exclusif** et que le report **Carry** est le résultat de la porte **ET**. L'expression booléenne d'un demi-additionneur est donc la suivante :

$$SUM = A \oplus B$$

$$CARRY = A.B$$

L'un des principaux inconvénients du circuit demi-additionneur, lorsqu'il est utilisé comme additionneur binaire, est qu'il n'est pas prévu de "report" à partir du circuit précédent lors de l'addition de plusieurs bits de données.

Par exemple, supposons que nous voulions additionner deux octets de données de 8-bits, tout bit de report résultant devrait pouvoir se déplacer à travers les motifs de bits en commençant par le bit le moins significatif (LSB). L'opération la plus compliquée que peut effectuer le demi-additionneur est "1 + 1", mais comme le demi-additionneur n'a pas d'entrée de retenue, la valeur ajoutée résultante serait incorrecte. Une façon simple de surmonter ce problème est d'utiliser un circuit additionneur binaire de type (Full Adder) ou additionneur complet.

L'additionneur complet

La principale différence entre l'additionneur complet et le demi-additionneur précédent est qu'un additionneur complet possède trois entrées. Les deux mêmes entrées de données monobit **A** et **B** que précédemment, plus une entrée de retenue (C_{in}) supplémentaire pour recevoir la retenue d'un étage précédent, comme indiqué par la Figure 5.39.

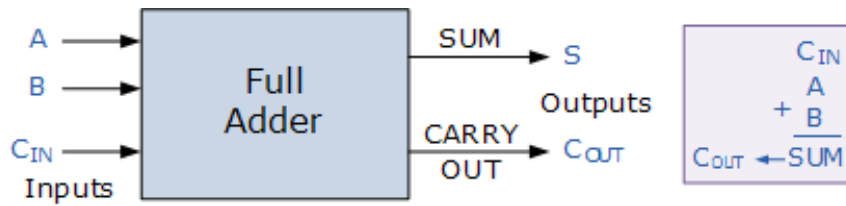


FIGURE 5.39 – Schéma fonctionnel de l'additionneur complet

Ensuite, l'additionneur complet est un circuit logique qui effectue une opération d'addition sur trois chiffres binaires et, tout comme le demi-additionneur, il génère également un report vers la colonne d'addition suivante. Une retenue d'entrée est une retenue possible à partir d'un chiffre moins significatif, tandis qu'une retenue de sortie représente une retenue vers un chiffre plus significatif (Figure 5.39).

L'additionneur complet peut être considéré comme deux demi-additionneurs connectés ensemble, le premier demi-additionneur transmettant sa retenue au second demi-additionneur comme indiqué dans la Figure 5.40.

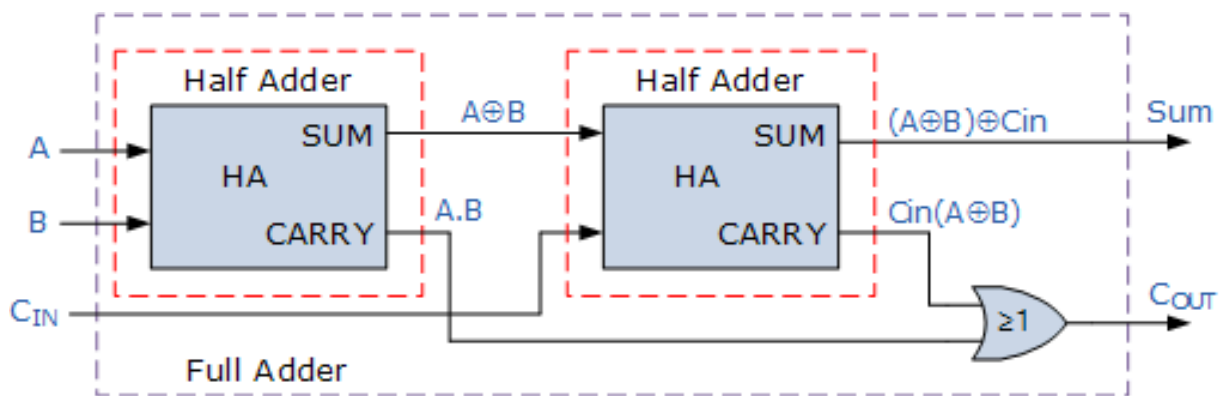


FIGURE 5.40 – Diagramme logique de l'additionneur binaire complet à l'aide de deux demi-additionneurs en cascade

Comme le circuit d'additionneur complet (Figure 5.40) est essentiellement constitué de deux demi-additionneurs connectés ensemble, la table de vérité de l'additionneur complet comprend une colonne supplémentaire pour prendre en compte l'entrée Carry-in, C_{IN} , ainsi que la sortie additionnée, Sum , et le bit Carry-out, C_{OUT} .

C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

FIGURE 5.41 – Table de vérité d'un additionneur complet

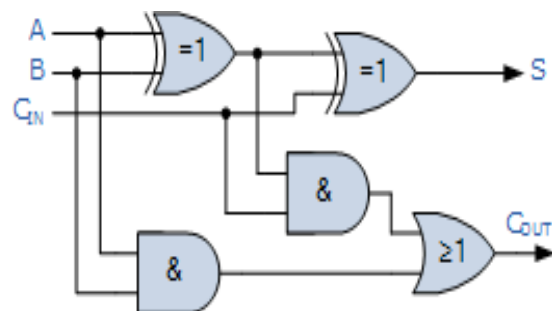


FIGURE 5.42 – Le circuit logique d'un additionneur complet

Alors les expressions booléennes pour un additionneur complet sont :

$$Sum = (A \oplus B) \oplus C_{IN}$$

$$C_{OUT} = A.B + C_{IN}.(A \oplus B)$$

L'additionneur complet a n-bits

Nous avons vu précédemment des additionneurs binaires simples de 1-bit peuvent être construits à partir de portes logiques de base. Mais si nous voulons additionner deux nombres de n-bits, il faut alors connecter en cascade un nombre **n** d'additionneurs complets de 1-bit pour produire ce que l'on appelle un additionneur de retenue.

Un additionneur de retenue (*ripple carry adder*) est simplement "n" additionneurs complets de 1-bit montés en cascade, chaque additionneur complet représentant une seule colonne pondérée dans une addition binaire longue. On l'appelle un additionneur de retenue car les signaux de retenue produisent un effet d'ondulation dans l'additionneur binaire de droite à gauche ($LSB \Rightarrow MSB$).

Par exemple, supposons que nous voulions "additionner" deux nombres de 4-bits, les deux sorties du premier additionneur complet fourniront la somme du premier chiffre (S) de l'addition plus un bit de retenue qui servira de chiffre de retenue de l'additionneur binaire suivant.

Le deuxième additionneur binaire de la chaîne produit également une sortie additionnée (le 2^{me} bit) plus un autre bit de report et nous pouvons continuer à ajouter d'autres additionneurs complets à la combinaison pour ajouter des nombres plus grands, en reliant la sortie du bit de report du premier additionneur binaire complet à l'additionneur complet suivant, et ainsi de suite. Un exemple d'un additionneur de 4 bits est donné par la Figure 5.43.

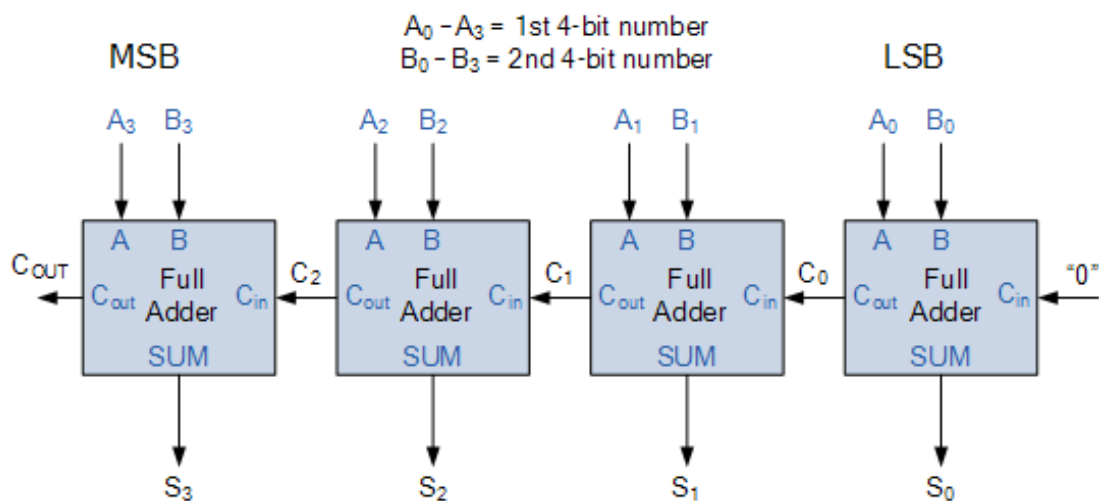


FIGURE 5.43 – Additionneur de retenue binaire 4-bit

L'un des principaux inconvénients de la "mise en cascade" d'additionneurs binaires de 1-bit pour additionner de grands nombres binaires est que si les entrées A et B changent, la somme à sa sortie ne sera pas valide tant qu'une entrée de retenue n'aura pas "traversé" chaque additionneur complet de la chaîne, car le MSB (*bit le plus significatif*) de la somme doit attendre tout changement de l'entrée de retenue du LSB (*bit le moins significatif*). Par conséquent, il y aura un retard fini avant que la sortie de l'additionneur ne réponde à tout changement dans ses entrées, ce qui entraîne un retard accumulé.

Remarque : Lorsque la taille des bits à ajouter n'est pas trop importante, par exemple 4 ou 8-bits, ou que la vitesse d'addition de l'additionneur n'est pas importante, ce retard peut ne pas être important. Cependant, lorsque la taille des bits est plus importante, par exemple 32 ou 64-bits utilisés dans les additionneurs multi-bits, ou que la sommation est requise à une vitesse d'horloge très élevée, ce retard peut devenir prohibitif, les processus d'addition n'étant pas achevés correctement en un cycle d'horloge.

Le Circuit Intégré 74LS83

Les circuits additionneurs complets 4 bits avec fonctions de report sont disponibles sous la forme de boîtiers de circuits intégrés standard, à savoir l'additionneur binaire TTL 4-bits 74LS83 ou 74LS283 et le CMOS 4008, qui peuvent additionner deux nombres binaires 4-bits et générer une sortie SUM et CARRY, comme illustré dans la Figure 5.44.

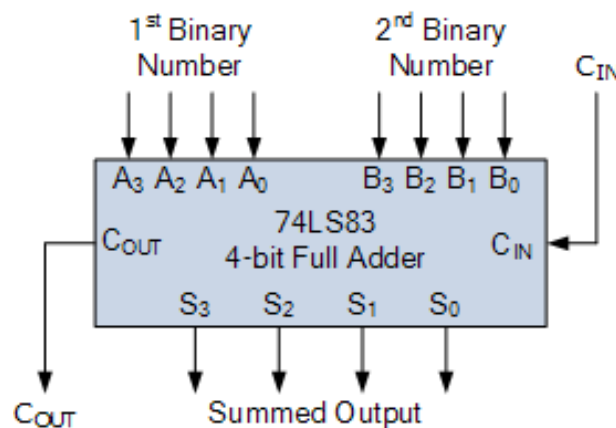


FIGURE 5.44 – Symbole logique du Circuit Intégré 74LS83

5.4.7 Le Soustracteur

Le soustracteur binaire est un autre type de circuit arithmétique combinatoire qui produit une sortie qui est la soustraction de deux nombres binaires.

Comme son nom l'indique, un soustracteur binaire est un circuit décisionnel qui soustrait deux nombres binaires l'un de l'autre, par exemple, $X - Y$ pour trouver la différence résultante entre les deux nombres.

Contrairement à l'additionneur binaire qui produit une SOMME et un bit de report lorsque deux nombres binaires sont additionnés, le soustracteur binaire produit une DIFFERENCE, D en utilisant un bit d'emprunt, B, de la colonne précédente. Alors évidemment, l'opération de soustraction est l'inverse de celle de l'addition.

Le demi-soustracteur

Un demi-soustracteur est un circuit logique qui effectue une opération de soustraction sur deux chiffres binaires. Le demi-soustracteur produit une somme et un bit d'emprunt pour l'étape suivante.

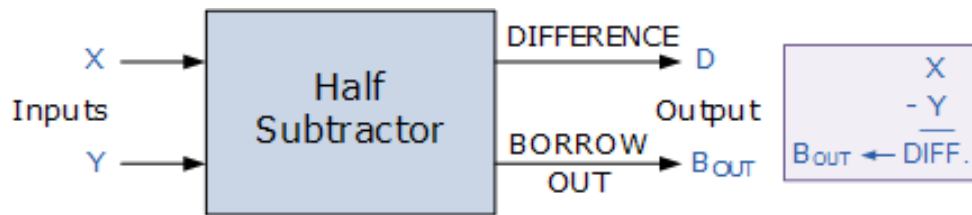


FIGURE 5.45 – Symbole d'un Demi-soustracteur avec emprunt

Y	X	DIFFERENCE	BORROW
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	0

FIGURE 5.46 – Table de vérité d'un Demi-soustracteur avec emprunt

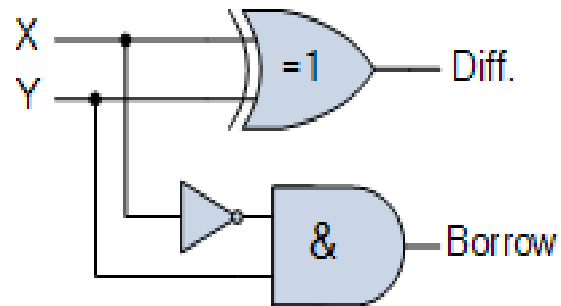


FIGURE 5.47 – Le circuit logique d'un Demi-soustracteur avec emprunt

D'après la table de vérité du demi-soustracteur (Figure 5.46), nous pouvons voir que la sortie DIFFERENCE (D) est le résultat de la porte **OU exclusif** et que la sortie BORROW (B_{out}) est le résultat de la combinaison **NON et ET**. Les expressions booléennes d'un demi-soustracteur est donc :

$$D = X \oplus Y$$

$$B_{out} = \overline{X}.Y$$

Si nous comparons les expressions booléennes du demi-soustracteur et du demi-additionneur, nous constatons que les deux expressions pour la SOMME (additionneur) et la DIFFÉRENCE (soustracteur) sont exactement les mêmes, ce qui est normal en raison de la fonction de porte OU exclusif. Les deux expressions booléennes pour le soustracteur binaire BORROW sont également très similaires à celles de l'additionneur CARRY. Ensuite,

tout ce qui est nécessaire pour convertir un demi-additionneur en un demi-soustracteur est l'inversion de l'entrée du menu X.

L'un des principaux inconvénients du circuit du demi-soustracteur, lorsqu'il est utilisé comme soustracteur binaire, est qu'il n'est pas prévu d'emprunter au circuit précédent lors de la soustraction de plusieurs bits de données les uns des autres. Il faut alors produire ce que l'on appelle un circuit (**soustracteur binaire complet**) pour prendre en compte cet emprunt d'entrée à partir d'un circuit précédent.

Le soustracteur complet

La principale différence entre le soustracteur complet et le circuit demi-soustracteur précédent est qu'un soustracteur complet possède trois entrées. Les deux entrées de données à un bit **X** et **Y** sont les mêmes que précédemment, plus une entrée d'emprunt supplémentaire (B_{in}) pour recevoir l'emprunt généré par le processus de soustraction d'une étape précédente, comme indiqué dans la Figure 5.48 .

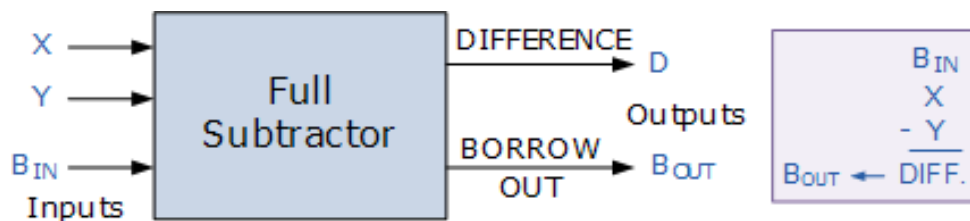


FIGURE 5.48 – Schéma fonctionnel d'un soustracteur binaire complet

Ensuite, le circuit combinatoire d'un soustracteur complet effectue l'opération de soustraction sur trois bits binaires en produisant des sorties pour la différence **D** et l'emprunt B_{out} . Tout comme le circuit de l'additionneur binaire, le soustracteur complet peut également être considéré comme deux demi-soustracteurs connectés ensemble, le premier demi-soustracteur transmettant son emprunt au second demi-soustracteur comme le montre la Figure 5.49.

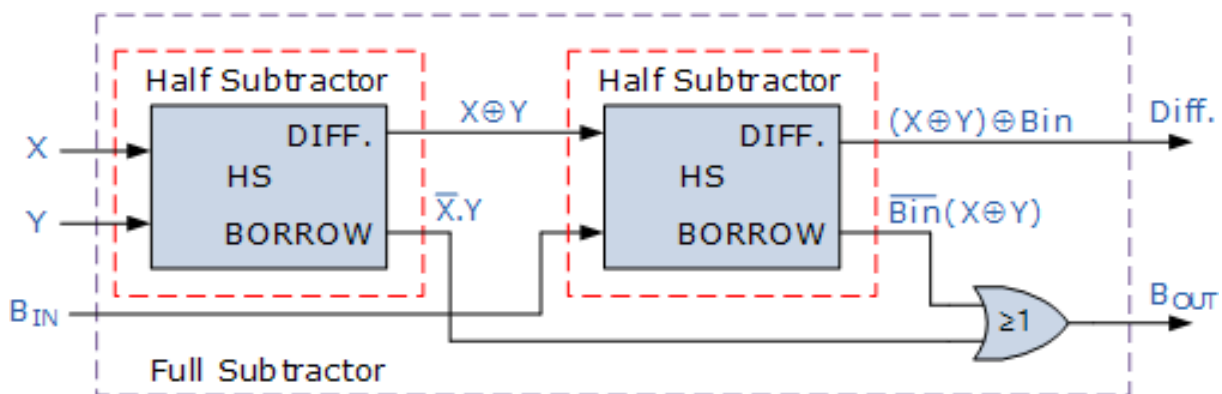


FIGURE 5.49 – Diagramme logique d'un soustracteur binaire complet

Comme le circuit du soustracteur complet de la Figure 5.49 représente deux demi-soustracteurs montés en cascade, la table de vérité du soustracteur complet aura huit combinaisons d'entrée différentes car il y a trois variables d'entrée, les bits de données et l'entrée B_{IN} (emprunt). Elle comprend également la sortie de différence, D , et le bit d'emprunt, B_{OUT} .

B_{in}	Y	X	Diff	B_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	1

FIGURE 5.50 – Table de vérité d'un soustracteur binaire complet

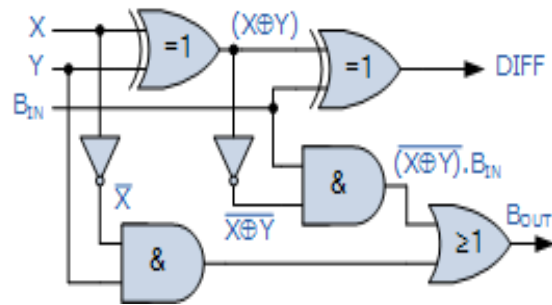


FIGURE 5.51 – Le circuit logique d'un soustracteur binaire complet

Alors l'expression booléenne pour un soustracteur complet sont :

$$Diff = X \cdot \overline{Y} \cdot \overline{B_{IN}} + \overline{X} \cdot Y \cdot \overline{B_{IN}} + \overline{X} \cdot \overline{Y} \cdot B_{IN} + X \cdot Y \cdot B_{IN}$$

qui peut également être simplifié :

$$Diff = X \oplus Y \oplus B_{IN}$$

Pour le bit BORROW OUT (B_{OUT}) :

$$B_{OUT} = \overline{X} \cdot Y \cdot \overline{B_{IN}} + \overline{X} \cdot \overline{Y} \cdot B_{IN} + \overline{X} \cdot Y \cdot B_{IN} + X \cdot Y \cdot B_{IN}$$

qui peut également être simplifié :

$$B_{OUT} = \overline{X} \cdot Y + (\overline{X} \oplus \overline{Y}) \cdot B_{IN}$$

5.4.8 Le soustracteur binaire n-bits

Comme pour l'additionneur binaire, nous pouvons également avoir un nombre n de soustracteurs binaires complets de 1-bit connectés ou "cascadés" ensemble pour soustraire deux nombres parallèles de n -bits l'un de l'autre. Par exemple, deux nombres binaires de 4-bits. Nous avons dit précédemment que la seule différence entre un additionneur complet et un soustracteur complet était l'inversion d'une des entrées.

Ainsi, en utilisant un additionneur de n -bits et un nombre n d'inverseurs (*NOT Gates*), le processus de soustraction devient une addition car nous pouvons utiliser la notation de

complément à deux sur tous les bits du sous-produit et régler l'entrée de report du bit le moins significatif sur un "1" logique (HIGH).

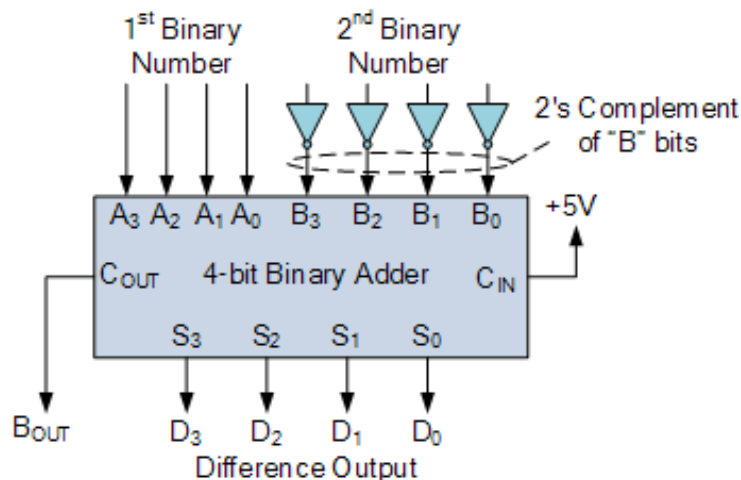


FIGURE 5.52 – Soustracteur binaire utilisant le complément à 2

Nous pouvons alors utiliser un circuit intégré d'additionneur complet 4-bits tel que le **74LS283** et le **CD4008** pour effectuer une soustraction en utilisant simplement le complément à deux sur les entrées de soustraction, B comme $X - Y$, ce qui revient à dire $X + (-Y)$, ce qui est égal à **X** plus le complément à deux de **Y**.

Si nous voulions à nouveau utiliser l'additionneur 4 bits pour l'addition, il nous suffirait de mettre l'entrée de retenue (C_{IN}) au niveau BAS, à la valeur logique "0". Comme nous pouvons utiliser les circuits intégrés additionneurs 4-bits tels que le **74LS83** ou le **74LS283** en tant qu'additionneur complet ou soustracteur complet, ils sont disponibles en tant que circuit additionneur/soustracteur unique avec une seule entrée de commande pour choisir entre les deux opérations.

5.4.9 Le Comparateur

Le comparateur numérique est un autre circuit logique combinatoire très utile utilisé pour comparer la valeur de deux chiffres binaires.

Le comparateur binaire ou numérique peut être construit en utilisant des portes AND, NOR et NOT standard pour comparer les signaux numériques présents à leurs bornes d'entrée et produire une sortie en fonction de la condition de ces entrées.

Par exemple, en plus de pouvoir additionner et soustraire des nombres binaires, nous devons pouvoir les comparer et déterminer si la valeur de l'entrée **A** est supérieure, inférieure ou égale à la valeur de l'entrée **B**, etc. Le comparateur numérique accomplit cette tâche en utilisant plusieurs portes logiques qui fonctionnent selon les principes de l'algèbre de Boole. Il existe deux principaux types de comparateurs numériques.

1. **Comparateur d'identité** :un comparateur d'identité est un comparateur numérique avec une seule borne de sortie. Lorsque $A = B$, soit $A = B = 1$ (HIGH), soit $A = B = 0$ (LOW).
2. **Comparateur de magnitude** :un comparateur de magnitude est un comparateur numérique qui possède trois bornes de sortie, une pour l'égalité, $A = B$ supérieur à, $A > B$ et inférieur à $A < B$.

Le but d'un comparateur numérique est de comparer un ensemble de variables ou de nombres inconnus, par exemple A (A1, A2, A3, An, etc.) à celui d'une constante ou d'une valeur inconnue telle que B (B1, B2, B3, Bn, etc.) et de produire une condition de sortie ou un indicateur en fonction du résultat de la comparaison. Par exemple, un comparateur de magnitude avec deux entrées de 1-bit (A et B) produirait les trois conditions de sortie suivantes lorsqu'elles sont comparées l'une à l'autre :

$$A > B; A = B; A < B$$

Ce qui veut dire : A est supérieur à B, A est égal à B, ou A est inférieur à B.

Ceci est utile si nous voulons comparer deux variables et produire une sortie lorsque l'une des trois conditions ci-dessus est atteinte. Par exemple, produire une sortie à partir d'un compteur lorsqu'un certain nombre de comptes est atteint. Considérons le comparateur simple à 1-bit ci-dessous.

comparateur numérique à 1-bit

Entrées		Sorties		
A	B	A = B	A > B	A < B
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

FIGURE 5.53 – Table de vérité 1-bit

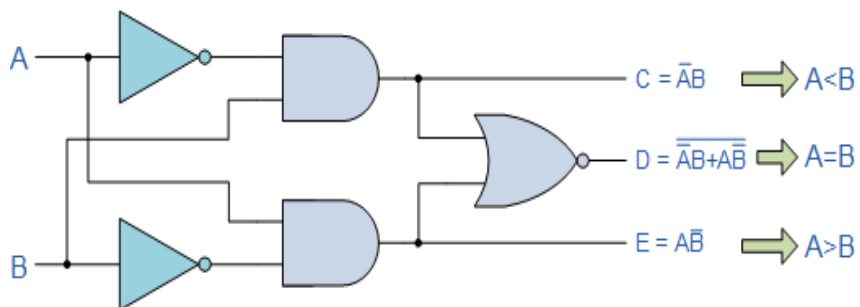


FIGURE 5.54 – Circuit électronique du comparateur numérique à 1-bit

comparateur numérique à 4-bits

Certains comparateurs numériques disponibles dans le commerce, tels que le comparateur de magnitude 4-bits TTL **74LS85** ou CMOS **4063**, possèdent des bornes d'entrée supplémentaires qui permettent de "cascader" plusieurs comparateurs individuels pour comparer des mots de plus de 4-bits avec des comparateurs de magnitude de "n" bits.

Ces entrées en cascade sont connectées directement aux sorties correspondantes du comparateur précédent, comme indiqué (Figure 5.56), pour comparer des mots de 8, 16 ou même 32-bits.

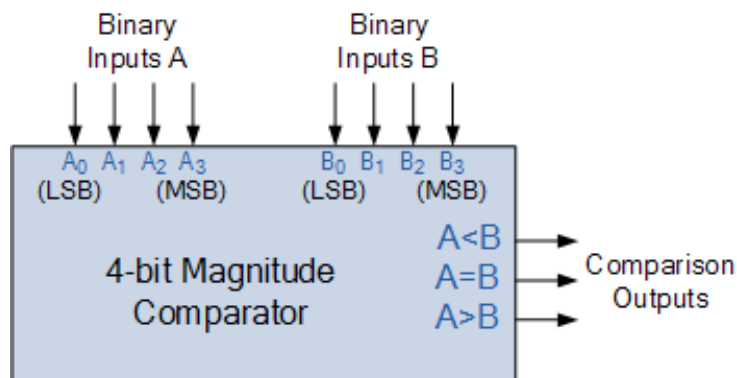


FIGURE 5.55 – Symbole d'un comparateur numérique à 4-bits

comparateur numérique à 8-bits

Lors de la comparaison de grands nombres binaires ou BCD comme dans l'exemple de 8-bits, pour gagner du temps, le comparateur commence par comparer le bit de poids fort (MSB). S'il y a égalité ($A = B$), il compare le bit suivant le plus bas et ainsi de suite jusqu'au bit le plus bas (LSB). Si l'égalité existe toujours, les deux nombres sont définis comme étant égaux.

Si une inégalité est trouvée, soit $A > B$, soit $A < B$, la relation entre les deux nombres est déterminée et la comparaison entre tous les bits d'ordre inférieur supplémentaires s'arrête. Les comparateurs numériques sont largement utilisés dans les convertisseurs analogiques-numériques (ADC) et les unités arithmétiques et logiques (ALU) pour effectuer diverses opérations arithmétiques.

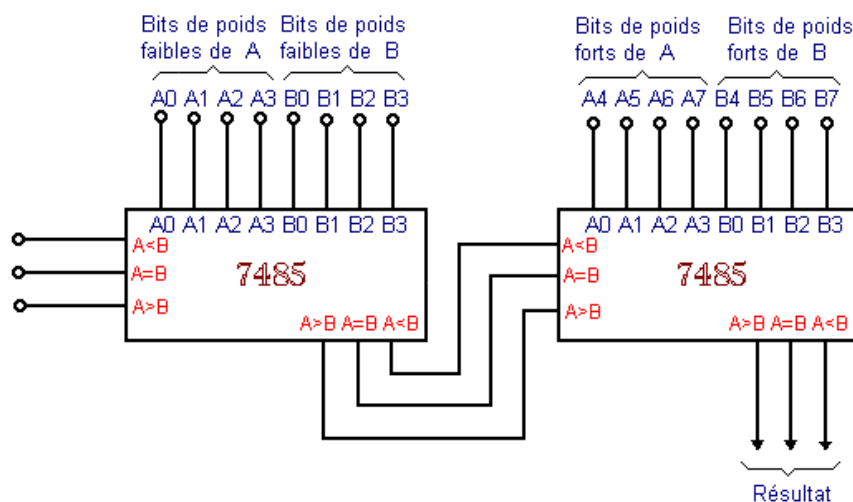


FIGURE 5.56 – Principe de mise en cascade de deux comparateur numérique à 4-bits

5.4.10 L'UAL

Définition

L'unité arithmétique et logique (UAL : "*En anglais arithmetic and logic unit, ALU*"), est l'organe de l'ordinateur chargé d'effectuer les calculs. Le plus souvent, l'UAL est incluse dans l'unité centrale de traitement (CPU) ou le microprocesseur (μP).

Fonctions de l'UAL

Les UAL peuvent être spécialisées ou pas. Les UAL élémentaires effectue des calculs sur des nombres entiers, et peuvent effectuer les opérations communes, que l'on peut séparer en quatre groupes :

1. **Les opérations arithmétiques** : addition, soustraction, changement de signe (*calcul du complément*), etc.
2. **Les opérations logiques** : compléments à un, à deux, et, ou, ou-exclusif, non, non-et, etc.
3. **Les comparaisons** : test d'égalité, supérieur, inférieur, et leur équivalents ($\leq, \geq$).
4. Les décalages et rotations.

Certaines UAL sont spécialisées dans la manipulation des nombres à virgule flottante, en simple ou double précision (*on parle d'unité de calcul en virgule flottante* (En anglais : *Floating-Point Unit*, FPU) ou dans les calculs vectoriels. Typiquement, ces unités savent accomplir les opérations suivantes : additions, soustractions, changement de signe, multiplications, divisions, comparaisons, modulus.

Remarque : Certaines UAL, le plus souvent de la classe des FPUs, sont susceptibles d'offrir des fonctions avancées, tel que :

- inverse ($\frac{1}{x}$)
- racine carrée ($\sqrt{x}$)
- logarithmes (ln, log)
- fonctions trigonométriques ($\sin(x)$, $\cos(x)$, $\tan(x)$, etc.);
- opération vectorielle (*produit scalaire, vectoriel, etc.*)

Schéma symbolique de l'UAL

La Figure 5.57 représente un schéma classique de l'UAL. Celle-ci possède deux entrées **A** et **B** sur lesquelles on présente les données à traiter. L'entrée **F** désigne l'opération à effectuer. Également, l'UAL possède deux sorties, **R** qui est le résultat de l'opération, et **D** qui représente les éventuels drapeaux.

Il existe différents drapeaux qui indiquent une erreur dans le résultat (*division par zéro, dépassement de mémoire...*), ou alors des codes conditions (*supérieur, inférieur, égal*

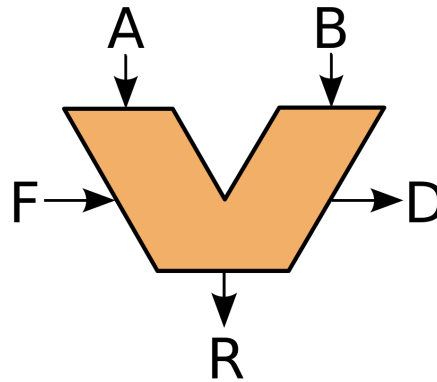


FIGURE 5.57 – Schéma symbolique de l'UAL

à zéro...). La plupart des UAL possèdent des drapeaux qui ont différentes significations, dont voici quelques exemples :

- **Report** (en anglais : *Carry-out*), transporte le report résultant d'une addition, la retenue résultant d'une opération de soustraction, ou le bit de dépassement résultant d'une opération de décalage binaire.
- **Zéro**, indique que tous les bits de **R** sont égaux à zéro.
- **Négatif**, indique que le résultat de l'opération arithmétique est négatif.
- **Débordement** (en anglais : *overflow*), indique que le résultat d'une opération arithmétique a excédé la capacité numérique de **R**.

Exemple d'une UAL 4-bits "le 74ls181"

Une UAL qui fut très utilisée au moment où seuls les circuits intégrés TTL étaient disponibles : le **74181**, une UAL de 4-bits. Ce circuit est destiné à être connecté en cascade avec d'autres pour traiter des entiers de taille supérieure à 4-bits.

Sa table de vérité décrivant quelle opération est accomplie suivant les valeurs des entrées M et S (Figure 5.60).

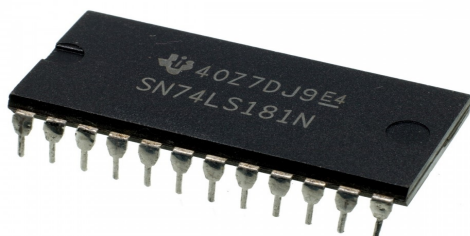


FIGURE 5.58 – Le circuit intégré UAL 74LS181

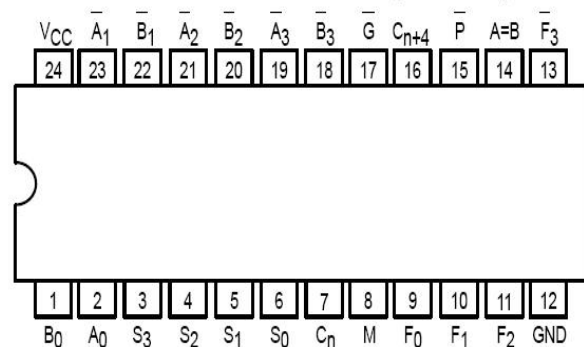


FIGURE 5.59 – Brochage du circuit intégré UAL 74LS181

SELECTION				ACTIVE-LOW DATA		
				M = H	M = L; ARITHMETIC OPERATIONS	
S3	S2	S1	S0	LOGIC FUNCTIONS	Cn = L (no carry)	Cn = H (with carry)
L	L	L	L	$F = \overline{A}$	$F = A \text{ MINUS } 1$	$F = A$
L	L	L	H	$F = \overline{AB}$	$F = AB \text{ MINUS } 1$	$F = AB$
L	L	H	L	$F = \overline{A} + B$	$F = \overline{AB} \text{ MINUS } 1$	$F = \overline{AB}$
L	L	H	H	$F = 1$	$F = \text{MINUS } 1 \text{ (2's COMP)}$	$F = \text{ZERO}$
L	H	L	L	$F = \overline{A} + \overline{B}$	$F = A \text{ PLUS } (A + \overline{B})$	$F = A \text{ PLUS } (A + \overline{B}) \text{ PLUS } 1$
L	H	L	H	$F = \overline{B}$	$F = AB \text{ PLUS } (A + \overline{B})$	$F = AB \text{ PLUS } (A + \overline{B}) \text{ PLUS } 1$
L	H	H	L	$F = A \oplus B$	$F = A \text{ MINUS } B \text{ MINUS } 1$	$F = A \text{ MINUS } B$
L	H	H	H	$F = A + \overline{B}$	$F = A + \overline{B}$	$F = (A + \overline{B}) \text{ PLUS } 1$
H	L	L	L	$F = \overline{AB}$	$F = A \text{ PLUS } (A + B)$	$F = A \text{ PLUS } (A + B) \text{ PLUS } 1$
H	L	L	H	$F = A \oplus B$	$F = A \text{ PLUS } B$	$F = A \text{ PLUS } B \text{ PLUS } 1$
H	L	H	L	$F = B$	$F = \overline{AB} \text{ PLUS } (A + B)$	$F = \overline{AB} \text{ PLUS } (A + B) \text{ PLUS } 1$
H	L	H	H	$F = A + B$	$F = (A + B)$	$F = (A + B) \text{ PLUS } 1$
H	H	L	L	$F = 0$	$F = A \text{ PLUS } A^\dagger$	$F = A \text{ PLUS } A \text{ PLUS } 1$
H	H	L	H	$F = \overline{AB}$	$F = AB \text{ PLUS } A$	$F = AB \text{ PLUS } A \text{ PLUS } 1$
H	H	H	L	$F = AB$	$F = \overline{AB} \text{ PLUS } A$	$F = \overline{AB} \text{ PLUS } A \text{ PLUS } 1$
H	H	H	H	$F = A$	$F = A$	$F = A \text{ PLUS } 1$

FIGURE 5.60 – Table de vérité de l'UAL 74ls181

Le schéma logique de ce circuit (UAL 74LS181) est donné par la Figure 5.61.

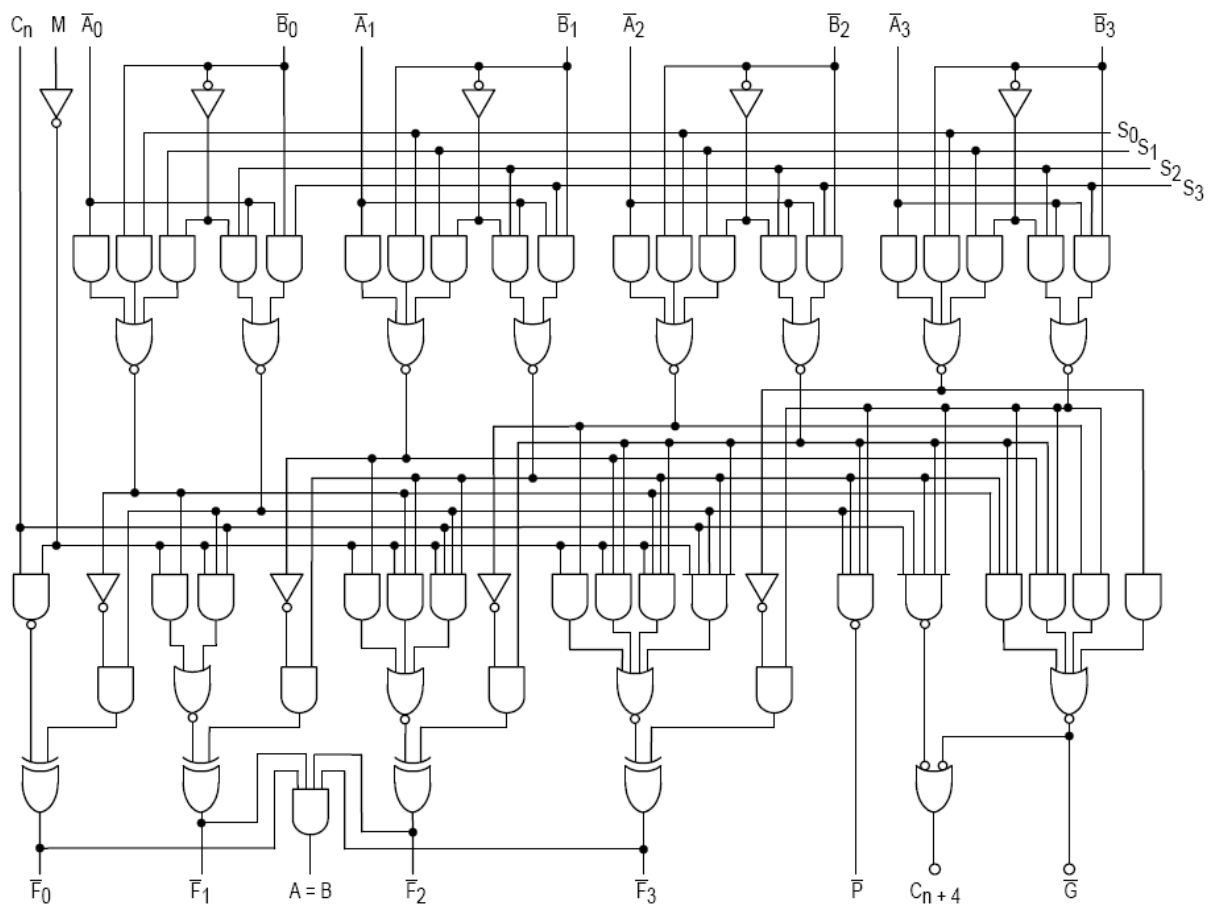


FIGURE 5.61 – Circuit logique de l'UAL 74ls181

5.5 Formes canoniques

Toutes les expressions booléennes peuvent être obtenues en utilisant les opérateurs NOT, AND et OR. Cependant, il est possible de réaliser toutes les opérations booléennes au moyen d'une seule sorte d'opérateurs : Opérateur NON-ET (NAND) et Opérateur NON-OU (NOR).

5.5.1 Utilisation d'opérateurs NAND

Par un montage convenable des opérateurs NAND, on réalise soit l'inversion ou complément logique, soit le produit logique, soit la somme logique.

Inversion logique

L'opérateur NAND monté selon la Figure 5.62 réalise une inversion logique. En effet :

$$S = \overline{E.E} \Rightarrow S = \overline{E} \quad (5.4)$$

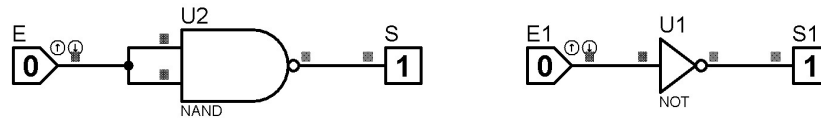


FIGURE 5.62 – Inverseur logique à base d'une porte logique NAND

Produit logique

Deux opérateurs NAND montés selon la Figure 5.63 réalisent un produit logique. En effet, la variable de sortie du premier opérateur, a pour expression $\overline{E_1.E_2}$. Le deuxième opérateur étant monté en inverseur logique, la variable de sortie S a pour expression :

$$S = \overline{\overline{E_1.E_2}} \Rightarrow S = E_1.E_2 \quad (5.5)$$

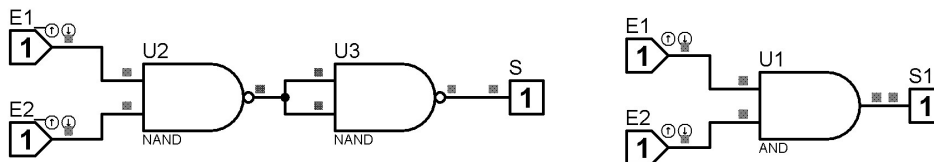


FIGURE 5.63 – Porte logique AND à base des portes logiques NAND

Somme logique

Trois opérateurs NAND montés selon la Figure 5.64 réalisent une somme logique. Les deux premiers opérateurs sont montés en inverseurs logiques et donnent en sortie les variables $\overline{E_1}$ et $\overline{E_2}$. A la sortie du troisième opérateur, la variable a pour expression :

$$S = \overline{\overline{E_1} \cdot \overline{E_2}} \quad (5.6)$$

En appliquant l'un des théorèmes de Morgan nous obtenons :

$$S = \overline{\overline{E_1} \cdot \overline{E_2}} \Rightarrow S = E_1 + E_2 \quad (5.7)$$

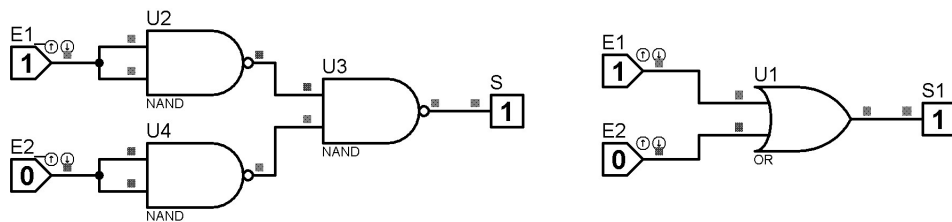


FIGURE 5.64 – Porte logique OR a base des portes logiques NAND

5.5.2 Utilisation d'opérateurs NOR

Par un montage convenable des opérateurs NOR, on réalise également soit l'inversion logique, soit le produit logique, soit la somme logique.

Inversion logique

L'opérateur NOR monte selon la Figure 5.65 réalise une inversion logique.

$$S = \overline{E + E} \Rightarrow S = \overline{E} \quad (5.8)$$

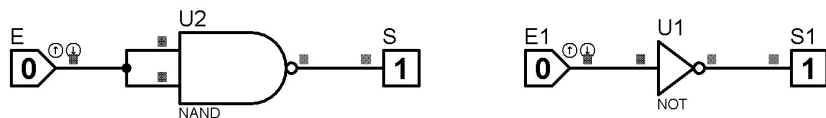


FIGURE 5.65 – Inverseur logique a base d'une porte logique NOR

Produit logique

Trois opérateurs NOR montés selon la Figure 5.66 réalisent un produit logique. Les deux premiers opérateurs sont montés en inverseurs logiques et donnent en sortie les

variables $\overline{E_1}$ et $\overline{E_2}$. A la sortie du troisième opérateur, la variable a pour expression :

$$S = \overline{\overline{E_1} + \overline{E_2}} \quad (5.9)$$

En appliquant l'un des théorèmes de Morgan nous obtenons :

$$S = \overline{\overline{E_1} + \overline{E_2}} \Rightarrow S = E_1.E_2 \quad (5.10)$$

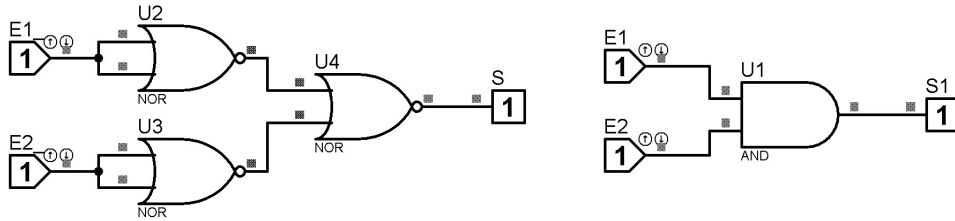


FIGURE 5.66 – Porte logique AND a base des portes logiques NOR

Somme logique

Deux opérateurs NOR montés selon la Figure 5.67 réalisent une somme logique. En effet, la variable de sortie du premier opérateur, a pour expression $\overline{E_1 + E_2}$. Le deuxième opérateur étant monté en inverseur logique, la variable S de sortie a pour expression :

$$S = \overline{\overline{E_1 + E_2}} \Rightarrow S = E_1 + E_2 \quad (5.11)$$

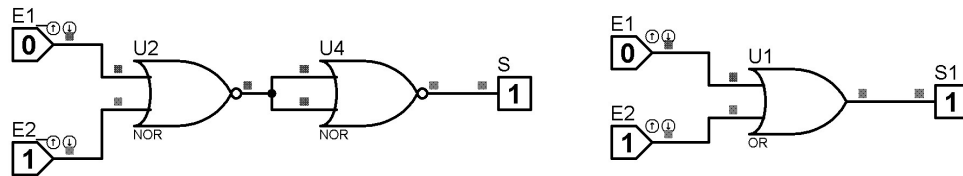


FIGURE 5.67 – Porte logique OR a base des portes logiques NOR

Chapitre 6

Table de KARNAUGH

6.1 Introduction

Une table de **Karnaugh** est une méthode graphique et simple pour trouver ou simplifier une fonction logique à partir de sa table de vérité. Elle utilise le code de Gray (aussi appelé binaire réfléchi), qui a comme propriété principale de ne faire varier qu'un seul bit entre deux mots successifs. Cette méthode a été développée par Maurice Karnaugh en 1953.

Un tableau de Karnaugh peut être vu comme une table de vérité particulière, à deux dimensions, destinées à faire apparaître visuellement les simplifications possibles.

Supposons $2n$ ou $2n + 1$ variables : on assignera par exemple n ou $n + 1$ variables au repérage des lignes, les n autres variables au repérage des colonnes. Chaque case élémentaire correspond alors à une seule ligne et à une seule colonne, donc à une seule combinaison des variables.

6.2 Méthode de recherche de l'équation

Pour trouver l'équation de la sortie, c'est simple. Il y a deux méthodes :

- Former une Somme des Produits.
- Former un Produit des Sommes.

6.2.1 Méthode de la Somme des Produits

Pour trouver une somme, il faut regrouper les valeurs de la sortie notée par la suite **S** égales à 1 ($S = 1$). Le nombre de 1 dans chaque groupe doit être égal à une puissance de 2 ($2^n, n = 1, 2, 3, \dots, etc$). Les groupes formés doivent être les moins nombreux possibles, mais ils doivent englober tous les 1. Un 1 peut être inclus dans plus d'un groupe, par contre aucun 0 ne doit être inclus. Les groupes sont composés d'une ou plusieurs colonnes et d'une ou plusieurs lignes.

Pour les tables à 4 variables, de préférence procéder dans l'ordre suivant :

- Le rectangle 16 cases puis,
- Les rectangles 8 cases puis,
- Les rectangles 4 cases puis,
- Les rectangles 2 cases et,
- Enfin les cases uniques.

Cette méthode, une fois assimilée, permet de trouver une équation au premier coup d'œil, et propose une alternative simple à la simplification d'équation, qui peut rapidement devenir fastidieuse.

6.2.2 Méthode du Produit des Sommes

Cette méthode ne regroupe pas les (1) mais les (0), pour trouver non pas une somme de produits mais un produit de sommes.

En regroupant les 0, on trouve la sortie **S** sous forme d'une somme, et par complément, on obtient **S'** sous forme de produit.

6.3 Avantages et utilisation

Les tables de Karnaugh sont surtout utilisées en électronique. En effet, la simplification de l'expression algébrique booléenne permet d'économiser des opérateurs logiques (*portes logiques*) et donc des circuits. Elle engendre aussi une économie de temps de conception et de fonds, tout en augmentant la fiabilité de l'ensemble.

6.4 Exemples

XY → ZT ↓	00	01	11	10
00	0	0	0	1
01	0	0	1	0
11	0	0	0	1
10	0	0	1	0

XY → ZT ↓	00	01	11	10
00	1	0	0	1
01	1	0	1	1
11	0	1	1	1
10	0	1	1	0

XY → ZT ↓	00	01	11	10
00	1	1	0	0
01	1	0	1	0
11	0	1	0	1
10	0	0	1	1

FIGURE 6.1 – Exemple de résolution des tableau de KARNAUGH

Les équations de sortie 6.1, 6.2, 6.3 représentent respectivement les équation logiques simplifiées tirées à partir des tableaux de KARNAUGH donnés par la Figure 6.1 du gauche

à droite.

$$S = X.(X \oplus Y \oplus Z \oplus T) \quad (6.1)$$

$$S = X.T + (\overline{Y \oplus Z}) \quad (6.2)$$

$$S = Y.T \oplus (\overline{X \oplus Z}) \quad (6.3)$$

Chapitre 7

Logique Séquentielle

7.1 Introduction

En théorie des circuits électroniques, la logique séquentielle est un type de logique dont les résultats ne dépendent pas seulement des données actuellement traitées (*les entrées*) mais aussi des données traitées précédemment. Elle s'oppose à la logique combinatoire, dont les résultats sont fonction et seulement fonction des entrées. En d'autres termes, la logique séquentielle utilise la notion de mémoire de stockage (*Bascules, registres, etc.*) alors que la logique combinatoire n'en a pas.

L'élément de base de la logique séquentielle est la **bascule**.

Une **bascule** est un circuit logique capable, dans certaines circonstances, de maintenir les valeurs de ses sorties malgré les changements des valeurs d'entrées, c'est-à-dire comportant un état « *mémoire* ». Il s'agit de l'élément qui permet le passage de la logique combinatoire à la logique séquentielle.

On distingue deux catégories principales de bascules dans un système séquentiel :

- **les bascules asynchrones** : dans ce mode de fonctionnement, la sortie logique peut changer d'état à tout moment quand une ou plusieurs entrées changent.
- **les bascules synchrones** : le changement d'état est commandé par un signal d'horloge, les informations évoluent en fonction du temps.

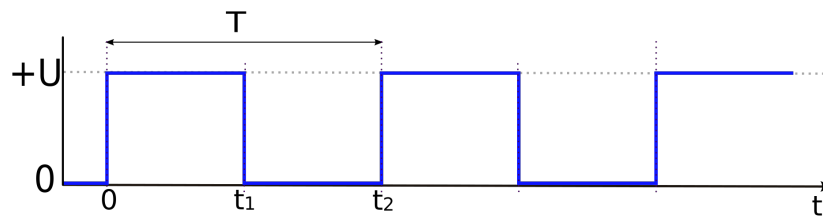
La différence entre ces deux types de bascules est que les bascules synchrones possèdent une entrée Horloge ce qui n'est pas le cas pour les bascules asynchrones.

7.2 Le Signal d'horloge

Un signal d'horloge est, en électronique, et particulièrement en électronique numérique, un signal électrique oscillant qui rythme les actions d'un circuit. Sa période est appelée cycle d'horloge (Figure 7.1).

À chaque cycle d'horloge, des calculs peuvent être effectués en utilisant les sorties

de bascules. L'horloge permet d'assurer que les données sont valides au cycle d'horloge suivant, c'est-à-dire que les calculs sont terminés et les résultats stabilisés. La durée du cycle doit donc être choisie en fonction de la durée maximale possible de chacun des calculs.

FIGURE 7.1 – Un signal d'horloge de période T

7.3 Les Bascules

Bascule ou (flip-flop) est un système permettant de mémoriser une information élémentaire (*mémoire à 1-bit*) ayant 2 états : Q et $\overline{Q}$, elle utilise un mécanisme de verrou (*latch*).

7.3.1 Bascule RS

Définition

La bascule **RS** est une bascule sans entrée d'horloge. Historiquement, c'est la première bascule créée en électronique. La bascule **RS** peut être réalisée avec des portes logiques de type « **NOR** » ou de type « **NAND** ». Il existe cependant des circuits intégrés spécialisés.

La Figure 7.2 représente le symbole générique d'une bascule RS plus le chronogramme de fonctionnement de cette bascule.

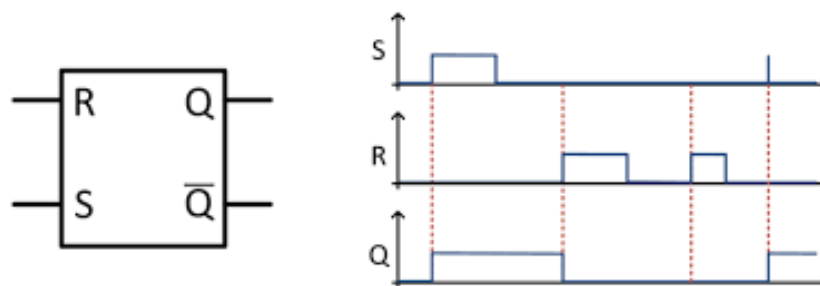


FIGURE 7.2 – Symbole d'une bascule RS

Bascule RS en technologie NAND

La manière la plus simple pour réaliser une bascule RS de base à un seul bit est de connecter ensemble une paire de portes logique NAND à 2 entrées couplées en croix, comme illustré (Figure 7.3), pour former une bascule Set-Reset, également connue sous le nom de bascule RS-NAND active sur l'état bas. on peut remarquer clairement il y ait une rétroaction de chaque sortie vers l'une des autres entrées de la porte NAND.

Ce dispositif se compose de deux entrées, l'une appelée Set, S , et l'autre Reset, R , avec deux sorties correspondantes Q et son inverse ou complément $\bar{Q}$ (not-Q), comme illustré dans la Figure 7.3.

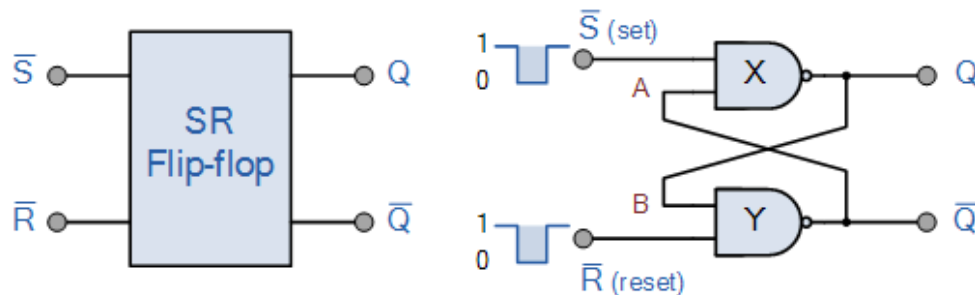


FIGURE 7.3 – Symbole et circuit logique d'une bascule RS en technologie NAND

Principe de mise à un

Considérons le circuit illustré dans la Figure 7.3. Si l'entrée R est au niveau logique "0" ($R = 0$) et l'entrée S est au niveau logique "1" ($S = 1$), la porte (NAND Y) a au moins une de ses entrées au niveau logique "0"; par conséquent, sa sortie $\bar{Q}$ doit être au niveau logique "1" (*principes de la porte NON-ET*). La sortie $\bar{Q}$ est également renvoyée à l'entrée "A" et donc les deux entrées de la porte (NAND X) sont au niveau logique "1", et donc sa sortie Q doit être au niveau logique "0".

Encore une fois, les principes des portes NAND. Si l'entrée de réinitialisation R change d'état et devient HAUT au niveau logique "1" avec S restant HAUT également au niveau logique "1", les entrées de la porte (NAND Y) sont maintenant $R = "1"$ et $B = "0"$. Comme l'une de ses entrées est toujours au niveau logique "0", la sortie à $\bar{Q}$ reste toujours HAUT au niveau logique "1" et il n'y a pas de changement d'état. Par conséquent, le circuit de bascule est dit "verrouillé" avec $\bar{Q} = 1$ et $Q = 0$.

Principe de mise à zéro

Dans ce deuxième état stable, $\bar{Q}$ est au niveau logique "0", ($\bar{Q} = 0$) sa sortie inverse à Q est au niveau logique "1", ($Q = 1$), et est donnée par $R = "1"$ et $S = "0"$.

Comme la porte (NAND X) a une de ses entrées au niveau logique "0", sa sortie Q doit être égale au niveau logique "1" (*encore une fois, les principes des portes NAND*). La sortie Q est renvoyée à l'entrée "B", de sorte que les deux entrées de la porte (NAND Y) sont au niveau logique "1", donc $\bar{Q} = 0$.

Si l'entrée S change maintenant d'état pour devenir un "1" logique avec l'entrée R qui reste à un "1" logique, la sortie $\overline{Q}$ reste toujours au niveau logique bas "0" et il n'y a pas de changement d'état. Par conséquent, l'état "Reset" des circuits de la bascule a également été verrouillé et nous pouvons définir cette action "set/reset" dans la table de vérité (Figure 7.4).

State	S	R	Q	$\overline{Q}$	Description
Set	1	0	0	1	Set $\overline{Q}$ » 1
	1	1	0	1	no change
Reset	0	1	1	0	Reset $\overline{Q}$ » 0
	1	1	1	0	no change
Invalid	0	0	1	1	Invalid Condition

FIGURE 7.4 – Table de vérité de la bascule NAND-RS

On peut voir que lorsque les deux entrées $S = 1$ et $R = 1$, les sorties Q et $\overline{Q}$ peuvent être au niveau logique 1 ou 0, selon l'état des entrées S ou R avant l'existence de cette condition d'entrée. Par conséquent, la condition $S = R = "1"$ ne change pas l'état des sorties Q et $\overline{Q}$.

Cependant, l'état des entrées $S = "0"$ et $R = "0"$ est une condition indésirable ou invalide et doit être évitée. La condition $S = R = "0"$ fait que les deux sorties Q et $\overline{Q}$ sont ensemble au niveau logique haut "1" alors que nous voudrions normalement que $\overline{Q}$ soit l'inverse de Q .

Le résultat est que la bascule perd le contrôle de Q et $\overline{Q}$, et si les deux entrées sont à nouveau commutées en niveau "HAUT" après cette condition à "1" logique, la bascule devient instable et passe à un état de données inconnu basé sur le déséquilibre, comme le montre le schéma de commutation de la Figure 7.5.

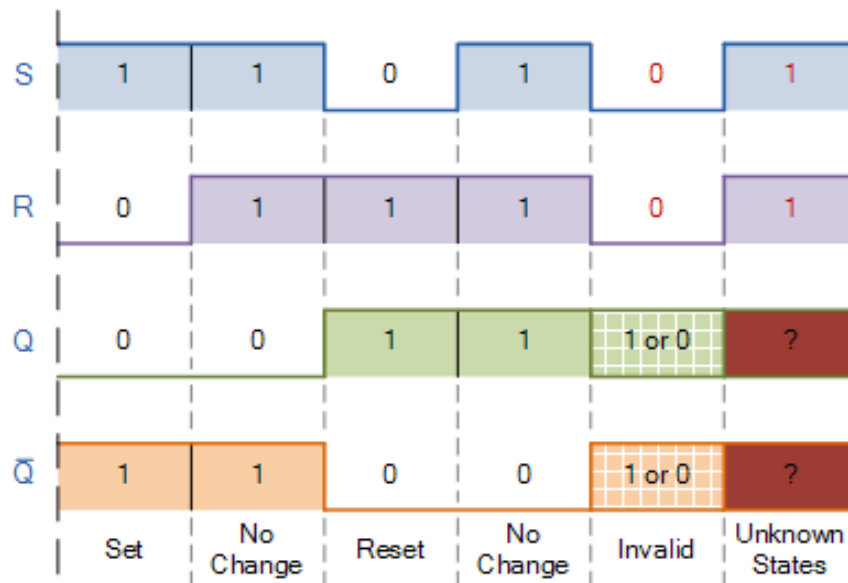


FIGURE 7.5 – Diagramme de commutation de la bascule flip-flop NAND-RS

Ce déséquilibre peut faire en sorte que l'une des sorties commute plus rapidement que l'autre, ce qui fait que la bascule passe à un état ou à un autre, qui peut ne pas être l'état requis, ce qui entraîne une corruption des données. Cette condition instable est généralement connue sous le nom d'état **méta-stable**.

Ensuite, une simple bascule NAND-RS peut être activée en appliquant un état logique bas (0) à son entrée **Set** et réinitialisée en appliquant ensuite un état logique (0) à son entrée **Reset**. On dit que la bascule RS est dans un état "invalide" (Méta-stable) si les deux entrées Set et Reset sont activées simultanément $Reset = Set = 0$.

Comme nous l'avons vu ci-dessus, la bascule RS de base à porte logique NAND nécessite des entrées logiques "0" pour basculer ou changer d'état de Q à $\bar{Q}$ et vice versa. Nous pouvons cependant transformer ce circuit de bascule de base en un circuit qui change d'état par l'application de signaux d'entrée positifs en ajoutant deux portes NAND (*En montage NOT*) supplémentaires connectées comme des inverseurs aux entrées S et R comme indiqué dans la Figure 7.6.

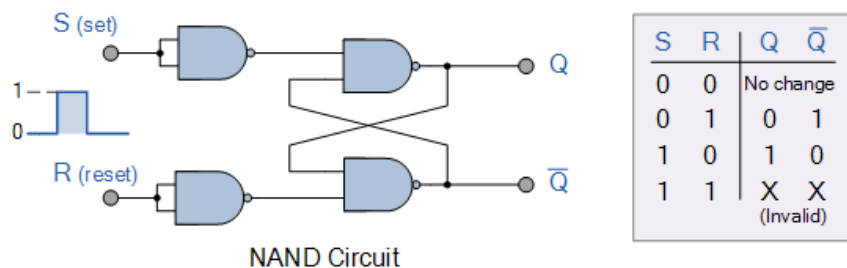


FIGURE 7.6 – Bascule NAND-RS active à l'état Haut

Bascule RS en technologie NOR

Outre l'utilisation de portes NAND, il est également possible de construire de simples bascules RS à un bit en utilisant deux portes logiques NOR couplées en croix et connectées dans la même configuration. Le circuit fonctionne de la même manière que le circuit à base de portes logiques NAND de la Figure 7.3, sauf que les entrées sont actives à l'état Haut et que la condition d'invalidité existe lorsque les deux entrées sont au niveau logique "1" (Set=Reset=1), comme illustré par la Figure 7.7.

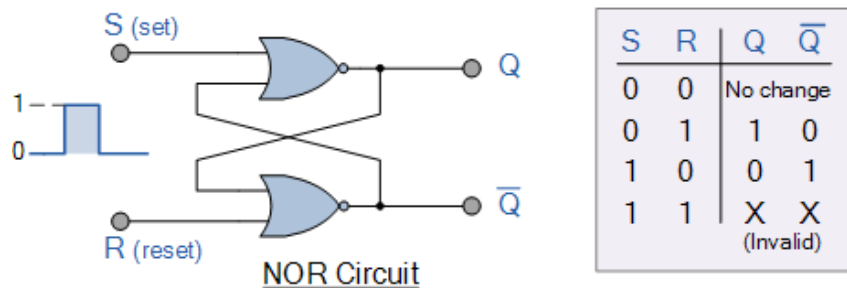


FIGURE 7.7 – Circuit logique d'une bascule RS en technologie NOR

Il est parfois souhaitable dans les circuits logiques séquentiels d'avoir une bascule RS bistable qui ne change d'état que lorsque certaines conditions sont remplies, indépendamment de l'état des entrées Set ou Reset (*R ou S*).

En connectant une porte AND à deux entrées en série avec chaque borne d'entrée de la bascule RS, on peut créer une bascule RS à déclenchement. Cette entrée conditionnelle supplémentaire est appelée entrée d'activation (**Enable**) et porte le préfixe "EN". L'ajout de cette entrée signifie que la sortie Q ne change d'état que lorsque ($EN = 1$) et peut donc être utilisée comme une entrée d'horloge (CLK), ce qui la rend sensible au niveau comme indiqué dans la Figure 7.8.

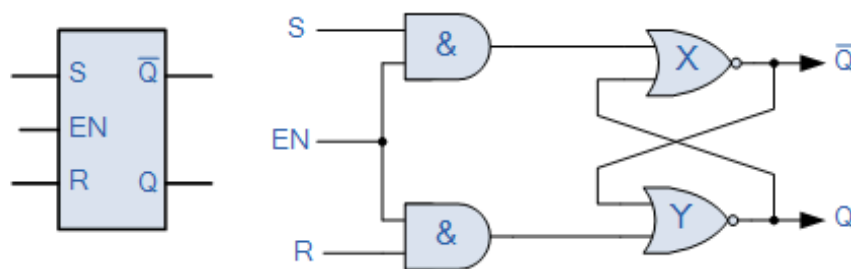


FIGURE 7.8 – Bascule RS avec entrée d'activation

Lorsque l'entrée de validation "EN" est au niveau logique "0", les sorties des deux portes AND sont également au niveau logique "0", (*principes des portes AND*) indépendamment de la condition des deux entrées S et R , verrouillant les deux sorties Q et $\bar{Q}$ dans leur dernier état connu.

Lorsque l'entrée de validation "EN" passe au niveau logique "1", le circuit répond comme une bascule RS bistable normale, les deux portes AND devenant transparentes aux signaux Set et Reset.

Cette entrée d'activation supplémentaire peut également être connectée à un signal d'horloge (**CLK**) pour ajouter une synchronisation d'horloge à la bascule, créant ainsi ce que l'on appelle parfois une "*Bascule RS cadencée*".

Ainsi, une bascule RS bistable avec entrée d'activation (**EN**) fonctionne comme une bascule bistable standard, mais les sorties ne sont activées que lorsqu'un "1" logique est appliqué à son entrée EN et désactivées par un "0" logique.

Exemples d'application

- *Commande d'un ascenseur*

on appuie sur le bouton, l'appel est enregistré et le voyant s'allume. si on relâche le bouton, le voyant reste allumé, il y a donc mémorisation.

- *Calculatrice avec un bouton ON et un bouton OFF*

le bouton ON met la calculatrice en marche. le bouton OFF arrête la calculatrice. si on appuie sur ON alors que la calculatrice est déjà en marche, elle reste en marche. si on appuie sur OFF alors qu'elle est arrêtée, elle reste arrêtée. Il y a donc mémorisation par une bascule RS.

7.3.2 Bascule D

Présentation de la Bascule D

La bascule de type D est une bascule RS modifiée avec l'ajout d'un inverseur pour empêcher les entrées S et R d'être au même niveau logique ($R = S$).

La bascule D surmonte l'un des principaux inconvénients du circuit bistable de base NAND-RS avec entrée d'activation **EN** en interdisant la condition d'entrée indéterminée ($R = S = 0$).

La bascule D est la plus importante de toutes les bascules synchronisées. En ajoutant un inverseur (porte NON) entre les entrées Set et Reset, les entrées S et R deviennent complémentaires l'une de l'autre, garantissant que les deux entrées S et R ne sont jamais égales ($R \neq S$) l'une à l'autre en même temps, ce qui nous permet de contrôler l'action de basculement de la bascule en utilisant une seule entrée D (*Data*).

Ensuite, cette entrée de données, appelée "D", est utilisée à la place du signal "Set" et l'inverseur est utilisé pour générer l'entrée complémentaire "Reset", ce qui permet de créer une bascule de type D sensible au niveau du signal CLK à partir d'une bascule RS sensible à un signal CLK, car maintenant $S = D$ et $R = \overline{D}$, comme indiqué dans la Figure 7.9 .

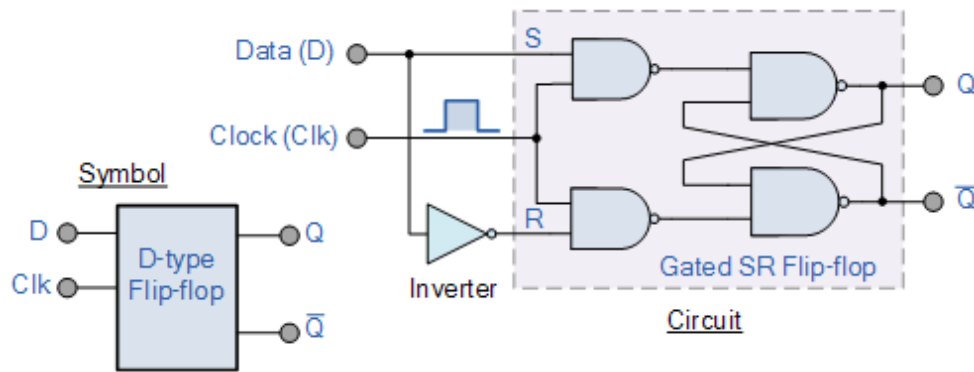


FIGURE 7.9 – Symbole et circuit logique de base d'une bascule D synchrone

Nous nous souvenons qu'une simple bascule RS nécessite deux entrées, une pour la mise à un de la sortie (SET) et une pour la remise à zéro de la sortie (RESET). En connectant un inverseur (*porte logique NOT*) à la bascule RS, nous pouvons mettre à zéro et à un la sortie Q de la **bascule D** en utilisant une seule entrée D car maintenant les deux signaux d'entrée sont complémentaires l'un de l'autre. Ce complément évite l'ambiguïté inhérente à la bascule RS lorsque les deux entrées sont à l'état BAS ($R = S = 0$), puisque cet état n'est plus possible.

Ainsi, cette entrée unique est appelée l'entrée "DATA". Si cette entrée de données est maintenue HAUT, la bascule est "SET $Q = 1$ " et lorsqu'elle est BAS, la bascule change et devient "RESET $Q = 0$ ". Cependant, cela serait plutôt inutile puisque la sortie de la bascule changerait toujours à chaque impulsion appliquée à cette entrée de données.

Pour éviter cela, une entrée supplémentaire appelée entrée "CLOCK" ou "ENABLE" est utilisée pour isoler l'entrée de données du circuit de verrouillage de la bascule après que les données souhaitées aient été stockées. L'effet est que la condition de l'entrée D n'est copiée sur la sortie Q que lorsque l'entrée horloge est active. Ceci constitue alors la base d'un autre dispositif séquentiel appelé "**D Flip Flop**".

Clk	D	Q		Description
↓ » 0	X	Q	$\overline{Q}$	Memory no change
↑ » 1	0	0	1	Reset Q » 0
↑ » 1	1	1	0	Set Q » 1

FIGURE 7.10 – Table de vérité de la bascule D

La **bascule D** stocke tout niveau logique appliqué à sa borne de données tant que l'entrée d'horloge est HAUT. Une fois que l'entrée de l'horloge passe au niveau BAS, les

entrées "set" et "reset" de la bascule sont toutes deux maintenues au niveau logique "1", de sorte qu'elle ne change pas d'état et stocke les données présentes sur sa sortie avant la transition de l'horloge. En d'autres termes, la sortie est "verrouillée" au niveau logique "0" ou "1" (Figure 7.10).

Utilisation de la bascule D pour la division de fréquence

Une des principales utilisations d'une bascule de type D est celle de diviseur de fréquence. Si la sortie $\overline{Q}$ d'une bascule de type D est connectée directement à l'entrée D , ce qui donne au dispositif une "rétroaction" en boucle fermée, des impulsions d'horloge successives feront "basculer" le bistable une fois tous les deux cycles d'horloge.

Dans la partie des compteurs, on va voir comment une bascule D peut être utilisée comme un "diviseur binaire" ou un "diviseur de fréquence" pour produire un circuit de comptage "divisé par 2", c'est-à-dire que la sortie a la moitié de la fréquence des impulsions d'horloge d'entrée. En plaçant une boucle de rétroaction autour de la bascule de type D, on peut construire un autre type de circuit de bascule appelé bascule de type T ou plus communément bistable de type T, qui peut être utilisé comme circuit de division par deux dans les compteurs binaires, comme indiqué dans la Figure 7.11.

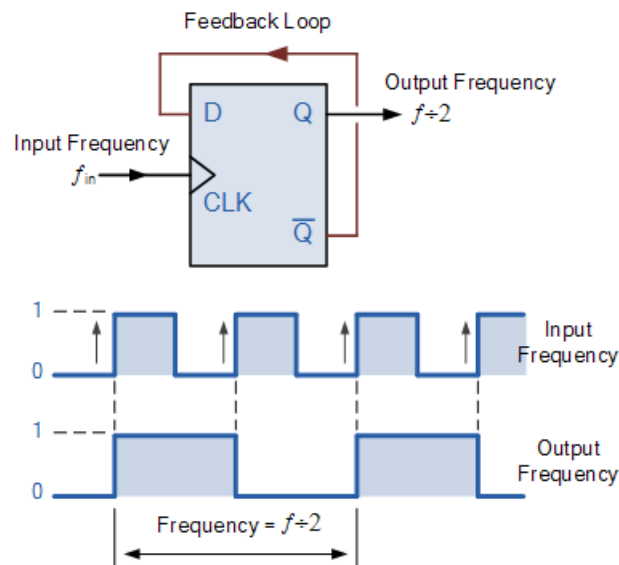


FIGURE 7.11 – Diviseur de fréquence par deux à l'aide d'une bascule D

Les formes d'onde de fréquence dans la Figure 7.11 montrent qu'en "renvoyant" la sortie de $\overline{Q}$ vers la borne d'entrée D , les impulsions de sortie à Q ont une fréquence qui est exactement la moitié ($\div 2$) de la fréquence d'horloge d'entrée, (f_{in}). En d'autres termes, le circuit produit une division de fréquence car il divise maintenant la fréquence d'entrée par un facteur de deux lorsque $Q = 1$ une fois tous les deux cycles d'horloge.

Les bascules D comme verrou "Data Latch"

En plus de la division de fréquence, une autre application utile de la **bascule D** est le verrouillage de données (Mémorisation). Une bascule D peut être utilisée comme un dispositif permettant de conserver ou de mémoriser les données présentes sur son entrée de données, agissant ainsi un peu comme un dispositif de mémoire à un bit, des circuits intégrés tels que le TTL-74LS74 ou le CMOS-4042 sont disponibles en format Quad exactement dans ce but. En connectant ensemble quatre verrous de données de 1-bit de sorte que toutes leurs entrées d'horloge soient connectées ensemble et soient "cadencées" en même temps, un verrou de données simple de "4-bits" peut être fait comme montré dans la Figure 7.12 .

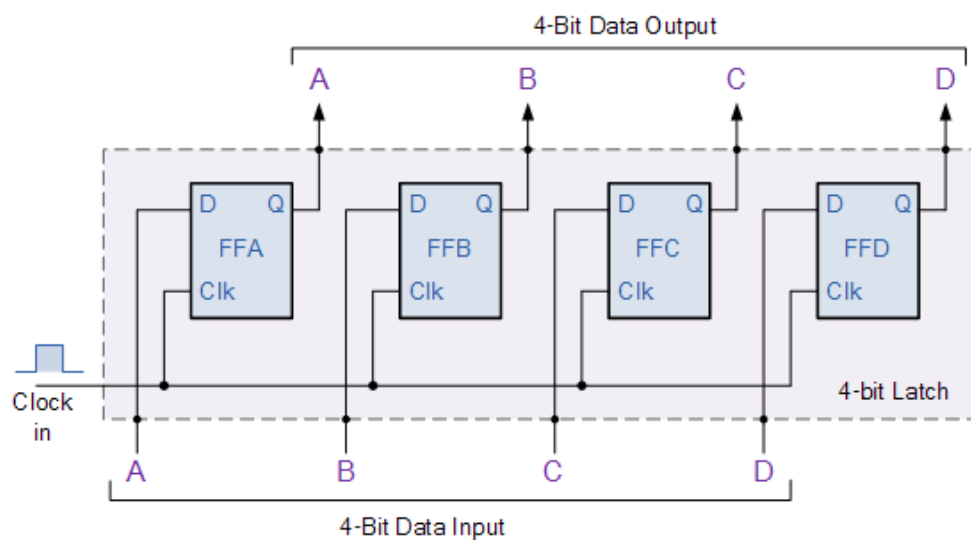


FIGURE 7.12 – Verrou de données 4-bits "*4-bits data latch*"

7.3.3 Bascule JK

Introduction

Contrairement à la bascule JK, le circuit de base de la bascule RS-NAND présente de nombreux avantages et utilisations dans les circuits logiques séquentiels, mais il souffre de deux problèmes de commutation fondamentaux.

1. La condition $Set = 0$ et $Reset = 0$ ($S = R = 0$) doit toujours être évitée.
2. Si Set ou Reset changent d'état alors que l'entrée de validation (EN) est haute, l'action de verrouillage correcte peut ne pas se produire.

Pour surmonter ces deux problèmes fondamentaux de conception de la bascule RS, la bascule JK a été développée.

Cette simple bascule JK est la plus utilisée de toutes les bascules et est considérée comme un circuit de (*bascule*) universel. Les deux entrées marquées "J" et "K" ne sont pas des abréviations d'autres mots, comme "S" pour Set et "R" pour Reset, mais sont

elles-mêmes des lettres autonomes choisies par son inventeur *Jack Kilby* pour distinguer la conception de la bascule des autres types.

Fonctionnement de la bascule JK

Le fonctionnement séquentiel de la bascule JK est exactement le même que celui de la bascule RS précédente avec les mêmes entrées "Set" et "Reset". La différence cette fois-ci est que la "**bascule JK**" n'a pas d'états d'entrée invalides ou interdits de la bascule RS, même lorsque S et R sont tous deux à "1" logique.

La bascule JK est essentiellement une bascule RS à déclenchement avec l'ajout d'un circuit d'entrée d'horloge qui empêche la condition de sortie illégale ou invalide qui peut se produire lorsque les deux entrées S et R sont égales au niveau logique "1" ($S = R = 1$). En raison de cette entrée cadencée supplémentaire, une bascule JK a quatre combinaisons de sortie possibles, "1 logique", "0 logique", "aucun changement" et "bascule". Le symbole d'une bascule JK est similaire à celui d'une bascule RS bistable vue dans la section précédente, à l'exception de l'ajout d'une entrée d'horloge (Figure 7.13).

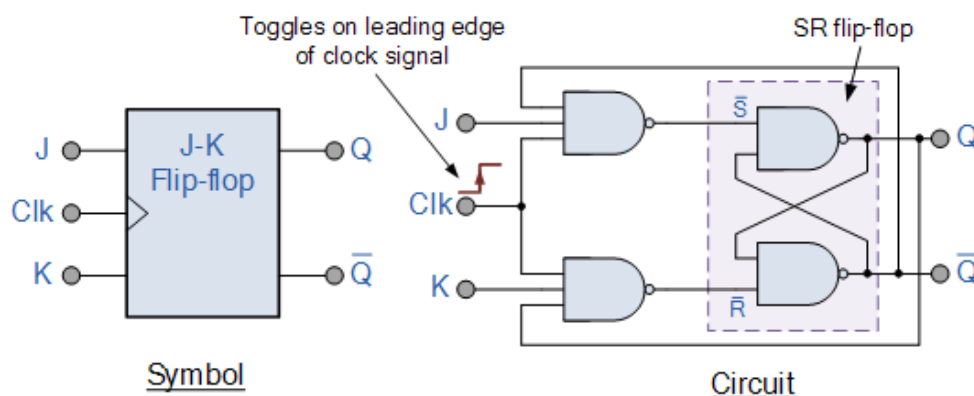


FIGURE 7.13 – Symbole et circuit logique d'une bascule JK

Les entrées S et R du précédente bascule RS ont été remplacées par deux entrées appelées respectivement J et K . Cela équivaut donc à : $J = S$ et $K = R$.

Les deux portes AND à deux entrées de la bascule RS à entrée d'activation (EN) ont été remplacées par deux portes NAND à 3 entrées, la troisième entrée de chaque porte étant connectée aux sorties Q et $\bar{Q}$. Ce couplage croisé de la bascule RS permet d'utiliser la condition précédemment invalide de l'état $S = "1"$ et $R = "1"$ pour produire une "**action de basculement**" car les deux entrées sont maintenant verrouillées (*interlocked*).

Si le circuit est maintenant "SET", l'entrée J est inhibée par l'état "0" de $\bar{Q}$ à travers la porte NAND inférieure. Si le circuit est "RESET", l'entrée K est inhibée par l'état "0" de Q à travers la porte NAND supérieure. Comme Q et $\bar{Q}$ sont toujours différents, nous pouvons les utiliser pour contrôler l'entrée. Lorsque les deux entrées J et K sont égales à "1" logique ($J = K = 1$), la bascule JK bascule (*change l'état*) comme indiqué dans la table de vérité dans la Figure .

	Clock	Input		Output		Description
	Clk	J	K	Q	$\overline{Q}$	
same as for the SR Latch	X	0	0	1	0	Memory no change
	X	0	0	0	1	
	$\overline{1}$	0	1	1	0	Reset Q » 0
	X	0	1	0	1	
	$\overline{1}$	1	0	0	1	Set Q » 1
	X	1	0	1	0	
toggle action	$\overline{1}$	1	1	0	1	Toggle
	$\overline{1}$	1	1	1	0	

FIGURE 7.14 – Table de vérité de la bascule JK

La bascule JK est donc fondamentalement une bascule RS avec une rétroaction qui permet à une seule de ses deux bornes d'entrée, SET ou RESET, d'être active à tout moment dans le cadre d'une commutation normale, éliminant ainsi la condition invalide observée précédemment dans le circuit de la bascule RS.

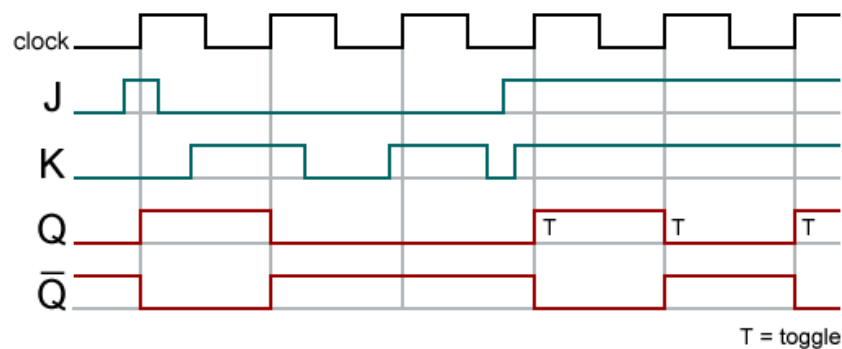


FIGURE 7.15 – Chronogramme de fonctionnement de la bascule JK

Cependant, si les entrées J et K sont toutes deux à "1" logique ($J = K = 1$), lorsque l'entrée d'horloge passe à "1", le circuit va "**basculer**" car ses sorties commutent et changent d'état de manière complémentaire. Ainsi, la bascule JK se comporte plus comme une bascule de type T lorsque les deux bornes sont "HIGH". Cependant, comme les sorties sont renvoyées vers les entrées, cela peut faire osciller la sortie sur Q entre SET et RESET de façon continue après avoir été complétée une fois.

7.3.4 Bascule T

La bascule T tire son nom du terme anglais 'toggle'. Si son entrée T est active, elle bascule à chaque impulsion d'horloge d'où son nom. Si son entrée T est inactive, elle conserve son état.

Elle n'existe pas intégrée sauf dans des PLD, FPGA... mais on peut la fabriquer avec une bascule D en reliant la sortie $\overline{Q}$ à l'entrée D (*toutefois on ne réalise ainsi qu'une bascule T avec $T=1$*), ou à l'aide d'une bascule JK en reliant J et K pour faire l'entrée T.

Comme la bascule T a la propriété de basculer à chaque impulsion d'horloge, elle réalise donc une division de fréquence par 2 ; elle servira aussi de base à la réalisation de compteurs (*association simple si actives au front descendant de l'horloge*). Intégrée dans un circuit, elle est souvent active sur un front descendant.

- Si $T = 1$, on bascule à chaque impulsion d'horloge.
- Si $T = 0$, la sortie ne change pas : état mémoire.

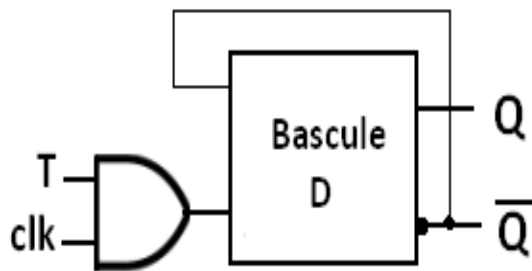


FIGURE 7.16 – Bascule T à base d'une bascule D

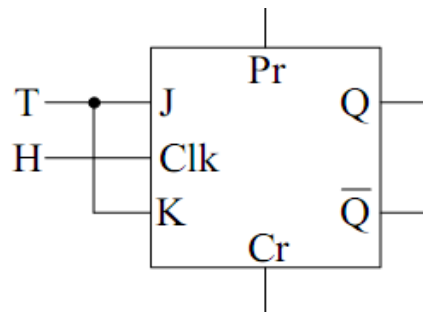


FIGURE 7.17 – Bascule T à base d'une bascule JK

7.4 Les Registres

7.4.1 Introduction

Un registre est un circuit séquentiel synchronisé avec l'unité central de traitement (CPU) qui emmagasine un certain nombre de bits, en utilisant un ensemble de bascules. C'est un ensemble de bascules synchronisées par le même signal d'horloge.

Les registres sont largement utilisés dans les systèmes de traitement numérique (*les ordinateurs par exemple*) pour réaliser des opérations : de mémorisation provisoire (*mémoire tampon*), de décalage, de rotation, ... etc.

On distingue donc deux types de registres :

1. les registres de mémorisation.
2. les registres de décalage.

7.4.2 Caractéristiques

Tout registre est caractérisé par :

- **La capacité** : nombre de bits du mot binaire qu'il peut mémoriser.
- **Le mode d'écriture ou de chargement** : dépend du nombre d'entrées :
 - *écriture série* : génération bit par bit, avec transmission par un seul fil conducteur.
 - *écriture parallèle* : génération globale du mot de n-bits, avec transmission par un bus de n-bits (*n fils conducteurs*).
- **Le mode de lecture** :
 - *lecture série* : exploitation bit par bit du mot (*une seule sortie*).
 - *lecture parallèle* : exploitation globale du mot (*n sorties*).

Selon le mode d'accès en écriture (*entrée*) et en lecture (*sortie*), série ou parallèle, on distingue quatre types de registre :

- Registre à entrée série et sortie série.
- Registre à entrée série et sortie parallèle.
- Registre à entrée parallèle et sortie série.
- Registre à entrée parallèle et sortie parallèle.

Ces quatre types peuvent être classés en deux catégories : les registres de mémorisation (tampon) et les registres à décalage.

7.4.3 Registre de Mémorisation

Ce registre permet la mémorisation d'une information binaire sous forme d'un mot de n-bits. Il est donc constitué de **n** bascules, mémorisant chacune un bit. L'information sur n-bits est chargée (*écrite*) puis elle est conservée et devient disponible en lecture.

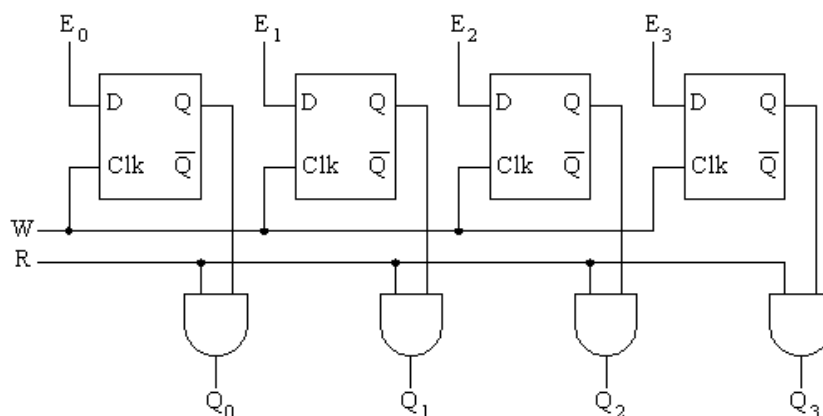


FIGURE 7.18 – Registre de mémorisation 4-bits réalisé à base de bascules D

Le registre mémorise les états des entrées E_0 , E_1 , E_2 et E_3 . Ces états sont conservés jusqu'au prochain signal de commande. Les données mémorisées peuvent être lues sur les sorties Q_0, Q_1, Q_2 et Q_3 .

7.4.4 Registre à Décalage

Un registre à décalage est un registre ayant la possibilité de décaler à droite ou à gauche son contenu.

Dans un registre à décalage les bascules sont interconnectées de façon à ce que l'état logique de la sortie de la bascule de $rang(i)$ puisse être transmis à la bascule de $rang(i+1)$ lorsqu'un signal d'horloge est appliqué à l'ensemble des bascules. On distingue quatre types de registres à décalage :

Registre à décalage entrée série - sortie série

Les bits d'information sont présentés séquentiellement bit après bit à l'entrée de la première bascule et se propagent à travers le registre à chaque impulsion du signal d'horloge, pour sortir par la dernière bascule.

Décalage à droite :

Dans ce cas l'entrée de la bascule D de $rang(i)$ doit être connectée à la sortie de la bascule de $rang(i-1)$ (Figure 7.19).

$$D1 = E; D_i = Q_{i-1} (i > 1)$$

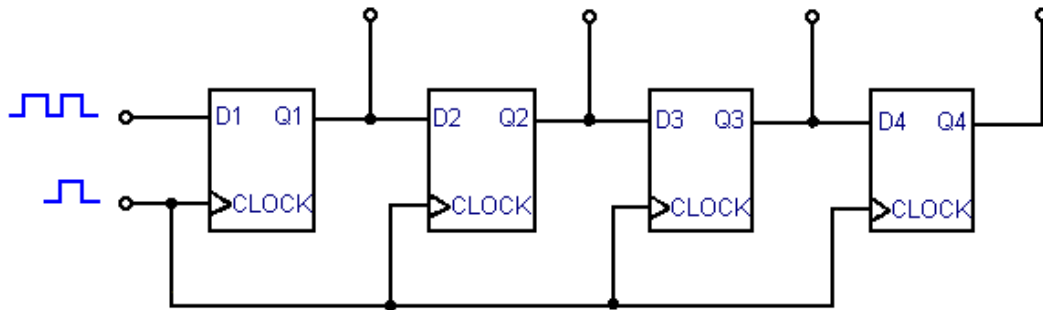


FIGURE 7.19 – Registre à décalage à droite 4-bits réalisé à base de bascules D

Décalage à gauche :

Dans ce cas l'entrée de la bascule D de $rang(i)$ doit être connectée à la sortie de la bascule de $rang(i+1)$.

$$D4 = E; D_i = Q_{i+1} (i < n; n = 4)$$

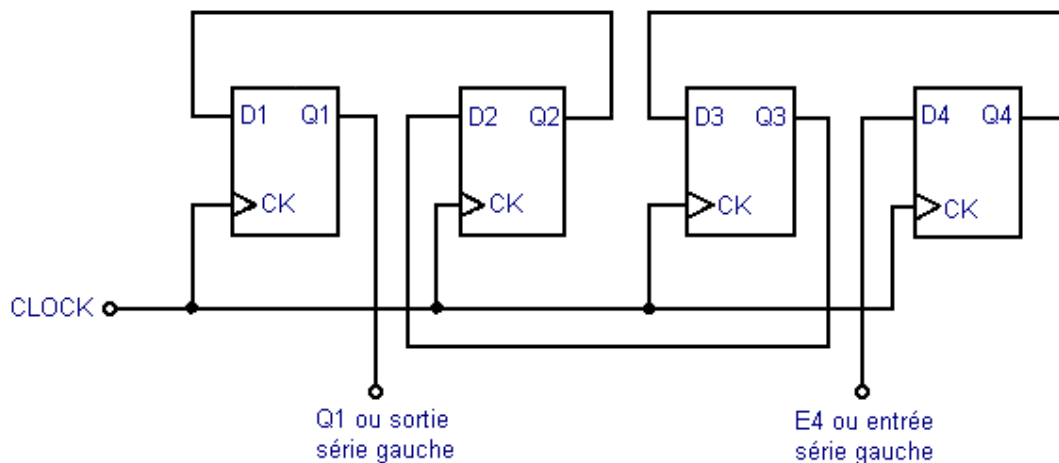


FIGURE 7.20 – Registre à décalage à gauche 4-bits réalisé à base de bascules D

Registre à décalage entrée série - sortie parallèle

Ce type de registre permet de transformer un codage temporel (*succession des bits dans le temps*) en un codage spatial (*information stockée en mémoire statique*).

La figure 7.21 donne un exemple de registre de 4-bits à entrée série et sortie parallèle réalisé avec des bascules D. Le signal R (*Read*) n'est pas obligatoire, il permet juste de commander la lecture des sorties en mêmes temps, de façon à éviter la lecture au moment du chargement.

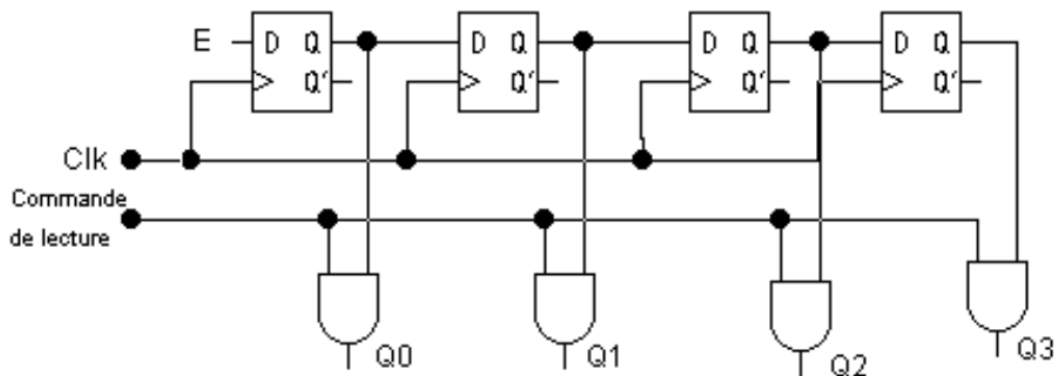


FIGURE 7.21 – Schema d'un registre à décalage entrée série - sortie parallèle

Registre à décalage entrée parallèle - sortie série

Les registres série-série ou série-parallèle permettent de décaler vers la droite des informations en les appliquant une à une sur l'entrée série.

Cela revient à dire que les données sont disponibles en série, soit sur un seul fil. Mais, il peut se présenter le cas où plusieurs données sont disponibles simultanément. Il faut

donc pouvoir faire entrer ces données en même temps dans le registre ; ceci se réalise par l'intermédiaire de plusieurs entrées dites parallèles.

L'opération qui consiste à positionner chaque bascule du registre avec le niveau présent sur l'entrée parallèle correspondante se nomme le chargement (*LOAD en anglais*) du registre. Ce chargement peut se faire de façon **Asynchrone** ou **Synchrone** à l'aide d'une entrée de commande appelée **SHIFT/LOAD**.

Si le chargement est asynchrone, dès que l'entrée (*SHIFT/LOAD*) est activée, chaque sortie du registre recopie l'état présent sur son entrée parallèle.

Si par contre le chargement est synchrone, il faut en plus appliquer une impulsion d'horloge pour que chaque bascule du registre mémorise l'état précédent sur son entrée parallèle. Si l'entrée (*SHIFT/LOAD*) n'est pas activée, le registre fonctionne en mode série-série.

Chargement Asynchrone :

La Figure 7.22 représente un registre parallèle-série 4-bits (*parce qu'il comporte 4 étages*) dont le chargement s'effectue de façon asynchrone.

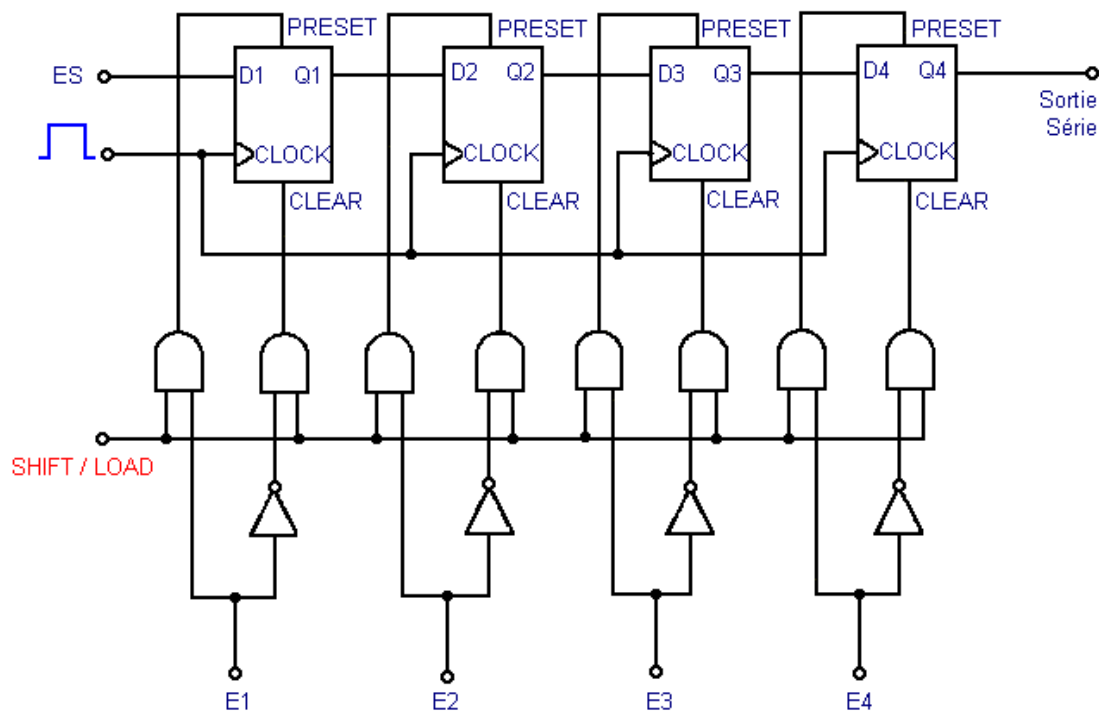


FIGURE 7.22 – registre parallèle-série 4-bits Asynchrone

Par rapport aux registres précédents, il apparaît un réseau combinatoire de portes logiques. Celles-ci agissent sur les entrées asynchrones **CLEAR** et **PRESET** de chaque bascule.

Le chargement du registre, opération qui consiste à positionner les sorties $Q1, Q2, Q3, Q4$ avec les niveaux logiques présents sur les entrées parallèles $E1, E2, E3, E4$, sera donc asyn-

chrone.

Les deux modes de fonctionnement, décalage et chargement, sont différenciés par l'entrée de commande *SHIFT/LOAD*.

Suivant le niveau appliqué à cette entrée, le registre fonctionne en mode **SHIFT**, c'est-à-dire en mode décalage ou bien en mode **LOAD**, c'est-à-dire en mode chargement.

EXAMEN DU MODE LOAD

Pour procéder au chargement (parallèle) du registre, il faut appliquer un niveau H à l'entrée *SHIFT / LOAD*.

En effet, les portes AND du réseau se trouvent validées et les entrées *E1, E2, E3 et E4* agissent ainsi sur les entrées **CLEAR** et **PRESET** de chaque bascule. Pour comprendre comment s'effectue le chargement asynchrone du registre, examinons l'action de l'entrée *E1* sur la première bascule.

Puisque l'entrée (*SHIFT/LOAD*) est portée au niveau H, le niveau logique appliquée en *E1* se retrouve sur l'entrée **PRESET** de la bascule, tandis que l'entrée **CLEAR** reçoit le niveau logique opposé à celui de *E1*.

Les entrées **CLEAR** et **PRESET** étant actives au niveau *H*, lorsque *E1* est au niveau *H*, l'entrée **PRESET** devient active et la sortie *Q1* de la bascule passe donc au niveau *H*.

Si par contre, l'entrée *E1* est au niveau *L*, c'est l'entrée **CLEAR** qui devient active et donc la bascule se porte au niveau *L*.

En résumé, lorsque l'entrée (*SHIFT/LOAD*) est au niveau *H*, la sortie *Q1* «recopie» l'entrée *E1*. Il en va de même pour les autres sorties *Q2, Q3, Q4* qui recopient respectivement les entrées *E2, E3, E4*.

Tant que nous sommes en phase de chargement, le signal d'horloge n'a aucune influence puisque l'une des deux entrées **CLEAR** ou **PRESET** est active, donc prioritaire.

EXAMEN DU MODE SHIFT

Ramenons maintenant l'entrée (*SHIFT/LOAD*) au niveau *L*. Dès lors, les entrées **CLEAR** et **PRESET** deviennent inactives puisqu'elles se portent au niveau *L* indépendamment du niveau des entrées parallèles. Le signal d'horloge devient prépondérant et le registre fonctionne en mode **SHIFT**.

A chaque impulsion d'horloge, la donnée présente en *D1* se transfère en *Q1*, celle présente en *Q1* se transfère en *Q2* et ainsi de suite jusqu'à la sortie *Q4*.

Il est à noter que la donnée présente en *Q4* est perdue à chaque impulsion d'horloge. Les données présentes sur (*E1, E2, E3, E4*) peuvent varier, elles n'ont aucune influence sur le fonctionnement en mode décalage du registre.

En résumé, l'information sur 4-bits présente sur les entrées parallèles est chargée sur un niveau haut de l'entrée (*SHIFT/LOAD*). En ramenant cette entrée au niveau *L*, l'information chargée se décale vers la droite d'un bit à chaque impulsion d'horloge.

Chargement Synchrone :

Le chargement parallèle des données peut s'effectuer de deux manières : synchrone ou asynchrone. Etudions le cas de chargement synchrone. Dans ce cas il faut appliquer les données aux entrées synchrones D_i . En fonction de l'ordre de chargement (écriture), chaque bascule recopie l'entrée E_i ou bien la sortie de la bascule $i-1$, à chaque front d'horloge. Il faut donc utiliser un multiplexeur 2 vers 1 à l'entrée D_i de chacune des bascules 1 à $n-1$.

Le registre précédent (Figure 7.22) permettait de pré-positionner son contenu de façon asynchrone. Nous allons voir à présent comment effectuer le chargement du registre de façon synchrone.

Pour obtenir ce résultat, il suffit de remplacer le réseau combinatoire de la Figure 7.22 par un autre réseau agissant non plus sur les entrées asynchrones **CLEAR** et **PRESET**, mais sur les entrées synchrones ($D1, D2, D3, D4$) des bascules.

Chaque partie du réseau propre à une bascule comprend une entrée de commande correspondant à l'entrée (*SHIFT/LOAD*), deux entrées de données et une sortie.

La Figure 7.23 montre comment est structuré le registre parallèle-série avec chargement synchrone.

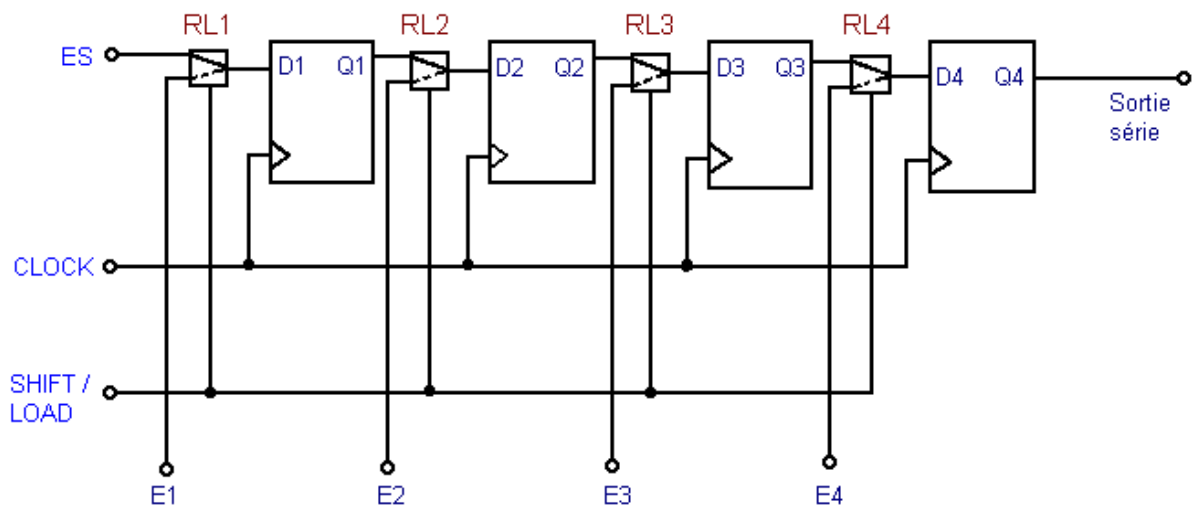


FIGURE 7.23 – structure d'un registre parallèle-série avec chargement synchrone

Chacune des parties du réseau combinatoire, repérées par les symboles ($RL1, RL2, RL3, RL4$) sur la Figure 7.23, peut être assimilée à un aiguillage de données logiques (*Multiplexage*).

Suivant le niveau de l'entrée de commande, le circuit «aiguillera» l'une ou l'autre des deux entrées vers la sortie. Autrement dit, la sortie «recopiera» l'une des deux entrées. Analysons par exemple, le premier réseau combinatoire $RL1$ du montage de la Figure 7.23, les trois autres étant strictement identiques au premier.

Supposons que lorsque l'entrée (*SHIFT/LOAD*) est à l'état 0, $D1 = ES$ et que $D1 = E1$ lorsque l'entrée (*SHIFT/LOAD*) est à l'état 1 (*l'inverse pouvant exister*).

EXAMEN DU MODE LOAD

Si l'entrée (SHIFT / LOAD) est à l'état 1, la sortie de chaque réseau est au même niveau logique que l'entrée parallèle correspondante.

Autrement dit, $D1 = E1$, $D2 = E2$, $D3 = E3$ et $D4 = E4$.

Nous pouvons dire que les quatre entrées ($E1, E2, E3, E4$) sont aiguillées respectivement vers les entrées ($D1, D2, D3, D4$) des bascules. Le registre est alors prêt à être chargé avec les niveaux présents sur les entrées parallèles.

Il suffit d'envoyer une impulsion d'horloge sur l'entrée CLOCK pour que $Q1 = E1$, $Q2 = E2$, $Q3 = E3$ et $Q4 = E4$. Le chargement du registre est donc bien synchrone.

EXAMEN DU MODE SHIFT

Si l'entrée (SHIFT/LOAD) est à l'état 0, le registre fonctionne en mode **SHIFT** ou décalage. En effet, l'entrée ES se trouve reliée à travers le premier réseau à l'entrée $D1$ de la première bascule.

De même, les sorties ($Q1, Q2, Q3$) sont reliées respectivement aux entrées ($D2, D3, D4$) au travers des deuxième, troisième et quatrième réseaux RL .

Nous pouvons dire que $D1 = ES$, $D2 = Q1$, $D3 = Q2$ et $D4 = Q3$. A chaque impulsion d'horloge, le contenu du registre est décalé d'un pas vers la droite. La donnée présente en ES est mémorisée en $Q1$, tandis que la donnée présente en $Q4$ est perdue.

En résumé, en combinant les deux modes de fonctionnement du registre, nous pouvons, dans un premier temps, charger celui-ci avec une information, puis dans un second temps, décaler celle-ci d'un ou plusieurs pas vers la droite.

7.5 Les Compteurs

Un compteur est un circuit séquentiel qui permet de dénombrer des impulsions appliquées sur son entrée d'horloge (CLK : *clock*) et de restitué sur ces sorties des informations sous forme binaire. A chaque impulsion l'état du compteur est modifié et entre deux impulsions son état reste stable, un compteur est donc aussi une sorte de registre mémoire. Un compteur doit démarrer vide ou plein suivant l'utilisation, l'entrée **RAZ** ou **CLR** ou **MR** ou **R** sert de remise à zéro. L'entrée **CE** (*count enable*) servant à valider le circuit ou interrompre le comptage.

7.5.1 Compteurs Asynchrones

Compteurs Asynchrones à base de bascules D

Le compteur asynchrone est basé sur le diviseur de fréquence. Il s'agit d'une simple bascule D dont la sortie inverseuse est envoyée sur son entrée D. Ainsi, à chaque front montant de l'entrée d'horloge, la donnée transférée sur la sortie est complémentée. La fréquence du signal est donc divisée par deux.

Un compteur asynchrone est simplement constitué de plusieurs diviseurs de fréquences (*autant que de bits requis*). Si l'on désire un compteur, les entrées d'horloge doivent être actives sur front descendant. Cela conduit au schéma de la Figure 7.24 :

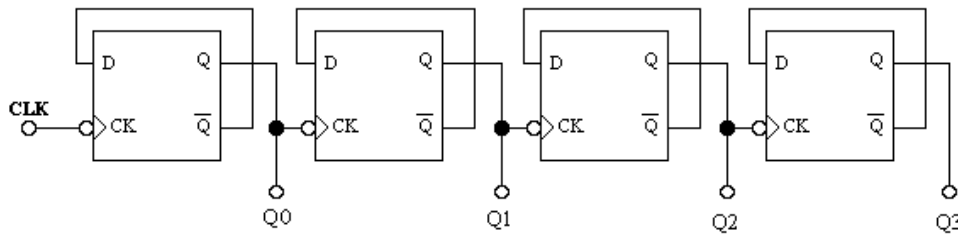


FIGURE 7.24 – Compteur Asynchrone modulo 16 à base des bascules D

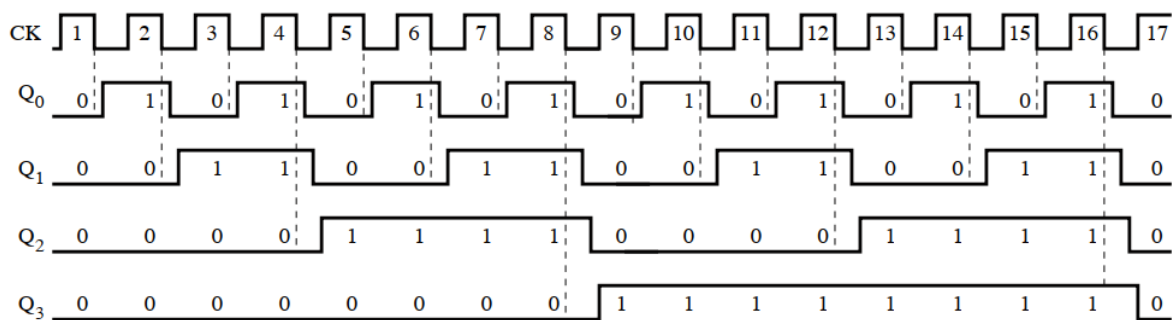


FIGURE 7.25 – Chronogramme d'un Compteur Asynchrone modulo 16 à base des bascules D

Comme vous pouvez le constater, ($Q_3Q_2Q_1Q_0$) sont successivement égales à 0000, 0001, 0010, ..., 1111, c'est-à-dire : 0, 1, 2, ..., 15. Bien sûr ce cycle continue indéfiniment, aussi longtemps que le signal d'horloge est présent. C'est la façon la plus simple de réaliser un compteur, et bien sûr ce type de compteur est disponible sous forme de divers circuits intégrés prêts à l'emploi.

Compteurs Asynchrones à base de bascules JK

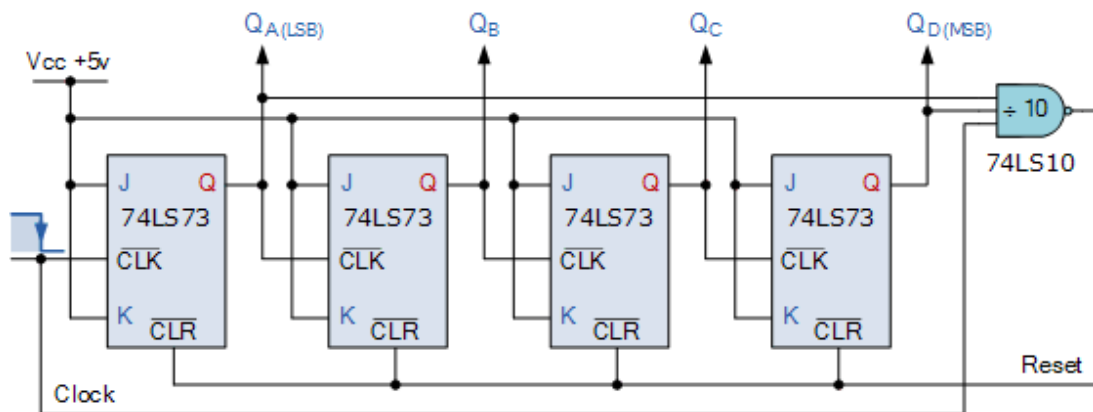


FIGURE 7.26 – Compteur à décade (*modulo 10*) asynchrone à base des bascules JK

Ce type de compteur asynchrone (Figure 7.26) compte vers le haut sur chaque front descendant du signal d'horloge d'entrée en partant de 0000 jusqu'à atteindre une sortie 1001 (9 *décimal*). Les deux sorties $Q_{A(LSB)}$ et $Q_{D(LSB)}$ sont maintenant égales à "1" logique. Lors de l'application de l'impulsion d'horloge suivante, la sortie de la porte (NON-ET 74LS10) change d'état, passant du niveau logique "1" au niveau logique "0".

Comme la sortie de la porte NON-ET est connectée aux entrées CLEAR ("CLR") de toutes les bascules (J-K 74LS73), ce signal provoque la réinitialisation de toutes les sorties Q à 0000 binaire au compte de 10. Comme les sorties Q_A et Q_D sont maintenant toutes deux égales à un "0" logique puisque les bascules viennent d'être réinitialisées, la sortie de la porte NAND revient à un niveau logique "1" et le compteur redémarre de nouveau à partir de 0000. Nous avons maintenant un compteur **décimal** ou **modulo-10**.

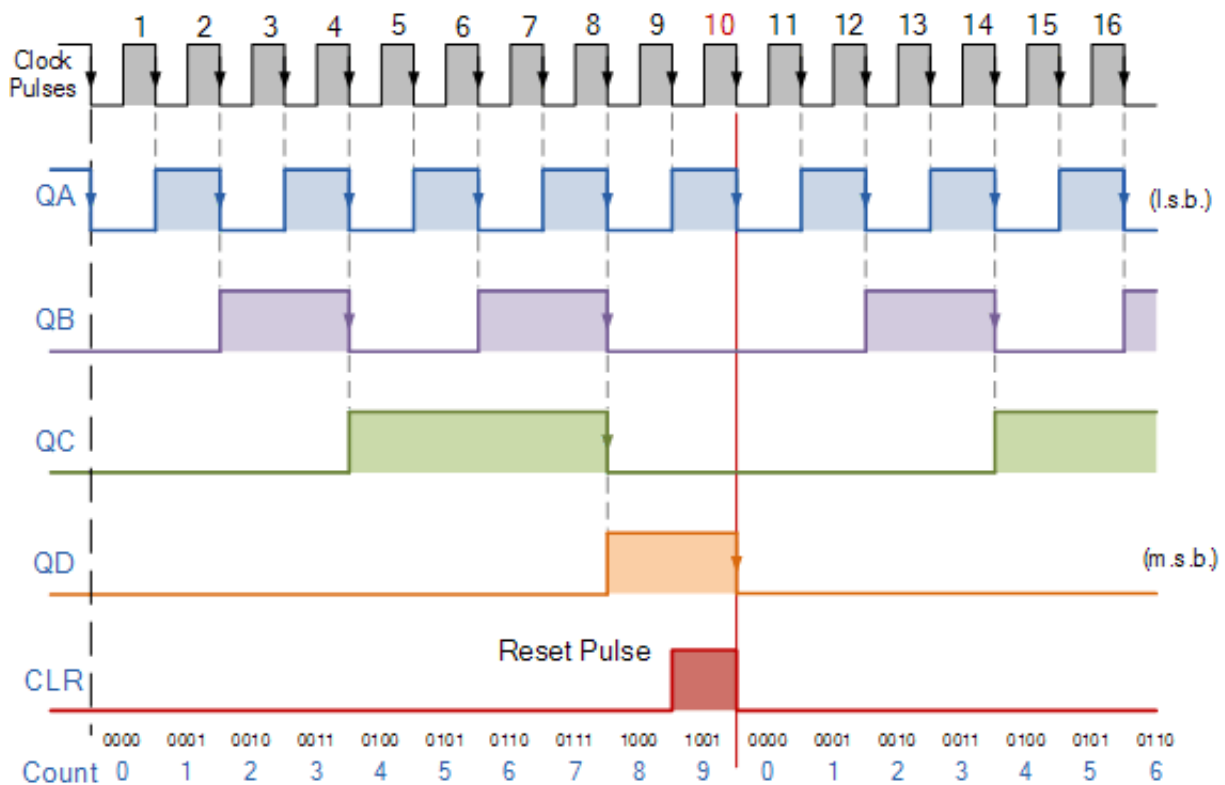


FIGURE 7.27 – Diagramme de synchronisation du compteur à décades

7.5.2 Décompteurs Asynchrones

Pour réaliser un décompteur le signal d'horloge est actif sur front montant. La Figure 7.28 montre un exemple de décompteur 3-bits, qui décompte de 7 à 0.

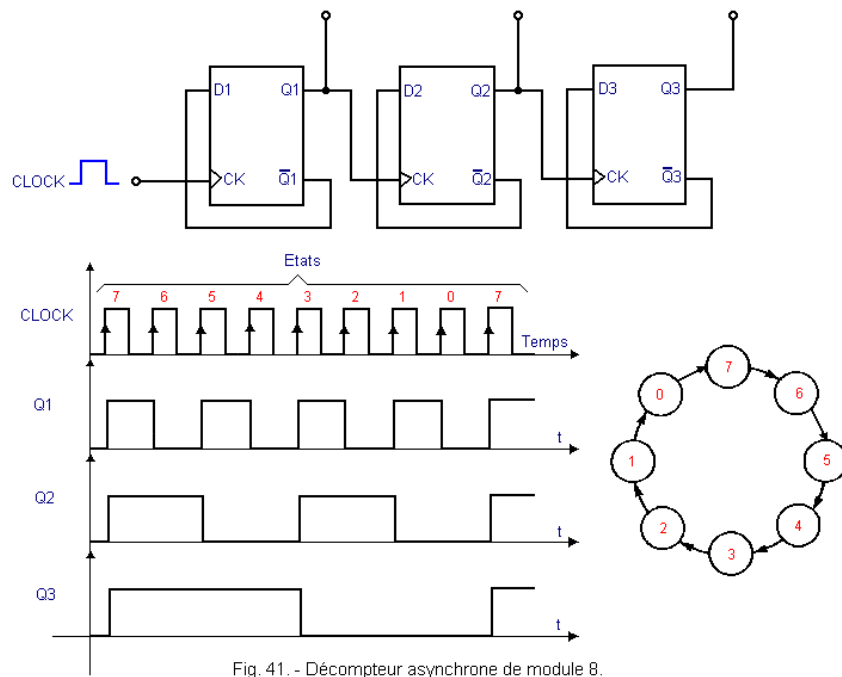


FIGURE 7.28 – Décompteur Asynchrone modulo 8 à base des bascules D

7.5.3 Limitations du comptage Asynchrones

Pourquoi ce type de compteur est-il qualifié d'asynchrone ou à Propagation ? Comme tout circuit logique, la bascule D ne réagit pas immédiatement à un changement sur son entrée d'horloge. Il existe un temps de transfert (*quelques nanosecondes*) entre Q_0 et Clk . Le même écart existe entre Q_1 et Q_0 , et entre Q_2 et Q_1 . Pour $n - bits$, le dernier signal est décalé de n fois cet écart élémentaire, ce qui peut devenir un décalage assez important.

Les signaux ne sont plus synchrones, et si une combinaison logique de ces signaux doit être réalisée, des glitches apparaissent (*transitions parasites 0-1-0 ou 1-0-1*). Souvent cela ne pose pas de problème car seule la dernière sortie est utilisée. Mais dans les autres cas l'utilisation d'un compteur synchrone est préférable.

7.5.4 Compteurs Synchrones

Dans un compteur synchrone, les bascules changent d'état simultanément, car elles sont pilotées par le même signal d'horloge. La façon la plus simple d'implémenter un compteur synchrone est d'utiliser des **bascules T** au lieu de bascules D. Une bascule T est très semblable à une bascule D. La différence est que l'entrée D (*Donnée*) est remplacée par une entrée T (*Toggle*) qui contrôle le changement d'état de la bascule. Si $T = 0$, la sortie garde sa valeur (*0 ou 1*) d'une période d'horloge à la suivante. Si $T = 1$, la bascule change d'état d'une période d'horloge à la suivante (*Elle passe à 1 si elle était à 0, ou à 0 si elle était à 1* "Basculément").

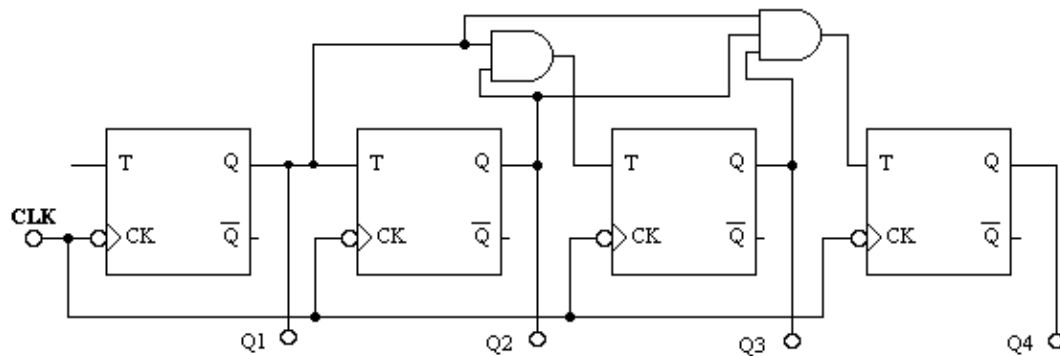


FIGURE 7.29 – Compteur synchrone modulo 8 à base des bascules T

Remarque : Les chronogrammes sont similaires à ceux des compteurs asynchrones. Notez que les signaux Set et Reset, omis pour raison de clarté, sont néanmoins nécessaires pour être au moins en mesure de remettre toutes les sorties à zéro à la mise sous tension.

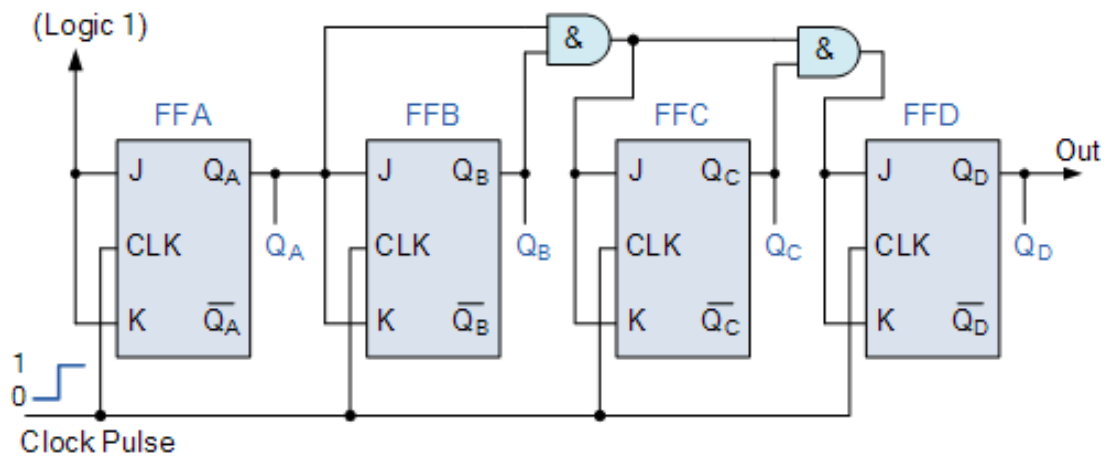


FIGURE 7.30 – Compteur synchrone Binaire 4-bits

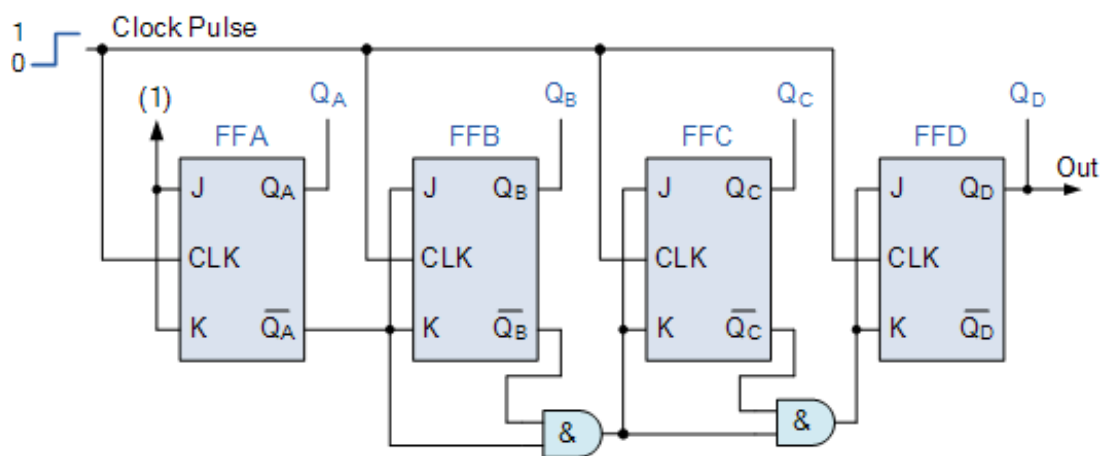


FIGURE 7.31 – Décompteur synchrone Binaire 4-bits

Chapitre 8

Applications

- 8.1 Génération du signal d'horloge (CLK) à base d'un circuit NE555
- 8.2 Conception et simulation d'un décodeur BCD/7-segment
- 8.3 Etude et Simulation d'une unité Arithmétique et logique (UAL 74LS181)
- 8.4 Conception, simulation et réalisation d'une Horloge Numérique
- 8.5 Étude et conception d'un dé électronique à base d'un compteur de Johnson
- 8.6 Étude et conception d'une PLL Numérique (*Phase Locked Loop*)

Voir polycopié de TP et de TD